CONTAINERISATION AND DEVOPS LAB

LAB RECORDS

SUBMITTED BY-

NAME – Ashmeet Negi
SAP - 500123864

SUBMITTED TO –

Dr. Prateek Raj Gautam

Lab-report-devops

LIST OF EXPERIMENTS

Experiment 1

Name: Ashmeet Negi

Course: Containerization and DevOps Lab

Lab 1: Virtual Machines vs Containers - A Comparative Study

Software and Hardware Requirements

Hardware

- 64-bit system with virtualization support enabled in BIOS
- Minimum 8 GB RAM (4 GB minimum acceptable)
- Internet Connection

Software (Windows Host)

- Oracle VirtualBox
- Vagrant
- Windows Subsystem for Linux (WSL 2)
- Ubuntu (WSL distribution)
- Docker Engine (docker.io)

Theory

Virtual Machine

A Virtual Machine emulates a complete physical computer, including its own operating system kernel, hardware drivers, and user space. Each VM runs on top of a hypervisor.

Characteristics:

- Full OS per VM
- Higher resource usage
- Strong isolation
- Slower startup time

Container

Containers virtualize at the operating system level. They share the host OS kernel while isolating applications and dependencies in user space.

Characteristics:

- Shared kernel
- Lightweight
- Fast startup
- Efficient resource usage

Part A: Install Vagrant and Automate VM

1. Install Oracle VM

2. Download Vagrant

3. Run Vagrant to Automate the VM

```
PS D:\container> vagrant -v
Vagrant 2.4.9
PS D:\container> vagrant init hashicorp/bionic64
A `Vagrantfile` has been placed in this directory. You are now
ready to `vagrant up` your first virtual environment! Please read
the comments in the Vagrantfile as well as documentation on
`vagrantup.com` for more information on using Vagrant.
PS D:\container> vagrant up
Bringing machine 'default' up with 'virtualbox' provider...
==> default: Box 'hashicorp/bionic64' could not be found. Attempting to find and install...
    default: Box Provider: virtualbox
    default: Box Version: >= 0
==> default: Loading metadata for box 'hashicorp/bionic64'
    default: URL: https://vagrantcloud.com/api/v2/vagrant/hashicorp/bionic64
==> default: Adding box 'hashicorp/bionic64' (v1.0.282) for provider: virtualbox
    default: Downloading: https://vagrantcloud.com/hashicorp/boxes/bionic64/versions/1.0.282/providers/virtualbox/unknow
n/vagrant.box
    default:
==> default: Successfully added box 'hashicorp/bionic64' (v1.0.282) for 'virtualbox'!
==> default: Importing base box 'hashicorp/bionic64'...
==> default: Matching MAC address for NAT networking...
==> default: Checking if box 'hashicorp/bionic64' version '1.0.282' is up to date...
==> default: Setting the name of the VM: container_default_1769226179638_63512
Vagrant is currently configured to create VirtualBox synced folders with
the `SharedFoldersEnableSymlinksCreate` option enabled. If the Vagrant
```

4. Run SSH into Vagrant

```
PS D:\container> vagrant ssh
Welcome to Ubuntu 18.04.3 LTS (GNU/Linux 4.15.0-58-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/advantage

  System information as of Sat Jan 24 03:52:11 UTC 2026

  System load:  0.0               Processes:           89
  Usage of /:   2.5% of 61.80GB   Users logged in:     0
  Memory usage: 11%               IP address for eth0: 10.0.2.15
  Swap usage:   0%


0 packages can be updated.
0 updates are security updates.
```

5. Perform Operations Inside VM

```
vagrant@vagrant:~$ df
Filesystem                  1K-blocks    Used Available Use% Mounted on
udev                           473252       0    473252   0% /dev
tmpfs                          100912    5056     95856   6% /run
/dev/mapper/vagrant--vg-root  64800356 1618284  59860628   3% /
tmpfs                          504556       0    504556   0% /dev/shm
tmpfs                            5120       0      5120   0% /run/lock
tmpfs                          504556       0    504556   0% /sys/fs/cgroup
vagrant                     337918972 9175036 328743936   3% /vagrant
tmpfs                          100908       0    100908   0% /run/user/1000
vagrant@vagrant:~$ free
              total        used        free      shared  buff/cache   available
Mem:        1009112       68064      456912        5056      484136      795592
Swap:       1003516           0     1003516
vagrant@vagrant:~$ exit
logout
```

6. Halt the Vagrant

```
PS D:\container> vagrant halt
==> default: Attempting graceful shutdown of VM...
```

7. Destroy Vagrant

```
PS D:\container> vagrant destroy
    default: Are you sure you want to destroy the 'default' VM? [y/N] y
==> default: Destroying VM and associated drives...
```

Experiment Setup – Part B: Containers using WSL (Windows)

Step 1: Install WSL 2

```
wsl --install
```

Reboot the system after installation.

Step 2: Install Ubuntu on WSL

```
wsl --install -d Ubuntu
```

Step 3: Install Docker Engine inside WSL

```
sudo apt update sudo apt
install -y docker.io sudo
systemctl start docker sudo
usermod -aG docker $USER
```

Logout and login again to apply group changes.

Docker Installation in WSL [INSERT SCREENSHOT: Docker installation process]

Docker Version Check

Step 4: Run Ubuntu Container with Nginx

```
docker pull ubuntu docker run -d -p 8080:80 --name

nginx-container nginx
```

Docker Pull and Run

Container Running Status

Step 5: Verify Nginx in Container

```
curl localhost:8080
```

Nginx Verification in Container

```
[INSERT SCREENSHOT: curl output showing Nginx welcome page from container]
```

Resource Utilization Observation

VM Observation Commands

```
free -h htop
systemd-analyze
```

VM Resource Usage

VM Boot Time Analysis

Container Observation Commands

```
docker stats
free -h
```

Container Resource Usage

Host System Resource Usage

Parameters to Compare and Observations

Parameter	Virtual Machine	Container
Boot Time	High	Very Low
RAM Usage	High	Low
CPU Overhead	Higher	Minimal
Disk Usage	Larger	Smaller

Isolation	Strong	Moderate

Result

The experiment demonstrates that containers are significantly more lightweight and resourceefficient compared to virtual machines, while virtual machines provide stronger isolation and full OS-level abstraction.

Virtual Machine:

- Resource overhead: High
- Isolation: Strong

Container:

- Resource overhead: Minimal
- Isolation: Good

Conclusion

Virtual Machines are suitable for full OS isolation and legacy workloads, whereas Containers are ideal for microservices, rapid deployment, and efficient resource utilization.

Experiment 2

Name: Ashmeet Negi

Course: Containerization and DevOps Lab

🐳 Lab 2 – Docker Installation, Configuration, and Running an Image

📖 Experiment 2

This experiment demonstrates how to pull a Docker image, run a container with port mapping, and perform container lifecycle operations.

🎯 Objective

- Pull an image from Docker Hub
- Run and manage a container
- Perform Docker lifecycle commands

🛠️ Prerequisite

Check Docker installation:

```
docker --version
```

🚀 Procedure

1️⃣ Pull Docker Image

```
docker pull nginx
```

📸 Output

2️⃣ Run Container with Port Mapping

```
docker run -d -p 8080:80 nginx
```

Open in browser:

http://localhost:8080

📸 Output

```
ashmeet@ashmeet:/mnt/c/Users/Arundun$ docker run -d -p 8080:80 ngi
ebc025eb75510454b9c9bec8d329e5cf212df6802e3bdc7a1c6504112489cc94
```

3️⃣ Verify Running Containers

```
docker ps
```

📸 Output

```
ashmeet@ashmeet:/mnt/c/Users/Arundun$ docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED         STATUS         PORTS
ebc025eb7551   nginx     "/docker-entrypoint..."  46 seconds ago  Up 40.00.0:8080->80/tcp, [::8080-
p   dreamy_mahavira
```

4️⃣ Stop the Container

```
docker stop <container_id>
```

📸 Output

```
ashmeet@ashmeet:/mnt/c/Users/Arundun$ docker ps -a
CONTAINER ID   IMAGE     COMMAND                  CREATED          STATUS                      PORTS
               NAMES
b8f59386fa9c   ubuntu    "/bin/bash"              44 seconds ago   Exited (0) 14 seconds ago
               mycontainer
ebc025eb7551   nginx     "/docker-entrypoint.…"   21 minutes ago   Up 21 minutes               0.0.0.0:8080->80/tcp
:]:8080->80/tcp   dreamy_mahavira
a0cf0fa3166e   nginx     "/docker-entrypoint.…"   9 hours ago      Exited (255) 22 minutago    0.0.0.0:8080->80/tcp
:]:8080->80/tcp   nginx-container
6a690ele2daf   busybox   "sh"                     2 days ago       Exited (255) 2 days ago
               container2
b217a45b07e4   nginx     "/docker-entrypoint.…"   2 days ago       Exited (255) 2 days ago     80/tcp
               container1
2a4009f51f94   mysql     "docker-entrypoint.s…"   8 days ago       Created
               mysql-container
ashmeet@ashmeet:/mnt/c/Users/Arundun$ docker stop mycontainer
mycontainer
ashmeet@ashmeet:/mnt/c/Users/Arundun$
```

5️⃣ Remove the Container

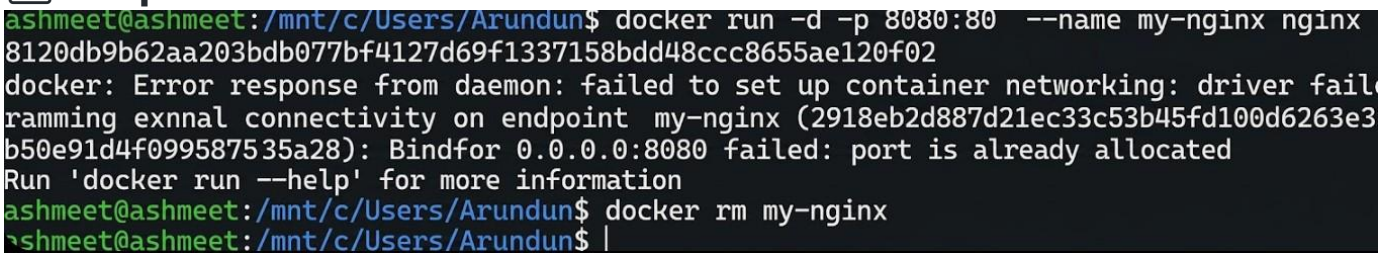
```
docker rm <container_id>
```

Output

6 Remove the Image

```
docker rmi nginx
```

Output



Result

Docker images were successfully pulled, containers were executed, and lifecycle commands were performed successfully.

Key Commands Summary

Command	Description
`docker --version`	Check Docker version
`docker pull nginx`	Download image
`docker run -d -p 8080:80 nginx`	Run container
`docker ps`	List running containers
`docker stop <id>`	Stop container
`docker rm <id>`	Remove container
`docker rmi nginx`	Remove image

Experiment 3

Name: Ashmeet Negi

Course: Containerization and DevOps Lab

🐳 Lab 3 – Deploy NGINX Using Different Base Images & Compare Image Layers

📖 Experiment 3

This experiment demonstrates how to deploy **NGINX** using Official, Ubuntu, and Alpine images and compare their size, layers, and performance.

🎯 Objective

- Deploy NGINX using different base images
- Build custom Docker images
- Compare image layers and size
- Perform functional tasks using NGINX

🛠 Prerequisites

Docker installed and running.

```
docker --version
```

🚀 Part 1: Deploy NGINX Using Official Image

Pull Image

```
docker pull nginx:latest
```

```
PS C:\Users\Arundun> wsl
ashmeet@ashmeet:/mnt/c/Users/Arundun$ docker pull nginx:latest
latest: Pulling from library/nginx
Digest: sha256:341bf0f3ce6c5277d6002cf6e1fb0319fa4252add24ab6a0e262e0056d313208
Status: Image is up to date for nginx:latest
docker.io/library/nginx:latest
```

Run Container

```
docker run -d --name nginx-official -p 8080:80 nginx
```

```
ashmeet@ashmeet:/mnt/c/Users/Arundun$ docker run -d --name nginx-official -p 8080:80 nginx
51816bd7e7606e97c103675a427e86a43dcc60955e903a57cf31f62e0aa9f198
```

Verify

```
curl http://localhost:8080
```

```
ashmeet@ashmeet:/mnt/c/Users/Arundun$ curl http://localhost:8080
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>

<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
Commercial support is available at
```

Image Check

```
docker images nginx
```

```
ashmeet@ashmeet:/mnt/c/Users/Arundun$ docker images nginx
REPOSITORY   TAG       IMAGE ID       CREATED        SIZE
nginx        latest    341bf0f3ce6c   2 weeks ago    240MB
```

Part 2: Custom NGINX Using Ubuntu Base Image

Build Image

```
docker build -t nginx-ubuntu .
```

dockerfile

C: > Users > Arundun > Desktop > docker > dockerfile

```
1   FROM ubuntu:22.04
2
3   RUN apt-get update && \
4       apt-get install -y nginx && \
5       apt-get clean && \
6       rm -rf /var/lib/apt/lists/*
7
8   EXPOSE 80
9
10  CMD ["nginx", "-g", "daemon off;"]
```

Run Container

```
docker run -d --name nginx-ubuntu -p 8081:80 nginx-ubuntu
```

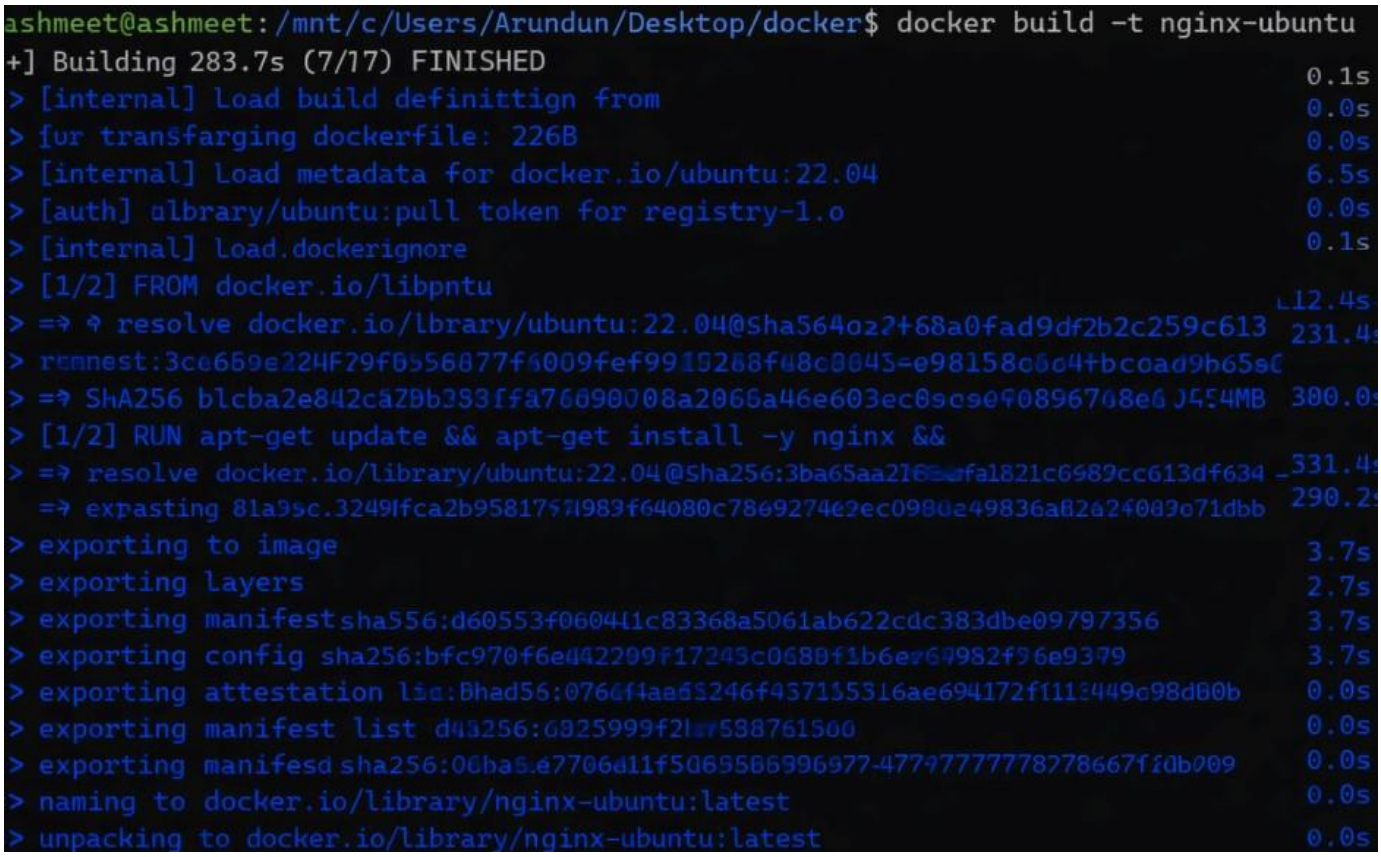


Image Size

```
docker images nginx-ubuntu
```

🏔 Part 3: Custom NGINX Using Alpine Base Image

Build Image

```
docker build -t nginx-alpine .
```

Run Container

```
docker run -d --name nginx-alpine -p 8082:80 nginx-alpine
```

new > dockerfile

```
1  FROM alpine:latest
2
3  RUN apk add --no-cache nginx
4
5  EXPOSE 80
6
7  CMD ["nginx", "-g", "daemon off;"]
```

Image Size

```
docker images nginx-alpine
```

```
ashmeet@ashmeet:/mnt/c/Users/Arundun/Desktop/docker$ docker build -t nginx-alpine .
[+] Building 6.1s (7/7) FINISHED
 => [internal] load build definition from dockerfile
 => => transferring dockerfile: 226B
 => [internal] load metadata for docker.io/library/ubuntu:22.04
 => [auth] library/ubuntu:pull token for registry-1.docker.io
 => [internal] load .dockerignore
 => => transferring context: 2B
 => [1/2] FROM docker.io/library/ubuntu:22.04@sha256:3ba65aa20f86a0fad9df2b2c259c613df006b2e6d0bfcc8a1
 => => resolve docker.io/library/ubuntu:22.04@sha256:3ba65aa20f86a0fad9df2b2c259c613df006b2e6d0bfcc8a1
 => CACHED [2/2] RUN apt-get update &&     apt-get install -y nginx &&     apt-get clean &&     rm -rf
 => exporting to image
 => => exporting layers
 => => exporting manifest sha256:6d0555f600461c33368ae50961ab6e23cdc383dbe0978783686b0e36a3c97347
 => => exporting config sha256:bfc970f6e4020fc1fa49ff9465f91533cd0c89c5fa2f0abf1bed5280fcb89cb9
 => => exporting attestation manifest sha256:974594a387191fdb3b52e58d7ea3fb52f471e5d6f485d25e7faf6ccc2
 => => exporting manifest list sha256:e56d6c3ab8e2b65695b8eec921ef3cbb2e4668adafce369e88bb9bce40d1568c
 => => naming to docker.io/library/nginx-alpine:latest
 => => unpacking to docker.io/library/nginx-alpine:latest
ashmeet@ashmeet:/mnt/c/Users/Arundun/Desktop/docker$
```

📊 Part 4: Image Size & Layer Comparison

Compare Images

```
docker images | grep nginx
```

```
ashmeet@ashmeet:/mnt/c/Users/Arundun/Desktop/docker$
docker run -d -name nginx-alpine -p 8082:80 nginx-alpine
fbd78c92bba2931f392a71aeffa651014a21d688d6adc7de7de14375L94f267a
ashmeet@ashmeet:/mnt/c/Users/Arundun/Desktop/docker$
```

Inspect Layers

```
docker history nginx docker
history nginx-ubuntu docker
history nginx-alpine
```

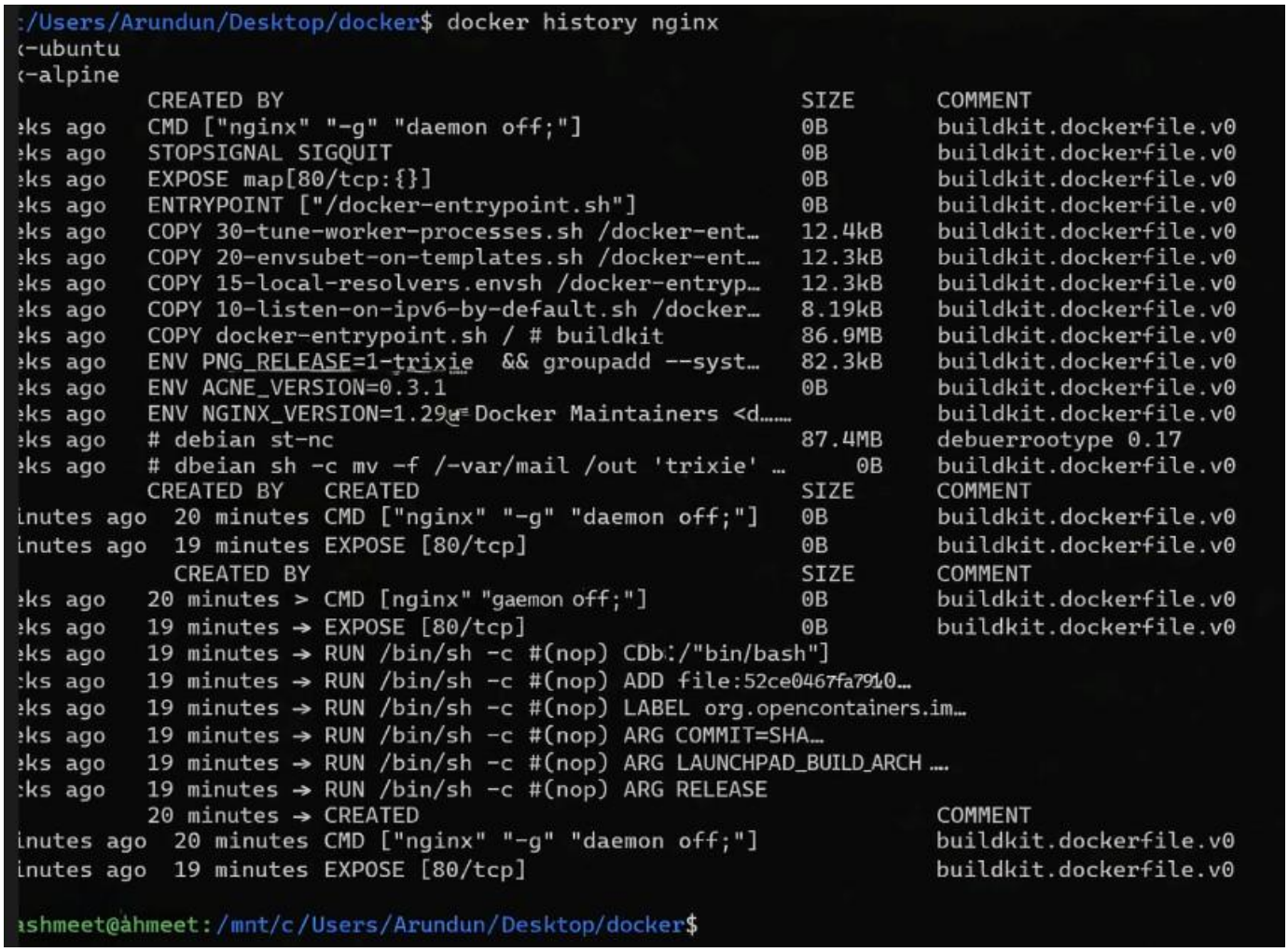
```
ashmeet@ashmeet:/mnt/c/Users/Arundun/Desktop/
docker$ docker images nginx-alpine
REPOSITORY     TAG      IMAGE ID        CREATED
nginx-alpine   latest   e56d6c3ab8e2    18 minutes ago 199MB
ashmeet@ashmeet:/mnt/c/Users/Arundun/Desktop/docker$
```

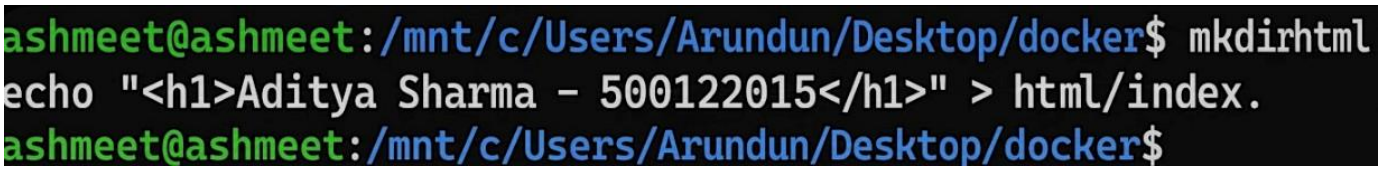
```
ashmeet@ashmeet:/mnt/c/Users/Arundun/Desktop/docker$ docker images | grep nginx
nginx-alpine     latest         e56d6ccc3ab8e2   18 minutes ago 199MB
nginx-ubuntu     latest         c855599b0ef3d    18 minutes ago 199MB
```

🌐 Part 5: Serve Custom HTML Page

```
mkdir html echo "<h1>Aditya Sharma - 500122015</h1>" >
html/index.html
```

```
docker run -d -p 8083:80 -v $(pwd)/html:/usr/share/nginx/html nginx
```





Reverse Proxy (Concept)

NGINX can:

- Act as reverse proxy
- Load balance containers
- Terminate SSL
- Serve static content

📋 Image Comparison Summary

Feature	Official NGINX	Ubuntu + NGINX	Alpine + NGINX
Image Size	Medium	Large	Very Small
Startup Time	Fast	Slow	Very Fast
Production Ready	Yes	Rarely	Yes

🏁 Conclusion

- Alpine → Smallest & fastest
- Ubuntu → Flexible but large
- Official → Best for production

Experimenr 4

Name: Ashmeet Negi

Course: Containerization and DevOps Lab

🧪 Experiment 4

Docker Essentials: Dockerfile, .dockerignore, Tagging & Publishing

🎯 Aim

To understand and implement Dockerfile creation, use of .dockerignore, image tagging, and publishing Docker images to Docker Hub.

📓 Theory

Docker is a containerization platform that allows applications to run in isolated environments called containers.
A Docker image is built using a Dockerfile, which contains step-by-step instructions.

Key Components:

- **Dockerfile** – Defines how an image is built.
- **.dockerignore** – Excludes unnecessary files from the build context.
- **Tagging** – Assigns a name and version to Docker images.
- **Publishing** – Uploading images to Docker Hub or a registry.

🛠 Requirements

- Docker installed (Docker Desktop / Docker
- Engine)
- Internet connection

Docker Hub account

🗁 Project Structure

```
project-folder/
│
├── Dockerfile
├── .dockerignore
├── app.py
└── README.md
```

Step 1: Create Dockerfile

Create a file named `Dockerfile` :

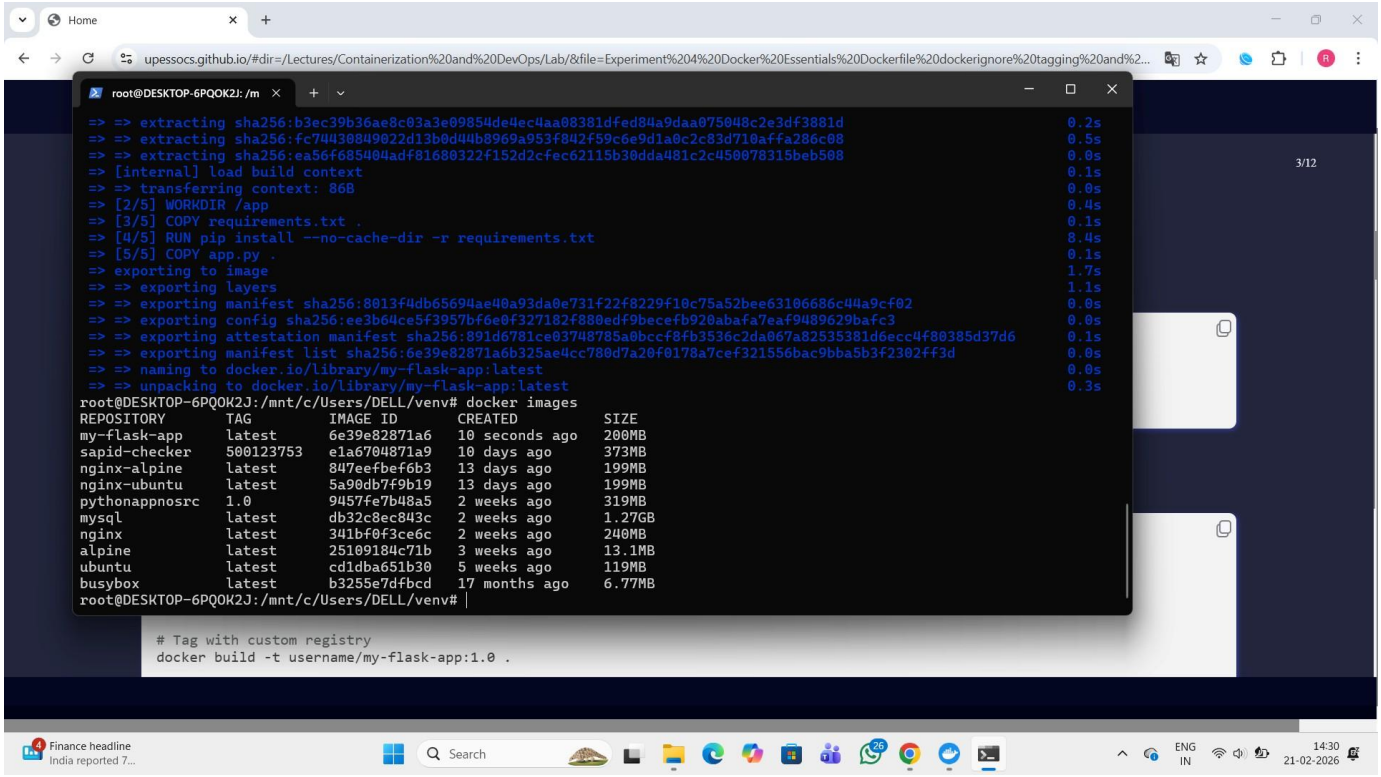
```
FROM ubuntu:latest

WORKDIR /app

COPY . .

RUN apt-get update && apt-get install -y python3

CMD ["bash"]
```



Explanation of Instructions

- `FROM` → Specifies base image
- `WORKDIR` → Sets working directory
- `COPY` → Copies files into container
- `RUN` → Executes commands during build
- `CMD` → Default command when container runs

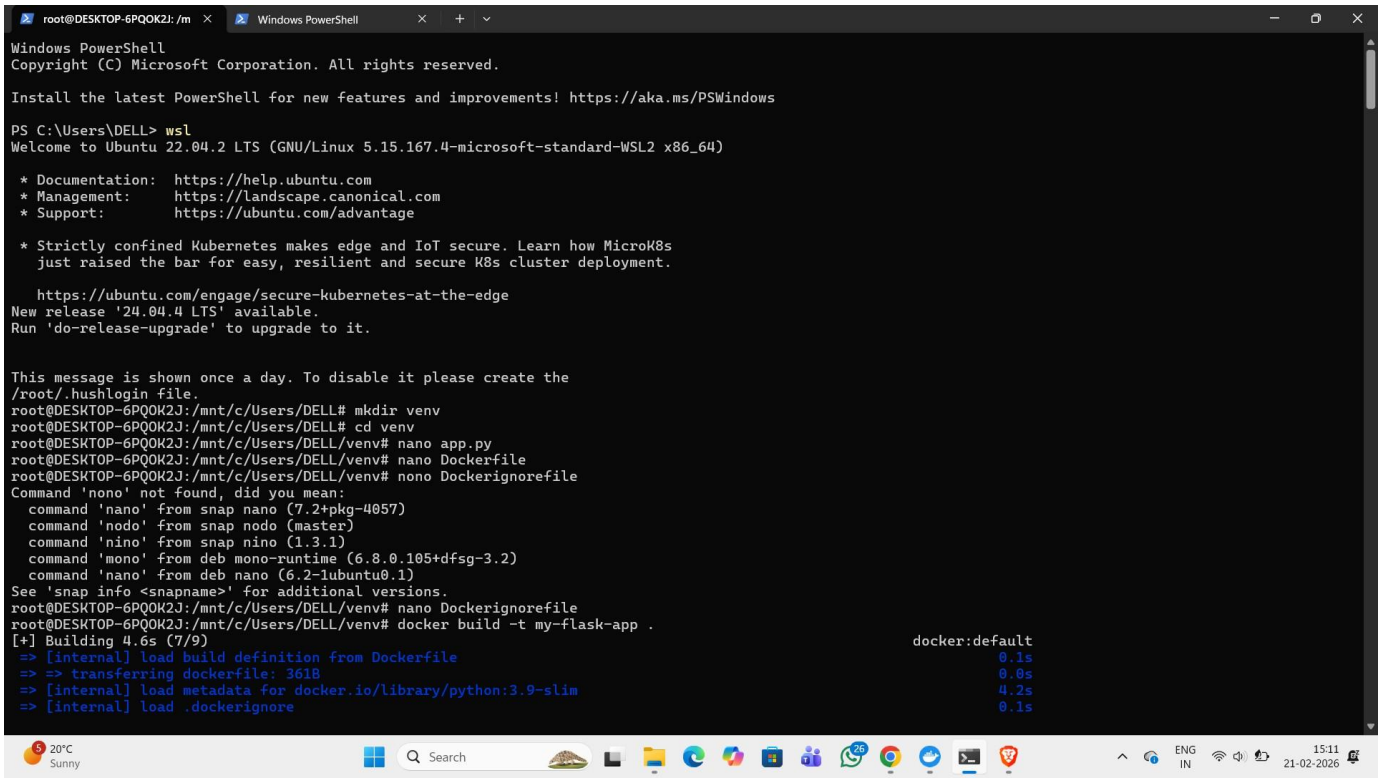
🚫 Step 2: Create .dockerignore

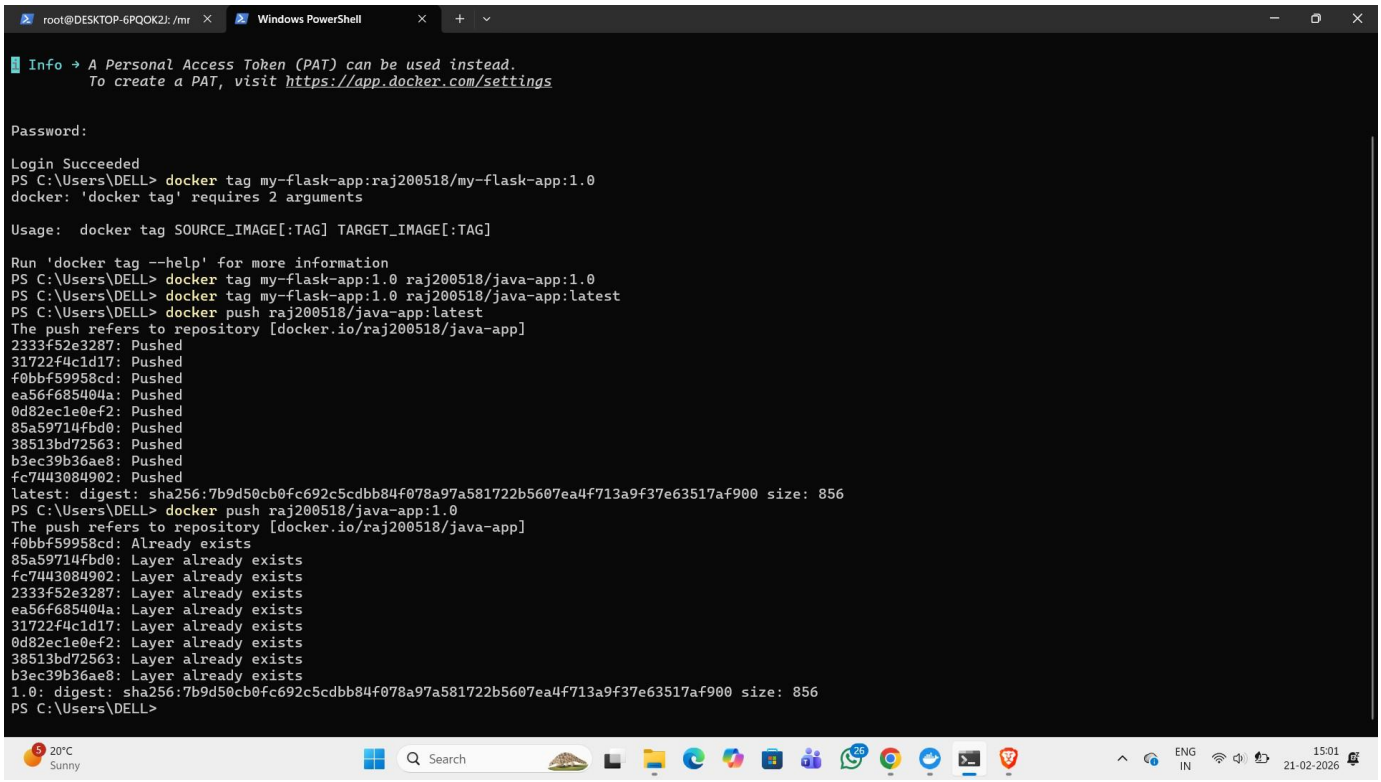
Create a file named `.dockerignore` :

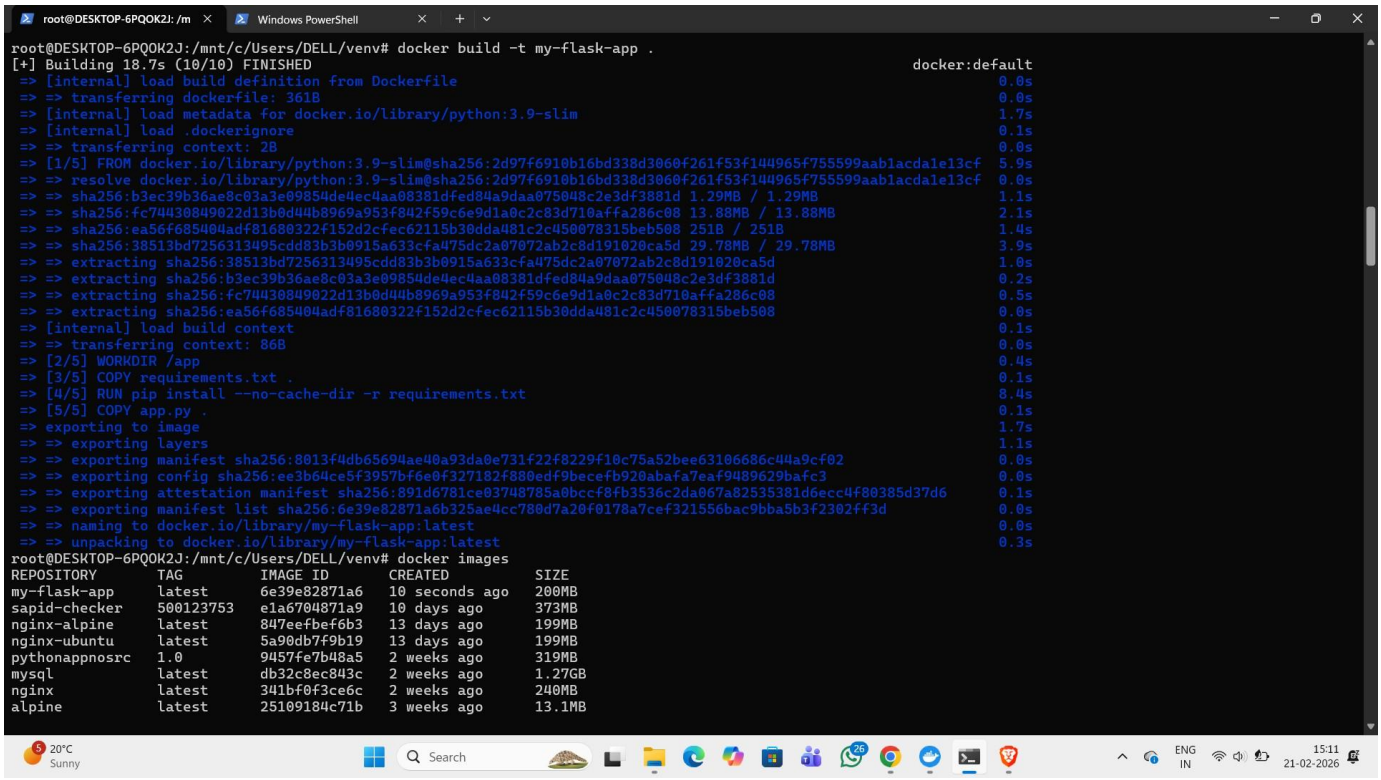
```
.git
node_modules
*.log
__pycache__
Dockerfile
```

Purpose of .dockerignore

- Reduces build size
- Improves performance
- Prevents unnecessary files from entering image


root@DESKTOP-6PQOK2J: /m
Windows PowerShell
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.
Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows
PS C:\Users\DELL> wsl
Welcome to Ubuntu 22.04.2 LTS (GNU/Linux 5.15.167.4-microsoft-standard-WSL2 x86_64)
* Documentation: https://help.ubuntu.com
* Management: https://landscape.canonical.com
* Support: https://ubuntu.com/advantage
* Strictly confined Kubernetes makes edge and IoT secure. Learn how MicroK8s
just raised the bar for easy, resilient and secure K8s cluster deployment.
https://ubuntu.com/engage/secure-kubernetes-at-the-edge
New release '24.04.4 LTS' available.
Run 'do-release-upgrade' to upgrade to it.
This message is shown once a day. To disable it please create the
/root/.hushlogin file.
root@DESKTOP-6PQOK2J:/mnt/c/Users/DELL# mkdir venv
root@DESKTOP-6PQOK2J:/mnt/c/Users/DELL# cd venv
root@DESKTOP-6PQOK2J:/mnt/c/Users/DELL/venv# nano app.py
root@DESKTOP-6PQOK2J:/mnt/c/Users/DELL/venv# nano Dockerfile
root@DESKTOP-6PQOK2J:/mnt/c/Users/DELL/venv# nono Dockerignorefile
Command 'nono' not found, did you mean:
command 'nano' from snap nano (7.2+pkg-4057)
command 'nodo' from snap nodo (master)
command 'nino' from snap nino (1.3.1)
command 'mono' from deb mono-runtime (6.8.0.105+dfsg-3.2)
command 'nano' from deb nano (6.2-1ubuntu0.1)
See 'snap info <snapname>' for additional versions.
root@DESKTOP-6PQOK2J:/mnt/c/Users/DELL/venv# nano Dockerignorefile
root@DESKTOP-6PQOK2J:/mnt/c/Users/DELL/venv# docker build -t my-flask-app .
[+] Building 4.6s (7/9) docker:default
=> [internal] load build definition from Dockerfile 0.1s
=> => transferring dockerfile: 361B 0.0s
=> [internal] load metadata for docker.io/library/python:3.9-slim 4.2s
=> [internal] load .dockerignore 0.1s
20°C Sunny
Search
ENG IN
15:11 21-02-2026


root@DESKTOP-6PQOK2J: /mr
Windows PowerShell
Info → A Personal Access Token (PAT) can be used instead.
To create a PAT, visit https://app.docker.com/settings
Password:
Login Succeeded
PS C:\Users\DELL> docker tag my-flask-app:raj200518/my-flask-app:1.0
docker: 'docker tag' requires 2 arguments
Usage: docker tag SOURCE_IMAGE[:TAG] TARGET_IMAGE[:TAG]
Run 'docker tag --help' for more information
PS C:\Users\DELL> docker tag my-flask-app:1.0 raj200518/java-app:1.0
PS C:\Users\DELL> docker tag my-flask-app:1.0 raj200518/java-app:latest
PS C:\Users\DELL> docker push raj200518/java-app:latest
The push refers to repository [docker.io/raj200518/java-app]
2333f52e3287: Pushed
31722f4c1d17: Pushed
f0bbf59958cd: Pushed
ea56f685404a: Pushed
0d82ec1e0ef2: Pushed
85a59714fbd0: Pushed
38513bd72563: Pushed
b3ec39b36ae8: Pushed
fc7443084902: Pushed
latest: digest: sha256:7b9d50cb0fc692c5cdbb84f078a97a581722b5607ea4f713a9f37e63517af900 size: 856
PS C:\Users\DELL> docker push raj200518/java-app:1.0
The push refers to repository [docker.io/raj200518/java-app]
f0bbf59958cd: Already exists
85a59714fbd0: Layer already exists
fc7443084902: Layer already exists
2333f52e3287: Layer already exists
ea56f685404a: Layer already exists
31722f4c1d17: Layer already exists
0d82ec1e0ef2: Layer already exists
38513bd72563: Layer already exists
b3ec39b36ae8: Layer already exists
1.0: digest: sha256:7b9d50cb0fc692c5cdbb84f078a97a581722b5607ea4f713a9f37e63517af900 size: 856
PS C:\Users\DELL>
20°C Sunny
Search
ENG IN
15:01 21-02-2026



🏗️ Step 3: Build Docker Image

Build the image using: `docker`

```
build -t myapp:1.0 .
```

Check available images:

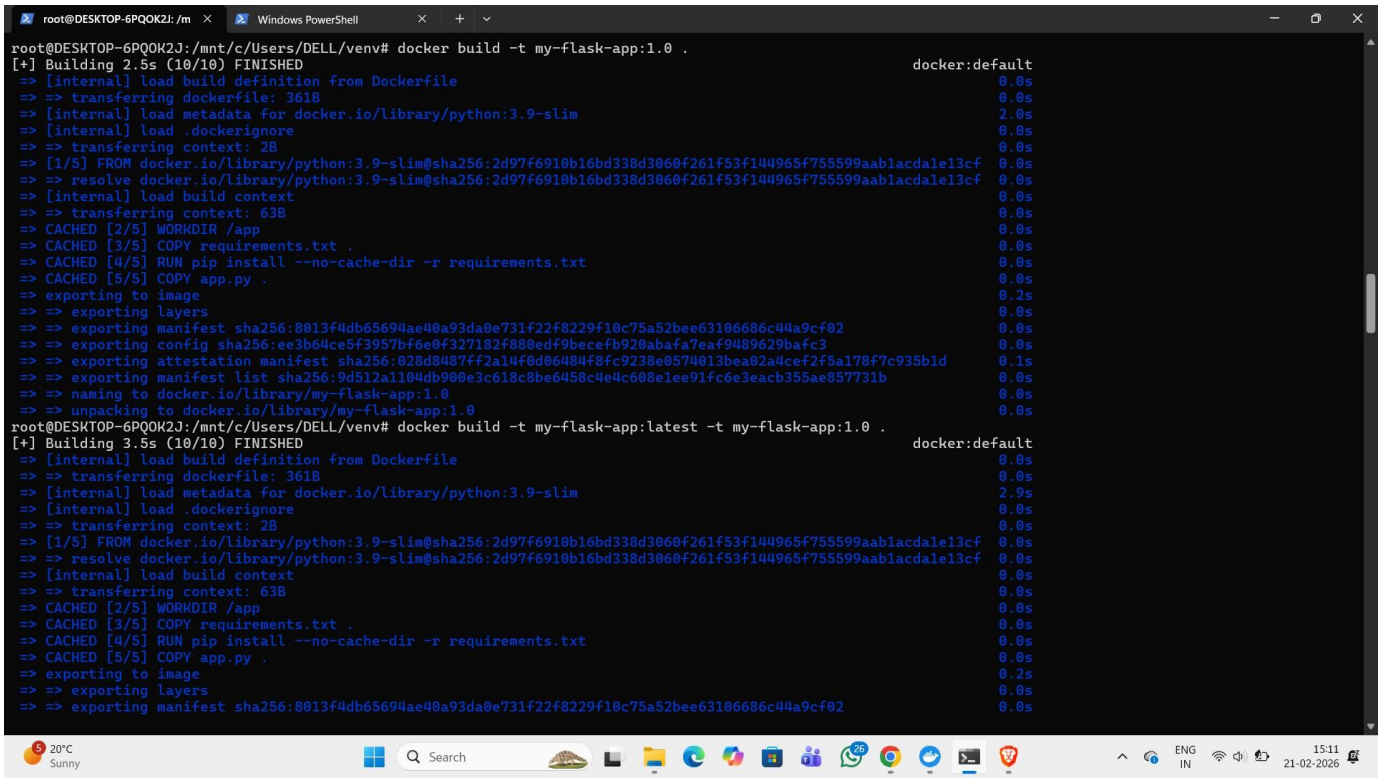
```
docker images
```

🏷️ Step 4: Tag the Image

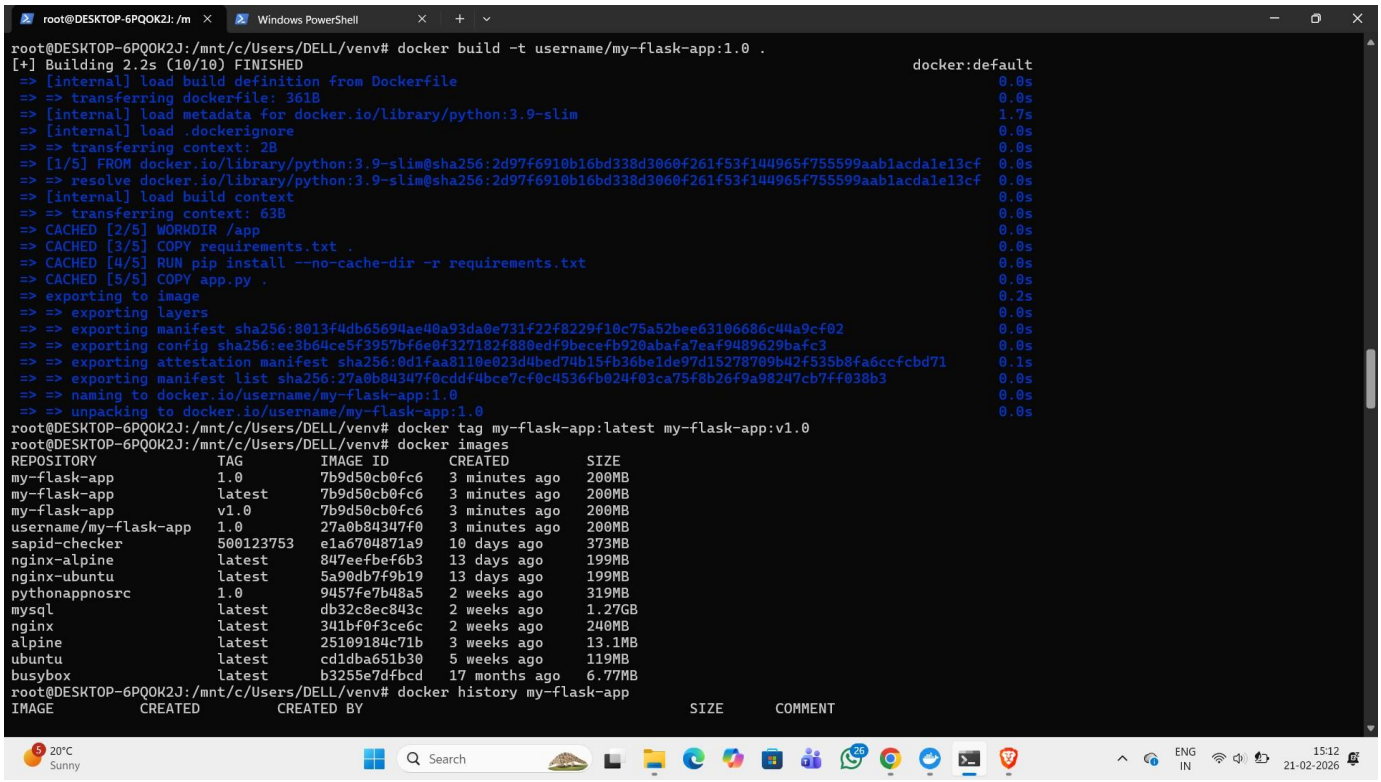
Tag the image for Docker Hub: `docker tag`

`myapp:1.0 username/myapp:1.0` Example: `docker`

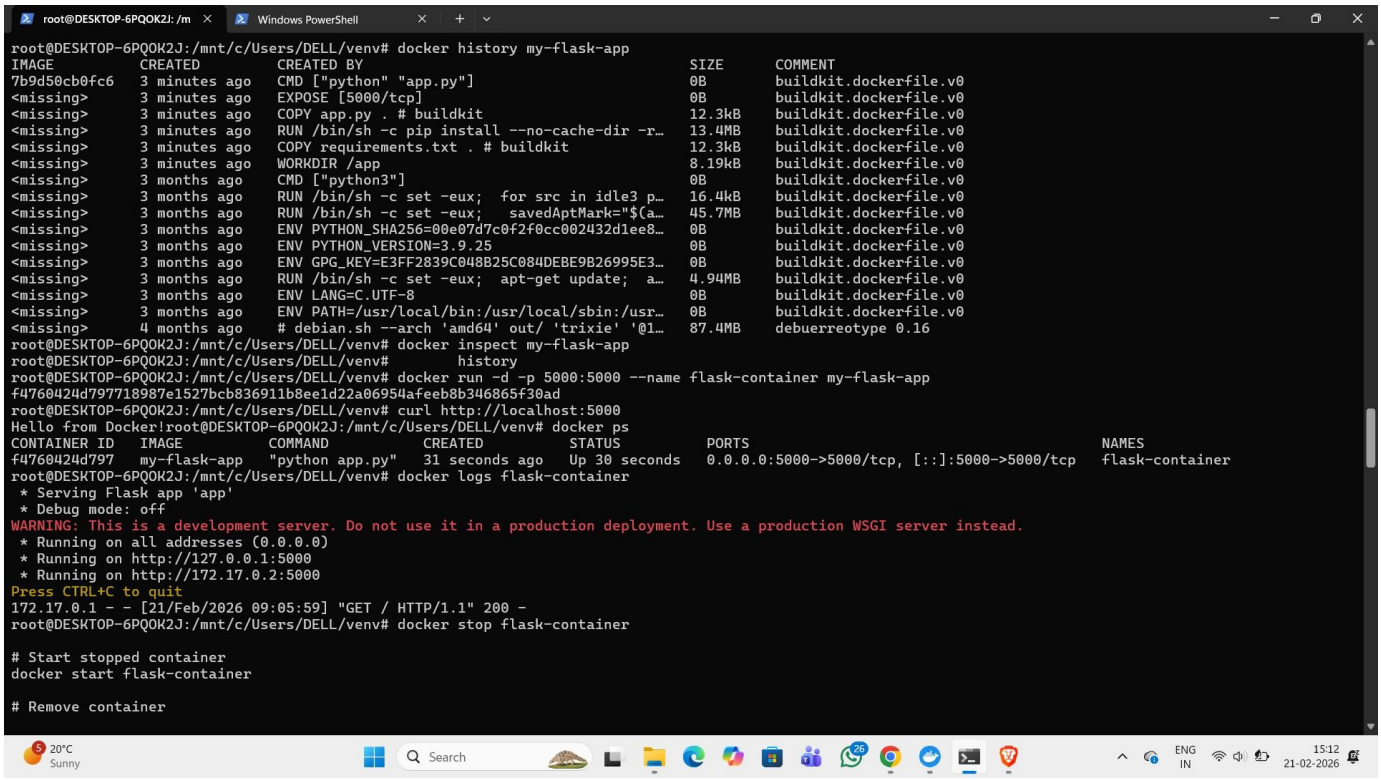
`tag myapp:1.0 rajvardhan/myapp:1.0`

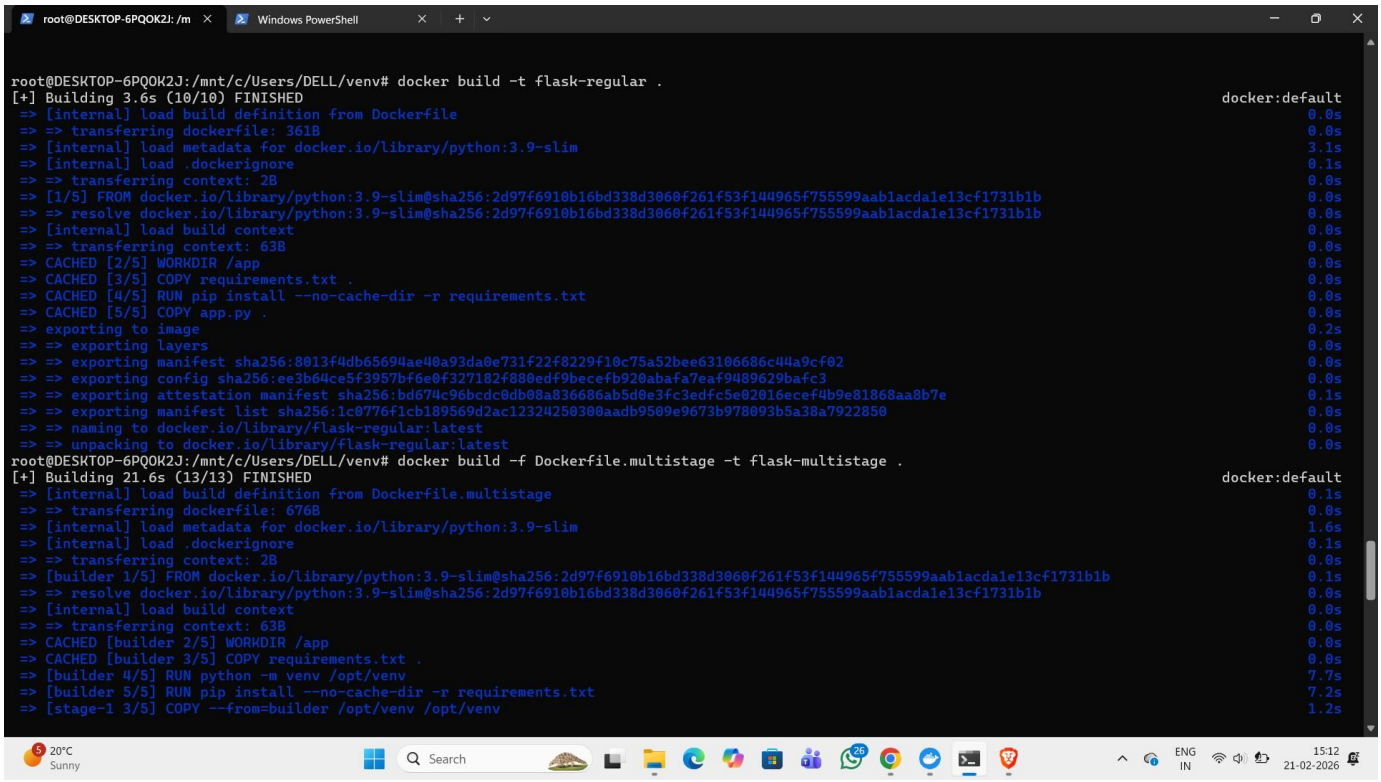


```
root@DESKTOP-6PQOK2J:/mnt/c/Users/DELL/venv# docker build -t my-flask-app:1.0 .
[+] Building 2.5s (10/10) FINISHED                                                                                  docker:default
 => [internal] load build definition from Dockerfile                                                                         0.0s
 => => transferring dockerfile: 361B                                                                                         0.0s
 => [internal] load metadata for docker.io/library/python:3.9-slim                                                           2.0s
 => [internal] load .dockerignore                                                                                            0.0s
 => => transferring context: 2B                                                                                              0.0s
 => [1/5] FROM docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd338d3060f261f53f144965f755599aab1acda1e13cf            0.0s
 => => resolve docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd338d3060f261f53f144965f755599aab1acda1e13cf            0.0s
 => [internal] load build context                                                                                            0.0s
 => => transferring context: 63B                                                                                             0.0s
 => CACHED [2/5] WORKDIR /app                                                                                                0.0s
 => CACHED [3/5] COPY requirements.txt .                                                                                     0.0s
 => CACHED [4/5] RUN pip install --no-cache-dir -r requirements.txt                                                          0.0s
 => CACHED [5/5] COPY app.py .                                                                                               0.0s
 => exporting to image                                                                                                       0.2s
 => => exporting layers                                                                                                      0.0s
 => => exporting manifest sha256:8013f4db65694ae40a93da0e731f22f8229f10c75a52bee63106686c44a9cf02                            0.0s
 => => exporting config sha256:ee3b64ce5f3957bf6e0f327182f880edf9becefb920abafa7eaf9489629bafc3                              0.0s
 => => exporting attestation manifest sha256:028d8487ff2a14f0d06484f8fc9238e0574013bea02a4cef2f5a178f7c935b1d                0.1s
 => => exporting manifest list sha256:9d512a1104db900e3c618c8be6458c4e4c608e1ee91fc6e3eacb355ae857731b                       0.0s
 => => naming to docker.io/library/my-flask-app:1.0                                                                          0.0s
 => => unpacking to docker.io/library/my-flask-app:1.0                                                                       0.0s
root@DESKTOP-6PQOK2J:/mnt/c/Users/DELL/venv# docker build -t my-flask-app:latest -t my-flask-app:1.0 .
[+] Building 3.5s (10/10) FINISHED                                                                                  docker:default
 => [internal] load build definition from Dockerfile                                                                         0.0s
 => => transferring dockerfile: 361B                                                                                         0.0s
 => [internal] load metadata for docker.io/library/python:3.9-slim                                                           2.9s
 => [internal] load .dockerignore                                                                                            0.0s
 => => transferring context: 2B                                                                                              0.0s
 => [1/5] FROM docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd338d3060f261f53f144965f755599aab1acda1e13cf            0.0s
 => => resolve docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd338d3060f261f53f144965f755599aab1acda1e13cf            0.0s
 => [internal] load build context                                                                                            0.0s
 => => transferring context: 63B                                                                                             0.0s
 => CACHED [2/5] WORKDIR /app                                                                                                0.0s
 => CACHED [3/5] COPY requirements.txt .                                                                                     0.0s
 => CACHED [4/5] RUN pip install --no-cache-dir -r requirements.txt                                                          0.0s
 => CACHED [5/5] COPY app.py .                                                                                               0.0s
 => exporting to image                                                                                                       0.2s
 => => exporting layers                                                                                                      0.0s
 => => exporting manifest sha256:8013f4db65694ae40a93da0e731f22f8229f10c75a52bee63106686c44a9cf02                            0.0s
```



```
root@DESKTOP-6PQOK2J:/mnt/c/Users/DELL/venv# docker build -t username/my-flask-app:1.0 .
[+] Building 2.2s (10/10) FINISHED                                                                                  docker:default
 => [internal] load build definition from Dockerfile                                                                         0.0s
 => => transferring dockerfile: 361B                                                                                         0.0s
 => [internal] load metadata for docker.io/library/python:3.9-slim                                                           1.7s
 => [internal] load .dockerignore                                                                                            0.0s
 => => transferring context: 2B                                                                                              0.0s
 => [1/5] FROM docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd338d3060f261f53f144965f755599aab1acda1e13cf            0.0s
 => => resolve docker.io/library/python:3.9-slim@sha256:2d97f6910b16bd338d3060f261f53f144965f755599aab1acda1e13cf            0.0s
 => [internal] load build context                                                                                            0.0s
 => => transferring context: 63B                                                                                             0.0s
 => CACHED [2/5] WORKDIR /app                                                                                                0.0s
 => CACHED [3/5] COPY requirements.txt .                                                                                     0.0s
 => CACHED [4/5] RUN pip install --no-cache-dir -r requirements.txt                                                          0.0s
 => CACHED [5/5] COPY app.py .                                                                                               0.0s
 => exporting to image                                                                                                       0.2s
 => => exporting layers                                                                                                      0.0s
 => => exporting manifest sha256:8013f4db65694ae40a93da0e731f22f8229f10c75a52bee63106686c44a9cf02                            0.0s
 => => exporting config sha256:ee3b64ce5f3957bf6e0f327182f880edf9becefb920abafa7eaf9489629bafc3                              0.0s
 => => exporting attestation manifest sha256:0d1faa8110e023d4bed74b15fb36be1de97d15278709b42f535b8fa6ccfcbd71                0.1s
 => => exporting manifest list sha256:27a0b84347f0cddf4bce7cf0c4536fb024f03ca75f8b26f9a98247cb7ff038b3                       0.0s
 => => naming to docker.io/username/my-flask-app:1.0                                                                         0.0s
 => => unpacking to docker.io/username/my-flask-app:1.0                                                                      0.0s
root@DESKTOP-6PQOK2J:/mnt/c/Users/DELL/venv# docker tag my-flask-app:latest my-flask-app:v1.0
root@DESKTOP-6PQOK2J:/mnt/c/Users/DELL/venv# docker images
REPOSITORY              TAG         IMAGE ID       CREATED         SIZE
my-flask-app            1.0         7b9d50cb0fc6   3 minutes ago   200MB
my-flask-app            latest      7b9d50cb0fc6   3 minutes ago   200MB
my-flask-app            v1.0        7b9d50cb0fc6   3 minutes ago   200MB
username/my-flask-app   1.0         27a0b84347f0   3 minutes ago   200MB
sapid-checker           500123753   e1a6704871a9   10 days ago     373MB
nginx-alpine            latest      847eefbef6b3   13 days ago     199MB
nginx-ubuntu            latest      5a90db7f9b19   13 days ago     199MB
pythonappnosrc          1.0         9457fe7b48a5   2 weeks ago     319MB
mysql                   latest      db32c8ec843c   2 weeks ago     1.27GB
nginx                   latest      341bf0f3ce6c   2 weeks ago     240MB
alpine                  latest      25109184c71b   3 weeks ago     13.1MB
ubuntu                  latest      cd1dba651b30   5 weeks ago     119MB
busybox                 latest      b3255e7dfbcd   17 months ago   6.77MB
root@DESKTOP-6PQOK2J:/mnt/c/Users/DELL/venv# docker history my-flask-app
IMAGE          CREATED         CREATED BY                                      SIZE      COMMENT
```





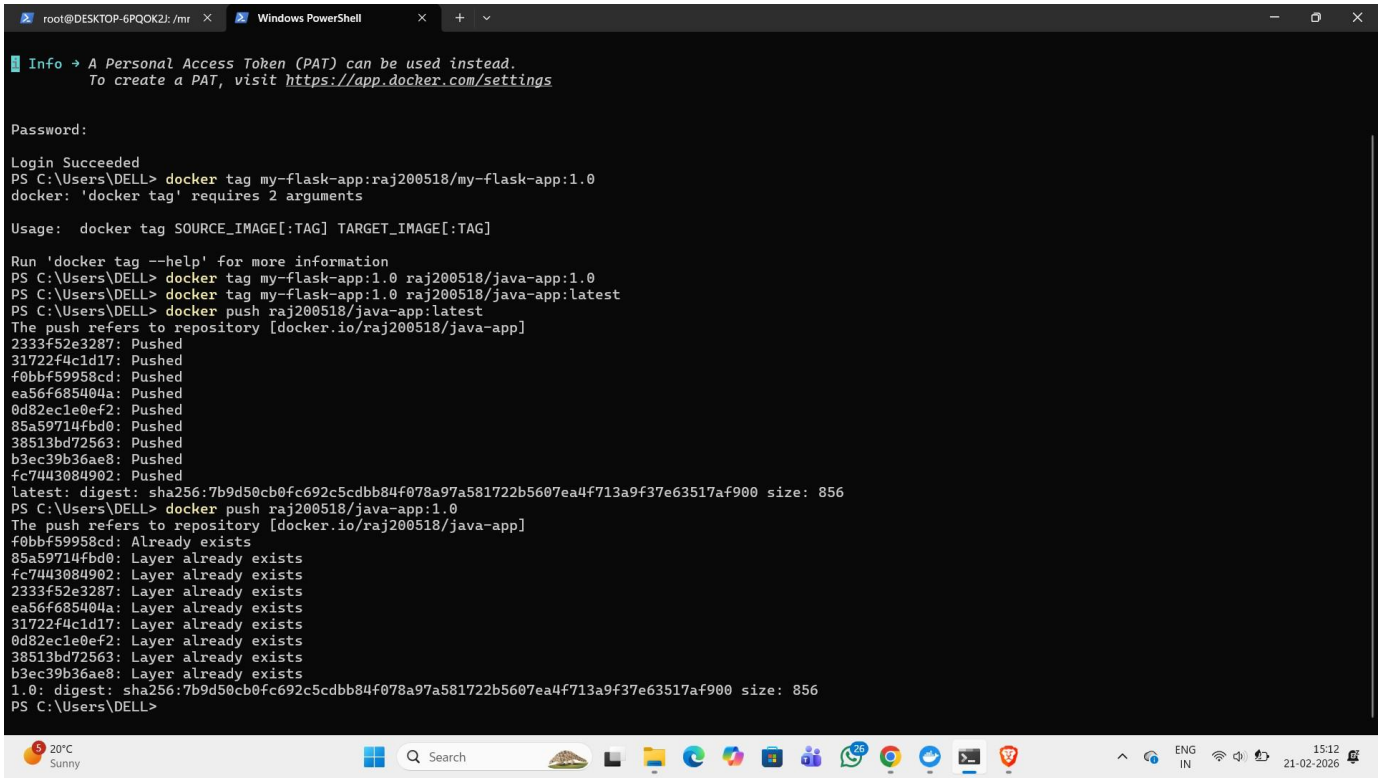
Step 5: Login to Docker Hub

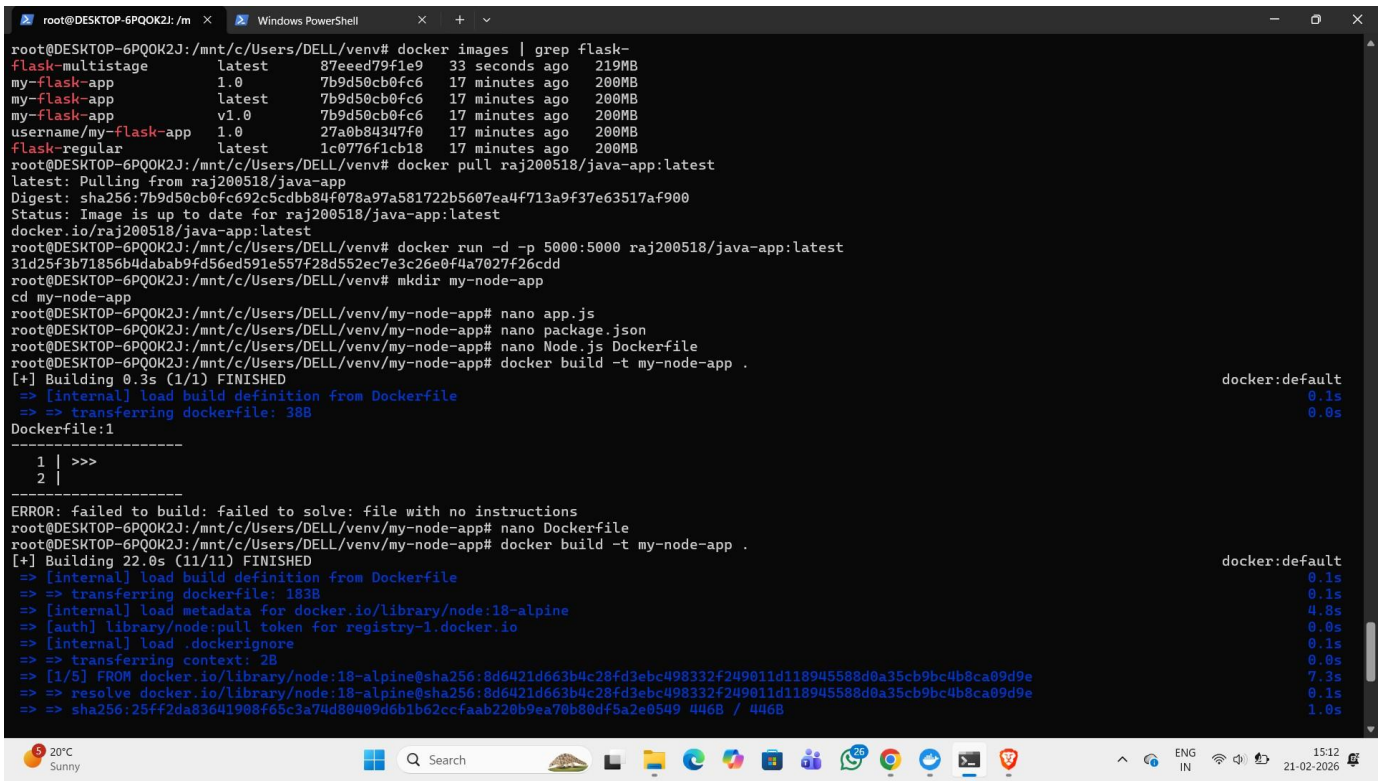
```
docker login
```

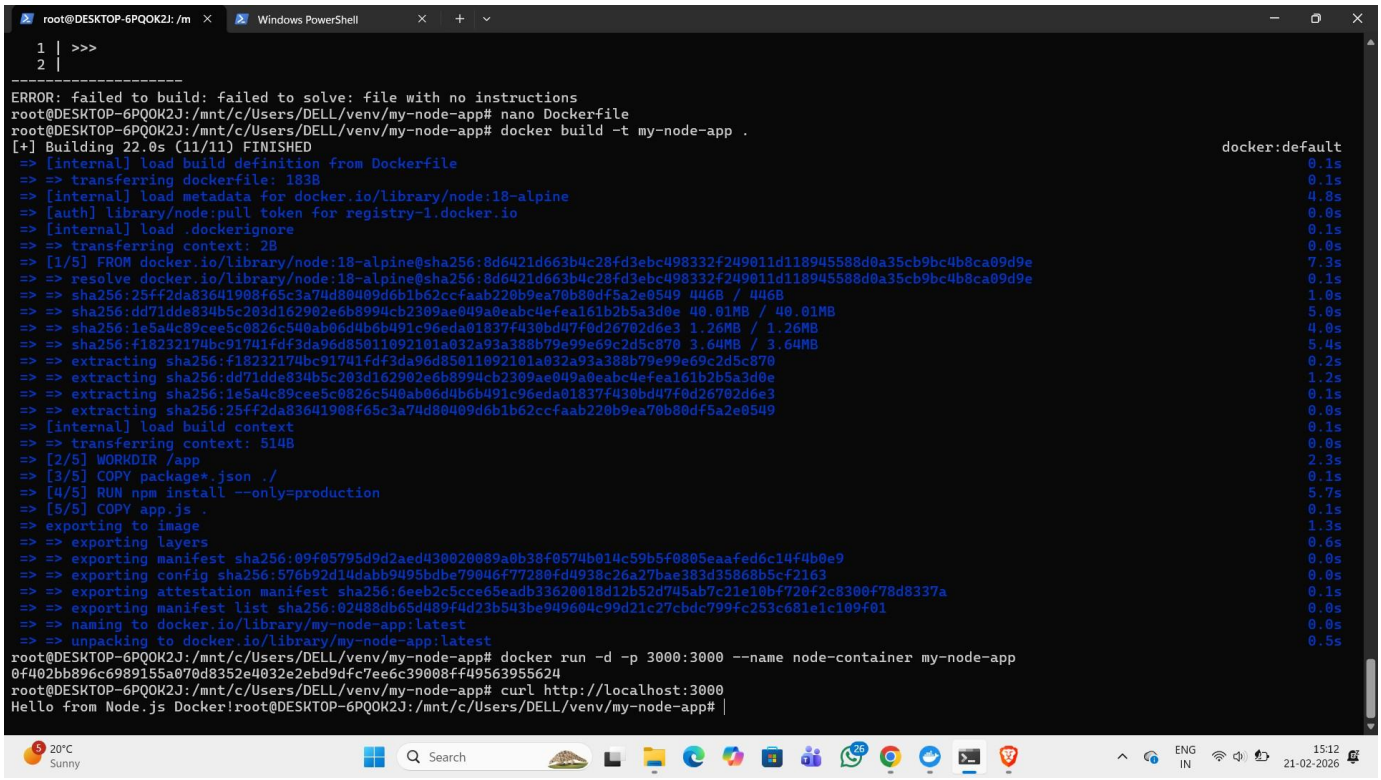
Enter your Docker Hub username and password.

Step 6: Push Image to Docker Hub

```
docker push username/myapp:1.0
```







Example:

```
docker push rajvardhan/myapp:1.0
```

Pull Image from Docker Hub

To download the image:

```
docker pull username/myapp:1.0
```

Result

The Docker image was successfully built, tagged, and pushed to Docker Hub.

📌 Conclusion

This experiment demonstrates the complete Docker workflow:

1. Writing a Dockerfile
2. Creating a .dockerignore file
3. Building a Docker image
4. Tagging the image
5. Publishing it to Docker Hub

These are essential skills for DevOps and containerized application deployment.

Experiment 5

Name: Ashmeet Negi

Course: Containerization and DevOps Lab

Experiment 5: Docker - Volumes, Environment Variables, Monitoring & Networks

Part 1: Docker Volumes - Persistent Data Storage

Lab 1: Understanding Data Persistence

The Problem: Container Data is Ephemeral

```
# Create a container that writes data
docker run -it --name test-container ubuntu /bin/bash

# Inside container: echo "Hello World"
> message.txt cat message.txt  # Shows
"Hello World" exit

# delete and make a new container
docker stop test-container docker
rm test-container
docker run -it --name test-container ubuntu /bin/bash cat
message.txt
# ERROR: File doesn't exist! Data was lost.
```

```
C:\Users\sshar>wsl
ubuntu@LAPTOP-1HQ4L4VM:/mnt/c/Users/sshar$ docker run -it --name test-container ubuntu /bin/bash
^Cubuntu@LAPTOP-1HQ4L4VM:/mnt/c/Users/sshar$ docker run -it --name test-container ubuntu /bin/bash
root@62830e77b471:/# echo "Hello World" > message.txt
root@62830e77b471:/# cat message.txt
Hello World
root@62830e77b471:/# exit
exit
ubuntu@LAPTOP-1HQ4L4VM:/mnt/c/Users/sshar$ docker stop test-container
test-container
ubuntu@LAPTOP-1HQ4L4VM:/mnt/c/Users/sshar$ docker rm test-container
test-container
```

Solution: Docker Volumes

Lab 2: Volume Types

1. Anonymous Volumes

```
# Create anonymous volume (auto-generated name) docker
run -d -v /app/data --name web1 nginx

# Check volume
docker volume ls
# Shows: anonymous volume with random hash

# Inspect container to see volume mount
docker inspect web1 | grep -A 5 Mounts
```

```
ubuntu@LAPTOP-1HQ4L4VM:/mnt/c/Users/sshar$ docker run -it --name test-container ubuntu /bin/bash
root@0144ae136758:/# cat message.txt
cat: message.txt: No such file or directory
root@0144ae136758:/#
```

2. Named Volumes

```
# Create named volume
docker volume create mydata

# Use named volume
docker run -d -v mydata:/app/data --name web2 nginx

# List volumes
docker volume ls
# Shows: mydata

# Inspect volume docker
volume inspect mydata
```

```
ubuntu@LAPTOP-1HQ4L4VM:/mnt/c/Users/sshar$ docker run -d -v /app/data --name web1 nginx
08e1b0ffc5f17b08a53a8660dac6741734cb90173a1106b2ee8e79ff684183da
ubuntu@LAPTOP-1HQ4L4VM:/mnt/c/Users/sshar$ docker volume ls
DRIVER    VOLUME NAME
local     0fb89adcf517cba306d9627c5057189aff60d3d4208c6dd394376ecc97d71e65
local     2c070eb8c1b010667c5bdf45e62a3949880759aff8dc2196d8ed6ee6b6c0a84e
local     40f7c143fd39499667fba94adf43ff8f86cc255efd4d7823c019c1e5f9e94cae
local     86b0547fce383299f6d62d9664a0ea963d276b02c7bee5fdd7f491f15b506447
local     193d4f09fb51cf4164ff660c074af5bba1b7d87e3e7ecc76ae25d4f8fde0e22a
local     365bf9f13890d54d2d01c6a3e13a5572dd0029db81fbbc77c98e3d945a2ee625
local     a9061212fe33f6e8f5266a1e67fd0e3cd4d8e4fdc061a51e0c2f86bb23c2e8d5
local     app-volume
local     assignment-1_pgdata
local     c15f95b8008d5c7b9afc33e96561bacdd69f679d6e798a8e500c821ee49c5f87
local     ca612b63e37b6f01f13c5c04f66b78b7a534aefbc9618c9a970440309b43b749
local     docker-compose-scale_mysql_data
local     docker-compose-scale_wp_content
local     docker-compose_mysql_data
local     docker-compose_wp_content
local     e1f3008a21cde203b05dfa218b5d4aa0b1a44c004b18c49b60deddbbbe59115d
local     e2fc5fd8c76b96650aae7c0bfba2d80b0f3d0cb05748645f617783dda7f770cf
local     ed131165468bbfd7c005ce2288b340a42f29ae8f767853bd8a7c49efbfc7a19b
local     f220705e5b7d12cb1796067d322c90612505b23dc0bc9852a5bb4ab6bfe947e9
local     k3d-mycluster-images
local     mydata
local     mysql-data
local     portainer_data
local     test
ubuntu@LAPTOP-1HQ4L4VM:/mnt/c/Users/sshar$ docker inspect web1 | grep -A 5 Mounts
        "Mounts": [
            {
                "Type": "volume",
                "Name": "e1f3008a21cde203b05dfa218b5d4aa0b1a44c004b18c49b60deddbbbe59115d",
                "Source": "/var/lib/docker/volumes/e1f3008a21cde203b05dfa218b5d4aa0b1a44c004b18c49b60deddbbbe59115d/_data",
                "Destination": "/app/data",
ubuntu@LAPTOP-1HQ4L4VM:/mnt/c/Users/sshar$
```

3. Bind Mounts (Host Directory)

```
# Create directory on host
mkdir ~/myapp-data

# Mount host directory to container
docker run -d -v ~/myapp-data:/app/data --name web3 nginx

# Add file on host echo "From Host" > ~/myapp-
data/host-file.txt

# Check in container docker exec web3 cat
/app/data/host-file.txt
# Shows: From Host
```

```
ubuntu@LAPTOP-1HQ4L4VM:/mnt/c/Users/sshar$ docker volume create mydata
mydata
ubuntu@LAPTOP-1HQ4L4VM:/mnt/c/Users/sshar$ docker run -d -v mydata:/app/data --name web2 nginx
9b02344d704ad761123e9112bf1ea5a5eb6a4b95f97d5fb4e9b79b1eb8992e25
ubuntu@LAPTOP-1HQ4L4VM:/mnt/c/Users/sshar$ docker volume ls
DRIVER    VOLUME NAME
local     0fb89adcf517cba306d9627c5057189aff60d3d4208c6dd394376ecc97d71e65
local     2c070eb8c1b010667c5bdf45e62a3949880759aff8dc2196d8ed6ee6b6c0a84e
local     40f7c143fd39499667fba94adf43ff8f86cc255efd4d7823c019c1e5f9e94cae
local     86b0547fce383299f6d62d9664a0ea963d276b02c7bee5fdd7f491f15b506447
local     193d4f09fb51cf4164ff660c074af5bba1b7d87e3e7ecc76ae25d4f8fde0e22a
local     365bf9f13890d54d2d01c6a3e13a5572dd0029db81fbbc77c98e3d945a2ee625
local     a9061212fe33f6e8f5266a1e67fd0e3cd4d8e4fdc061a51e0c2f86bb23c2e8d5
local     app-volume
local     assignment-1_pgdata
local     c15f95b8008d5c7b9afc33e96561bacdd69f679d6e798a8e500c821ee49c5f87
local     ca612b63e37b6f01f13c5c04f66b78b7a534aefbc9618c9a970440309b43b749
local     docker-compose-scale_mysql_data
local     docker-compose-scale_wp_content
local     docker-compose_mysql_data
local     docker-compose_wp_content
local     e1f3008a21cde203b05dfa218b5d4aa0b1a44c004b18c49b60deddbbbe59115d
local     e2fc5fd8c76b96650aae7c0bfba2d80b0f3d0cb05748645f617783dda7f770cf
local     ed131165468bbfd7c005ce2288b340a42f29ae8f767853bd8a7c49efbfc7a19b
local     f220705e5b7d12cb1796067d322c90612505b23dc0bc9852a5bb4ab6bfe947e9
local     k3d-mycluster-images
local     mydata
local     mysql-data
local     portainer_data
local     test
ubuntu@LAPTOP-1HQ4L4VM:/mnt/c/Users/sshar$ docker volume inspect mydata
[
    {
        "CreatedAt": "2026-02-28T03:03:26Z",
        "Driver": "local",
        "Labels": null,
        "Mountpoint": "/var/lib/docker/volumes/mydata/_data",
        "Name": "mydata",
        "Options": null,
        "Scope": "local"
    }
]
ubuntu@LAPTOP-1HQ4L4VM:/mnt/c/Users/sshar$
```

Lab 3: Practical Volume Examples

Example 1: Database with Persistent Storage

```
# MySQL with named volume
docker run -d \
  --name mysql-db \
  -v mysql-data:/var/lib/mysql \
-e MYSQL_ROOT_PASSWORD=secret \
mysql:8.0

# Check data persists
docker stop mysql-db
docker rm mysql-db

# New container with same volume
docker run -d \
  --name new-mysql \
  -v mysql-data:/var/lib/mysql \
-e MYSQL_ROOT_PASSWORD=secret \
mysql:8.0 # Data is preserved!
```

```
ubuntu@LAPTOP-1HQ4L4VM:/mnt/c/Users/sshar$ cd Downloads/Containerization-and-DevOps
ubuntu@LAPTOP-1HQ4L4VM:/mnt/c/Users/sshar/Downloads/Containerization-and-DevOps$ cd Lab
ubuntu@LAPTOP-1HQ4L4VM:/mnt/c/Users/sshar/Downloads/Containerization-and-DevOps/Lab$ cd Lab-5
ubuntu@LAPTOP-1HQ4L4VM:/mnt/c/Users/sshar/Downloads/Containerization-and-DevOps/Lab/Lab-5$ ls
Dockerfile  screenshots  screenshots-2  app.py  requirements.txt
ubuntu@LAPTOP-1HQ4L4VM:/mnt/c/Users/sshar/Downloads/Containerization-and-DevOps/Lab/Lab-5$ mkdir ~/myapp-data
mkdir: cannot create directory ‘/home/ubuntu/myapp-data’: File exists
ubuntu@LAPTOP-1HQ4L4VM:/mnt/c/Users/sshar/Downloads/Containerization-and-DevOps/Lab/Lab-5$ mkdir ~/myapp-data2
ubuntu@LAPTOP-1HQ4L4VM:/mnt/c/Users/sshar/Downloads/Containerization-and-DevOps/Lab/Lab-5$ docker run -d -v ~/myapp-data2:/app/data --name web3 nginx
d2a02f3ed848943547eddc258db24bc285e4ebe17099cc52a7781cb30cd68a81
ubuntu@LAPTOP-1HQ4L4VM:/mnt/c/Users/sshar/Downloads/Containerization-and-DevOps/Lab/Lab-5$ echo "From Host" > ~/myapp-data/host-file.txt
ubuntu@LAPTOP-1HQ4L4VM:/mnt/c/Users/sshar/Downloads/Containerization-and-DevOps/Lab/Lab-5$ docker exec web3 cat /app/data/host-file.txt
cat: /app/data/host-file.txt: No such file or directory
ubuntu@LAPTOP-1HQ4L4VM:/mnt/c/Users/sshar/Downloads/Containerization-and-DevOps/Lab/Lab-5$ docker exec web3 cat /myapp-data2/data/host-file.txt
cat: /myapp-data2/data/host-file.txt: No such file or directory
ubuntu@LAPTOP-1HQ4L4VM:/mnt/c/Users/sshar/Downloads/Containerization-and-DevOps/Lab/Lab-5$
```

Example 2: Web App with Configuration Files

```
# Create config directory
mkdir ~/nginx-config

# Create nginx config file echo 'server {
listen 80;     server_name localhost;
location / {         return 200 "Hello from
mounted config!";     }
}' > ~/nginx-config/nginx.conf

# Run nginx with config bind mount
docker run -d \
  --name nginx-custom \
  -p 8080:80 \
  -v ~/nginx-config/nginx.conf:/etc/nginx/conf.d/default.conf \
nginx

# Test
curl http://localhost:8080
```

```
ubuntu@LAPTOP-1HQ4L4VM:/mnt/c/Users/sshar/Downloads/Containerization-and-DevOps/Lab/Lab-5$ docker run -d \
  --name mysql-db \
  -v mysql-data:/var/lib/mysql \
  -e MYSQL_ROOT_PASSWORD=secret \
  mysql:8.0
2ffc6a0583e4ba71e26cd88a2973bb2ee8ae57b62a1b07988f0c796021e83d03
ubuntu@LAPTOP-1HQ4L4VM:/mnt/c/Users/sshar/Downloads/Containerization-and-DevOps/Lab/Lab-5$ docker stop mysql-db
mysql-db
ubuntu@LAPTOP-1HQ4L4VM:/mnt/c/Users/sshar/Downloads/Containerization-and-DevOps/Lab/Lab-5$ docker rm mysql-db
mysql-db
ubuntu@LAPTOP-1HQ4L4VM:/mnt/c/Users/sshar/Downloads/Containerization-and-DevOps/Lab/Lab-5$ docker run -d \
  --name new-mysql \
  -v mysql-data:/var/lib/mysql \
  -e MYSQL_ROOT_PASSWORD=secret \
  mysql:8.0
0931b03c8786f999a8e4b459b9bec937447300d419eb8432c6dc12bc692457a8
ubuntu@LAPTOP-1HQ4L4VM:/mnt/c/Users/sshar/Downloads/Containerization-and-DevOps/Lab/Lab-5$
```

—

Lab 4: Volume Management Commands

```
# List all volumes
docker volume ls
```

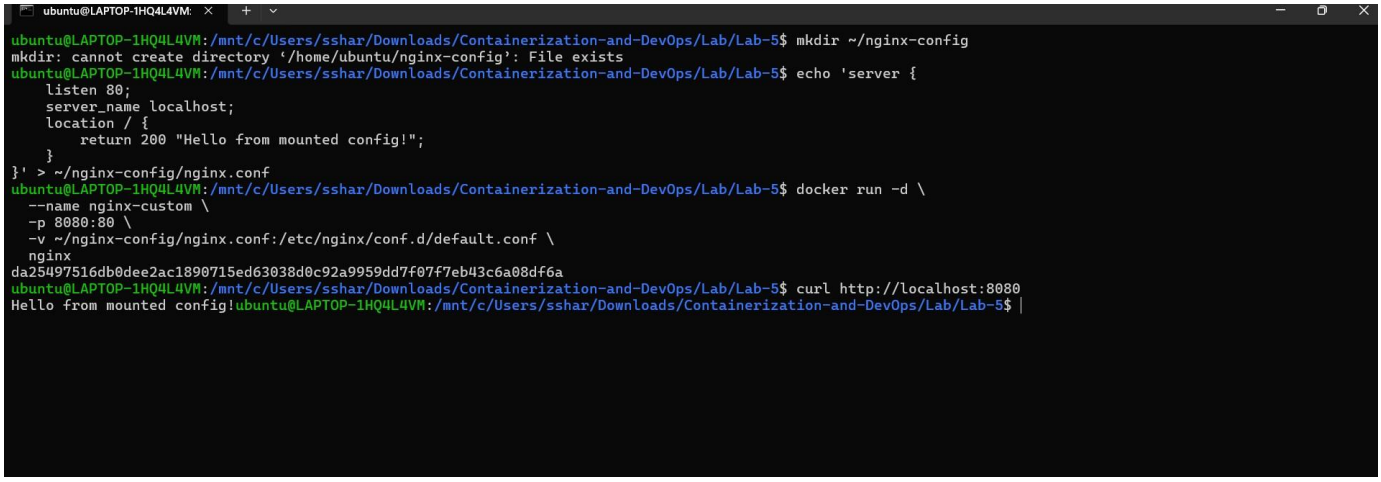
```
# Create a volume docker volume
create app-volume

# Inspect volume details docker
volume inspect app-volume

# Remove unused volumes
docker volume prune

# Remove specific volume
docker volume rm volume-name

# Copy files to/from volume
docker cp local-file.txt container-name:/path/in/volume
```



Part 2: Environment Variables

Lab 1: Setting Environment Variables

Method 1: Using -e flag

```
# Single variable
docker run -d \   --
name app1 \
  -e DATABASE_URL="postgres://user:pass@db:5432/mydb" \
  -e DEBUG="true" \
-p 3000:3000 \   my-
node-app

# Multiple variables
docker run -d \
  -e VAR1=value1 \
  -e VAR2=value2 \
-e VAR3=value3 \
my-app
```

Method 2: Using –env-file

```
# Create .env file echo
"DATABASE_HOST=localhost" > .env echo
"DATABASE_PORT=5432" >> .env echo
"API_KEY=secret123" >> .env

# Use env file
docker run -d \   --
env-file .env \   --
name app2 \   my-app

# Use multiple env files
docker run -d \
  --env-file .env \   --env-
file .env.secrets \   my-app
```

Method 3: In Dockerfile

```
# Set default environment variables
ENV NODE_ENV=production
ENV PORT=3000
ENV APP_VERSION=1.0.0

# Can be overridden at runtime
```

Lab 2: Environment Variables in Applications

Python Flask Example

```
# app.py import os from
flask import Flask app
= Flask(__name__)

# Read environment variables db_host =
os.environ.get('DATABASE_HOST', 'localhost') debug_mode =
os.environ.get('DEBUG', 'false').lower() == 'true' api_key =
os.environ.get('API_KEY')
@app.route('/config')
def config():
return {
        'db_host': db_host,
        'debug': debug_mode,
        'has_api_key': bool(api_key)
    }

if __name__ == '__main__':
    port = int(os.environ.get('PORT', 5000))
app.run(host='0.0.0.0', port=port, debug=debug_mode)
```

Dockerfile with Environment Variables

```
FROM python:3.9-slim

# Set environment variables at build time
ENV PYTHONUNBUFFERED=1
ENV PYTHONDONTWRITEBYTECODE=1

WORKDIR /app

COPY requirements.txt .
RUN pip install -r requirements.txt

COPY app.py .

# Default runtime environment variables
ENV PORT=5000
ENV DEBUG=false

EXPOSE 5000

CMD ["python", "app.py"]
```

Lab 3: Test Environment Variables

```
# Run with custom env vars
docker run -d \   --name
flask-app \
  -p 5000:5000 \
  -e DATABASE_HOST="prod-db.example.com" \
  -e DEBUG="true" \
-e PORT="8080" \
flask-app

# Check environment in running container
docker exec flask-app env
docker exec flask-app printenv DATABASE_HOST

# Test the endpoint curl
http://localhost:5000/config
```

Part 3: Docker Monitoring

Lab 1: Basic Monitoring Commands

`docker stats` - Real-time Container Metrics

```
# Live stats for all containers
docker stats

# Live stats for specific containers
docker stats container1 container2

# Specific format output docker
stats --format "table \t\t\t"

# No-stream (single snapshot)
docker stats --no-stream

# All containers (including stopped)
docker stats --all
```

Useful Format Options:

```
# Custom format docker stats --format "Container:  |
CPU:  | Memory: "

# JSON output
docker stats --format json --no-stream

# Wide output
docker stats --no-stream --no-trunc
```

Lab 2: `docker top` - Process Monitoring

```
# View processes in container
docker top container-name

# View with full command line
docker top container-name -ef
# Compare with host processes
ps aux | grep docker
```

Lab 3: `docker logs` - Application Logs

```
# View logs docker logs
container-name

# Follow logs (like tail -f)
docker logs -f container-name
```

```
# Last N lines docker logs --tail 100
container-name

# Logs with timestamps docker
logs -t container-name

# Logs since specific time docker logs --since
2024-01-15 container-name

# Combine options
docker logs -f --tail 50 -t container-name
```

Lab 4: Container Inspection

```
# Detailed container info
docker inspect container-name

# Specific information docker inspect --
format='' container-name docker inspect -
-format='' container-name docker inspect
--format='' container-name

# Resource limits docker inspect --
format='' container-name docker inspect --
format='' container-name
```

Lab 5: Events Monitoring

```
# Monitor Docker events in real-time
docker events

# Filter events docker events --filter
'type=container' docker events --filter
'event=start' docker events --filter
'event=die'
# Since specific time docker
events --since '2024-01-15'

# Format output docker
events --format '  '
```

Lab 6: Practical Monitoring Script

```
#!/bin/bash # monitor.sh - Simple
Docker monitoring

echo "=== Docker Monitoring Dashboard ==="
echo "Time: $(date)" echo
```

```
echo "1. Running Containers:"
docker ps --format "table \t\t"
echo

echo "2. Resource Usage:" docker stats --no-stream
--format "table \t\t\t\t" echo

echo "3. Recent Events:" docker events --since '5m' --until '0s'
--format '  ' | tail -5 echo

echo "4. System Info:"
docker system df
```

Part 4: Docker Networks

Lab 1: Understanding Docker Network Types

List Networks

```
# Default networks
docker network ls

# Output:
# NETWORK ID     NAME      DRIVER    SCOPE
# abc123         bridge    bridge    local
# def456         host      host      local
# ghi789         none      null      local
```

Lab 2: Network Types Explained

1. Bridge Network (Default)

```
# Containers on bridge network can communicate
# Each container gets own IP, isolated from host

# Create custom bridge network
docker network create my-network

# Inspect network docker network
inspect my-network

# Run containers on custom network docker run -d --
name web1 --network my-network nginx docker run -d --
name web2 --network my-network nginx

# Containers can communicate using container names docker
exec web1 curl http://web2
```

2. Host Network

```
# Container uses host's network directly # No
network isolation, shares host's IP docker run -d -
-name host-app --network host nginx

# Access directly on host port 80
curl http://localhost
```

3. None Network

```
# No network access docker run -d --name isolated-app --network
none alpine sleep 3600

# Test - no network interfaces
docker exec isolated-app ifconfig
# Only loopback interface
```

4. Overlay Network (Swarm)

```
# For Docker Swarm - multi-host networking docker
network create --driver overlay my-overlay
```

Lab 3: Network Management Commands

```
# Create network docker network create app-network docker network create --driver bridge --
subnet 172.20.0.0/16 --gateway 172.20.0.1 my-subne

# Connect container to network docker network connect
app-network existing-container

# Disconnect container from network docker network
disconnect app-network container-name

# Remove network docker
network rm network-name

# Prune unused networks
docker network prune
```

Lab 4: Multi-Container Application Example

Web App + Database Communication

```
# Create network docker network
create app-network
```

```
# Start database
docker run -d \   --
name postgres-db \
  --network app-network \
  -e POSTGRES_PASSWORD=secret \   -v
pgdata:/var/lib/postgresql/data \
postgres:15

# Start web application
docker run -d \
  --name web-app \
  --network app-network \
  -p 8080:3000 \
  -e DATABASE_URL="postgres://postgres:secret@postgres-db:5432/mydb" \
  -e DATABASE_HOST="postgres-db" \   node-app # Web app can
connect to database using "postgres-db" hostname
```

Lab 5: Network Inspection & Debugging

```
# Inspect network docker
network inspect bridge

# Check container IP docker inspect --
format='' container-name

# DNS resolution test docker exec container-name
nslookup another-container

# Network connectivity test docker exec container-name ping
-c 4 google.com docker exec container-name curl -I
http://another-container

# View network ports
docker port container-name
```

Quick Reference Cheatsheet

Volumes:

```
docker volume create <name> docker run -
v <volume>:/path docker run -v
/host/path:/container/path docker volume
ls docker volume rm <name>
```

Environment Variables:

```
docker run -e VAR=value
docker run --env-file .env
# In Dockerfile: ENV VAR=value
```

Monitoring:

```
docker stats docker logs -
f <container> docker top
<container> docker inspect
<container> docker events
```

Networks:

```
docker network create <name> docker run --
network <name> docker network connect
<network> <container> docker network inspect
<network>
```

Practice Exercises

Exercise 1: Database Backup

```
# Create a PostgreSQL container with volume
# Backup data using docker cp or volume backup techniques
# Restore to new container
```

Exercise 2: Multi-Service

Setup

```
# Create: web app + database + cache
# Use custom network for communication
# Set environment variables for configuration
# Monitor all services
```

Exercise 3: Log Analysis

```
# Run a container that generates logs
# Use docker logs with various filters
# Redirect logs to a file on host using bind mount
```

Exercise 4: Network Isolation

```
# Create two separate networks
# Put containers in different networks
# Test connectivity between networks
# Connect a container to both networks
```

Cleanup

```
# Stop and remove all containers
docker stop $(docker ps -aq)
docker rm $(docker ps -aq)
# Remove all volumes
docker volume prune -f

# Remove all networks (except defaults)
docker network prune -f

# Remove unused images
docker image prune -f
```

Key Takeaways

1. **Volumes** persist data beyond container lifecycle
2. **Environment variables** configure containers dynamically
3. **Monitoring commands** help debug and optimize containers
4. **Networks** enable secure container communication
5. **Always use named volumes** for production data
6. **Custom networks** provide better isolation and DNS
7. **Monitor resource usage** to prevent issues
8. **Use .env files** for sensitive configuration

> **This experiment covers essential Docker features for building, configuring, and managing production-ready containerized applications.**

Experiment 6

Name: Ashmeet Negi

Course: Containerization and DevOps Lab

Experiment 6 – Docker Run vs Docker Compose

This lab demonstrates how to run containers using **docker run** and how the same configuration can be defined using **Docker Compose**. Docker Compose simplifies management of multi-container applications by storing configuration in a YAML file.

1. Important `docker run` Options

Option	Purpose
`-p`	Port mapping
`-v`	Volume mount
`-e`	Environment variables
`--network`	Connect container to network
`--restart`	Restart policy
`--memory`	Limit RAM usage
`--cpus`	Limit CPU usage
`--name`	Container name
`-d`	Run container in background

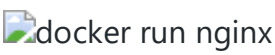
2. Example: Running Nginx using docker run

```
docker run -d \
  --name my-nginx \
  -p 8080:80 \
  -v ./html:/usr/share/nginx/html \
  -e NGINX_HOST=localhost \   --
restart unless-stopped \
nginx:alpine
```

Explanation

Part	Meaning
`-d`	Run container in background
`--name my-nginx`	Container name
`-p 8080:80`	Host port 8080 → Container port 80
`-v ./html:/usr/share/nginx/html`	Mount local html folder
`-e`	Set environment variable
`--restart unless-stopped`	Restart automatically
`nginx:alpine`	Docker image

Screenshot – Running Container


docker run nginx

3. Same Configuration Using Docker Compose

`docker-compose.yml`

```
version: '3.8'
```

```
services:
nginx:
    image: nginx:alpine
container_name: my-nginx
ports:
-    "8080:80"     volumes:
-    ./html:/usr/share/nginx/html
environment:
      NGINX_HOST: localhost
restart: unless-stopped
```

Run the container

```
docker compose up -d
```

Screenshot – Docker Compose Running


docker compose up

4. Docker Run vs Docker Compose Mapping

Docker Run	Docker Compose
-p 8080:80	ports:
-v host:container	volumes:
-e KEY=value	environment:
--name	container_name:
--network	networks:
--restart	restart:
--memory	deploy.resources.limits.memory
--cpus	deploy.resources.limits.cpus

5. Lab Example 1 – Nginx Web Server

Using docker run

```
docker run -d \
  --name lab-nginx \
  -p 8081:80 \
  -v $(pwd)/html:/usr/share/nginx/html \
nginx:alpine
```

Check running containers

```
docker ps
```

Open browser

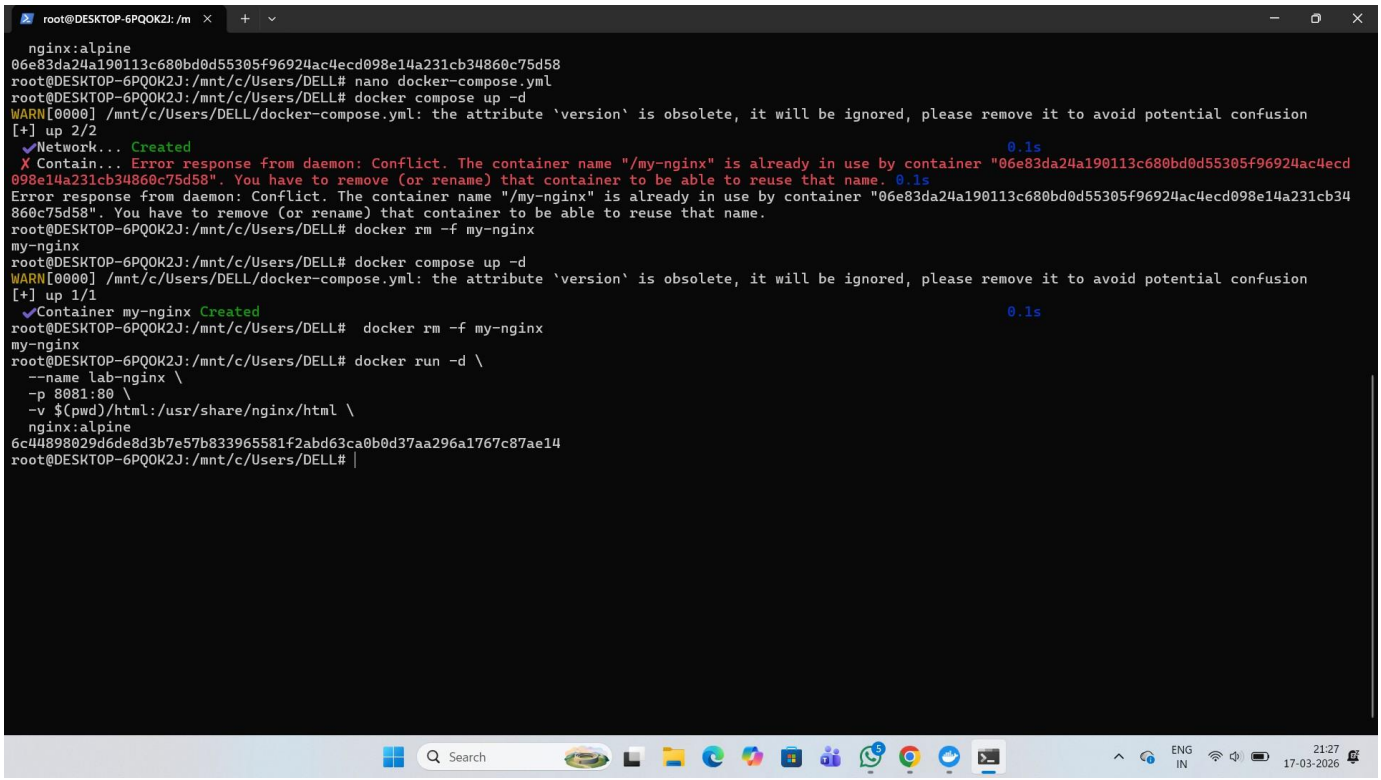
```
http://localhost:8081
```

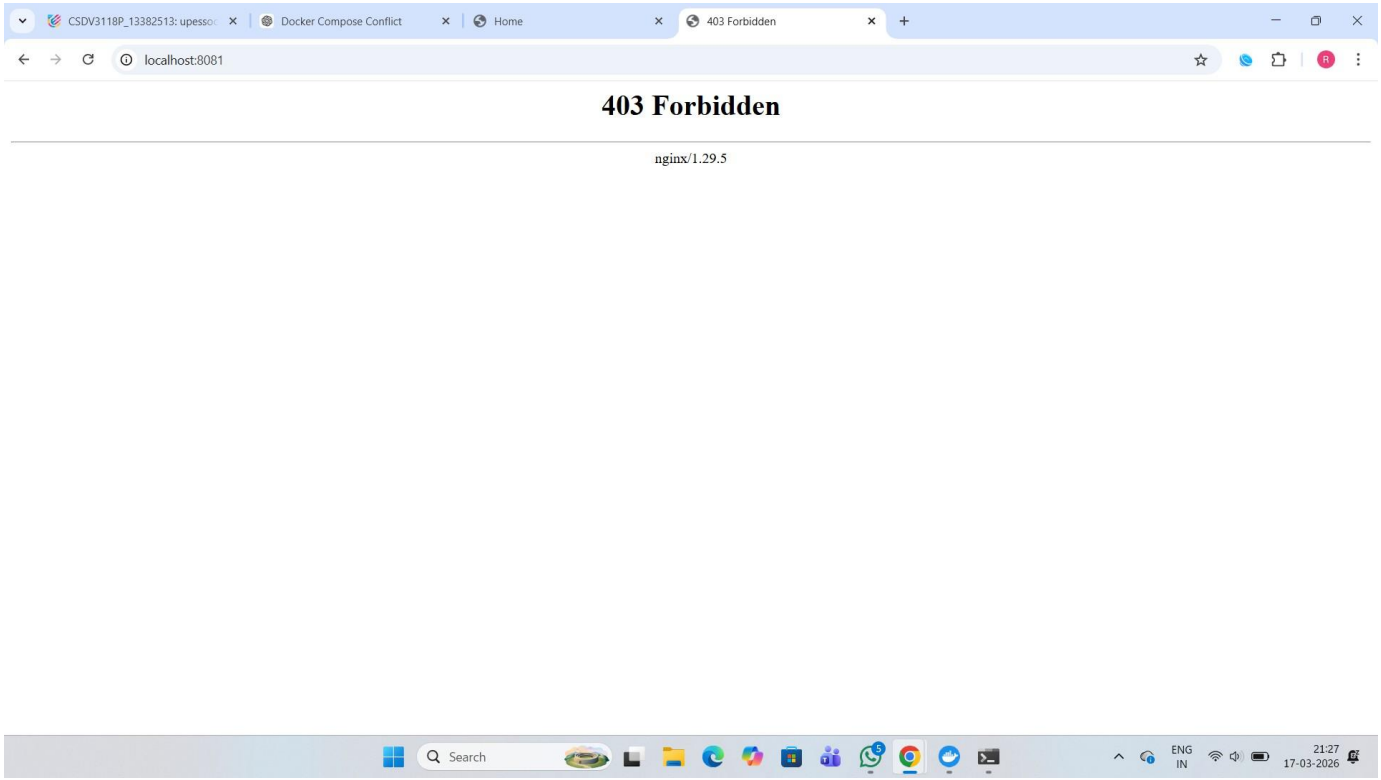
Stop and remove

```
docker stop lab-nginx

docker rm lab-nginx
```

Screenshot – Browser Output





Using Docker Compose

```yaml
version: '3.8'

services:
nginx:
```

```
    image: nginx:alpine
container_name: lab-nginx
ports:
- "8081:80"     volumes:
- ./html:/usr/share/nginx/html
```

Run

```
docker compose up -d
```

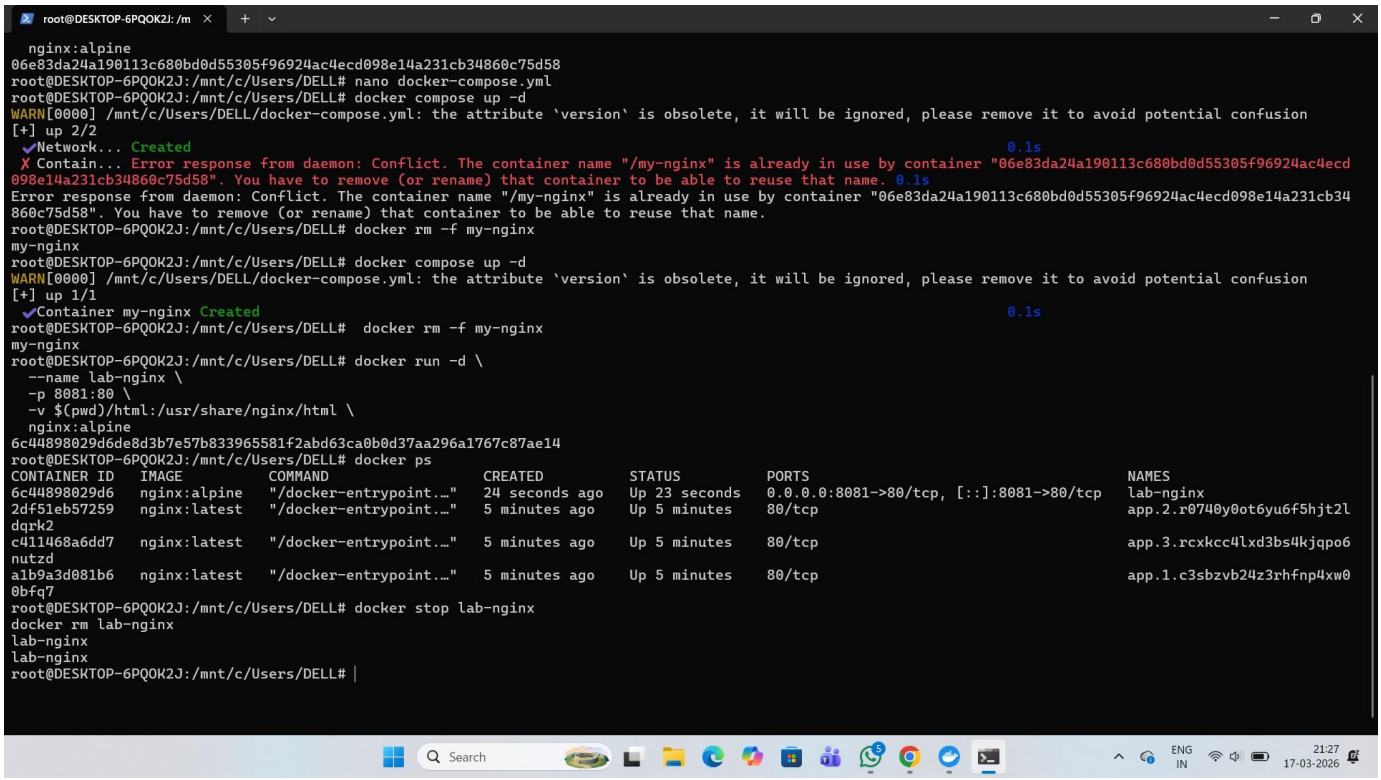
Check containers

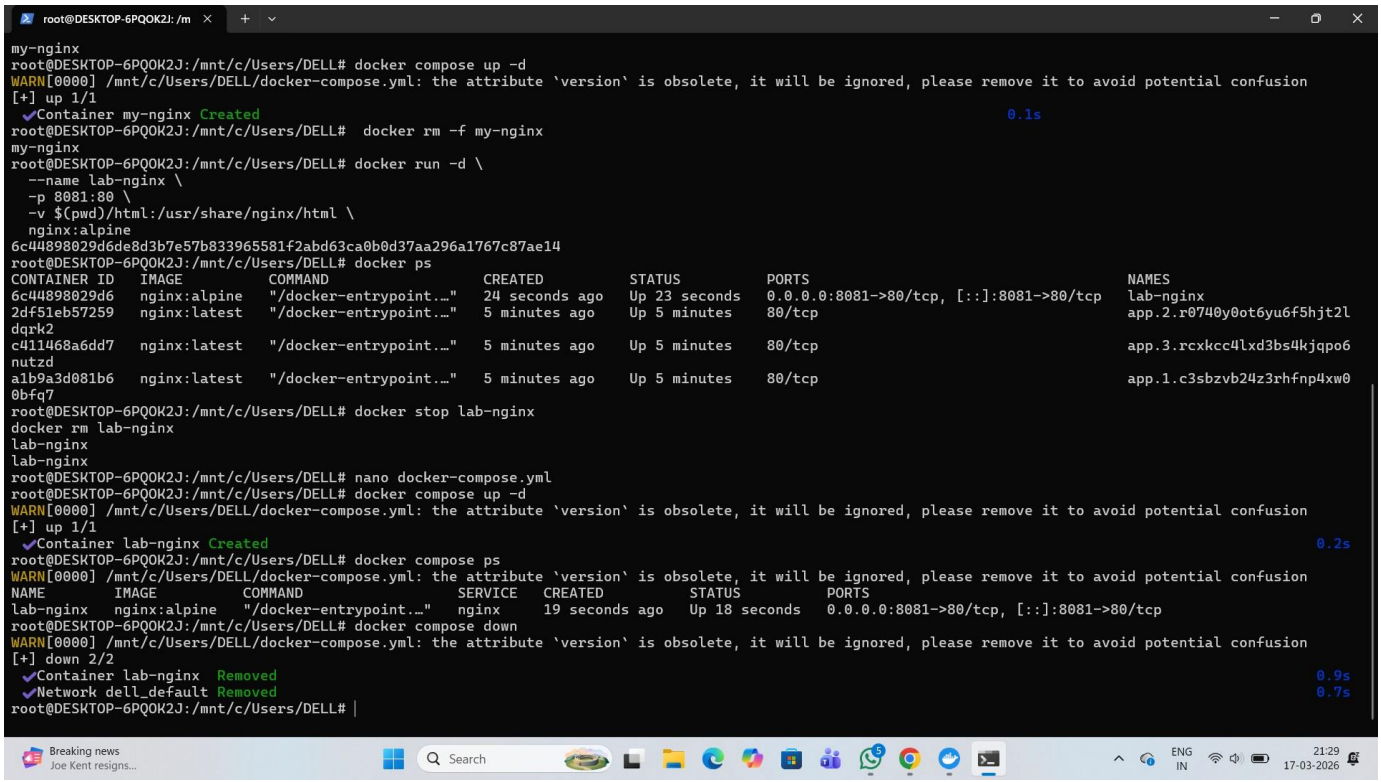
```
docker compose ps
```

Stop

```
docker compose down
```

Screenshot – Compose Containers





6. Lab Example 2 – WordPress with MySQL

Using docker run

Create network

```
docker network create wp-net
```

Run MySQL

```
docker run -d \   --
name mysql \
  --network wp-net \
  -e MYSQL_ROOT_PASSWORD=secret \
-e MYSQL_DATABASE=wordpress \
mysql:5.7
```

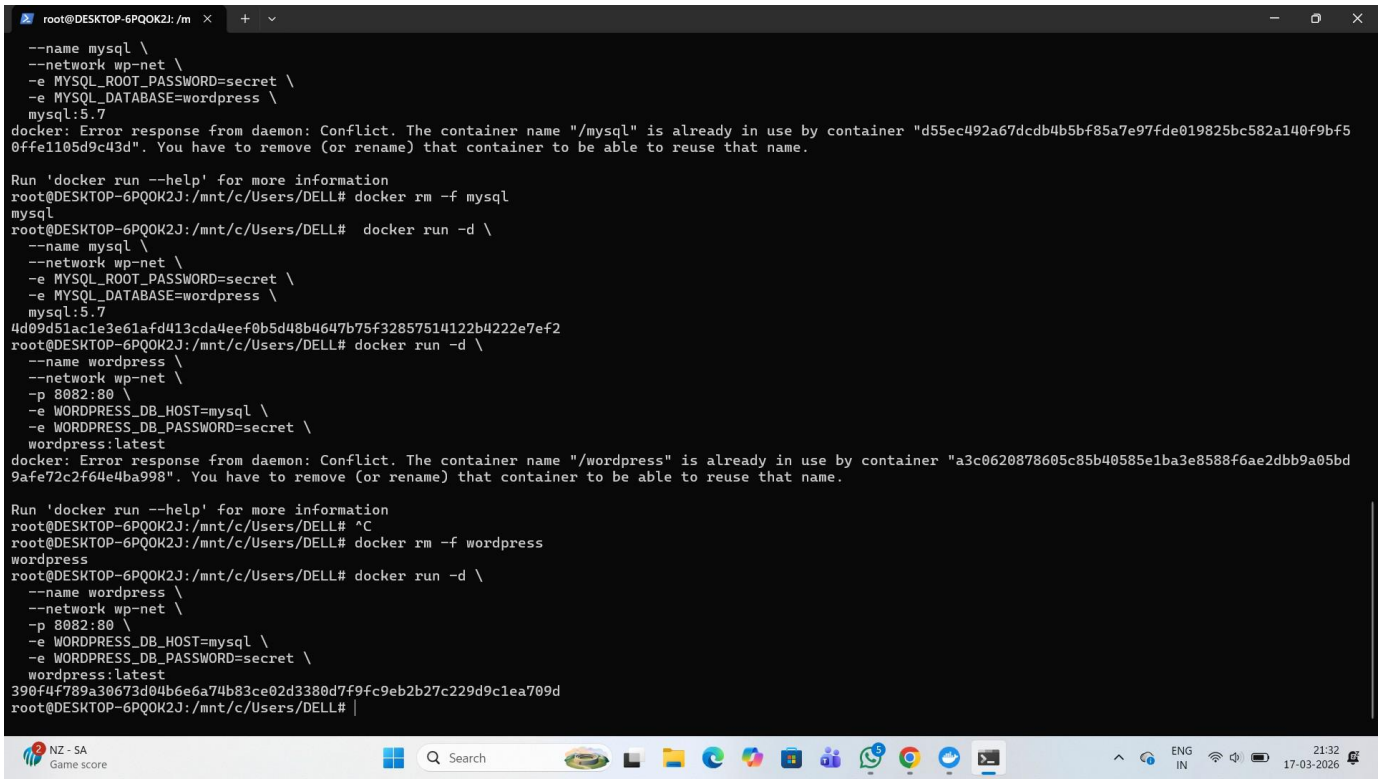
Run WordPress

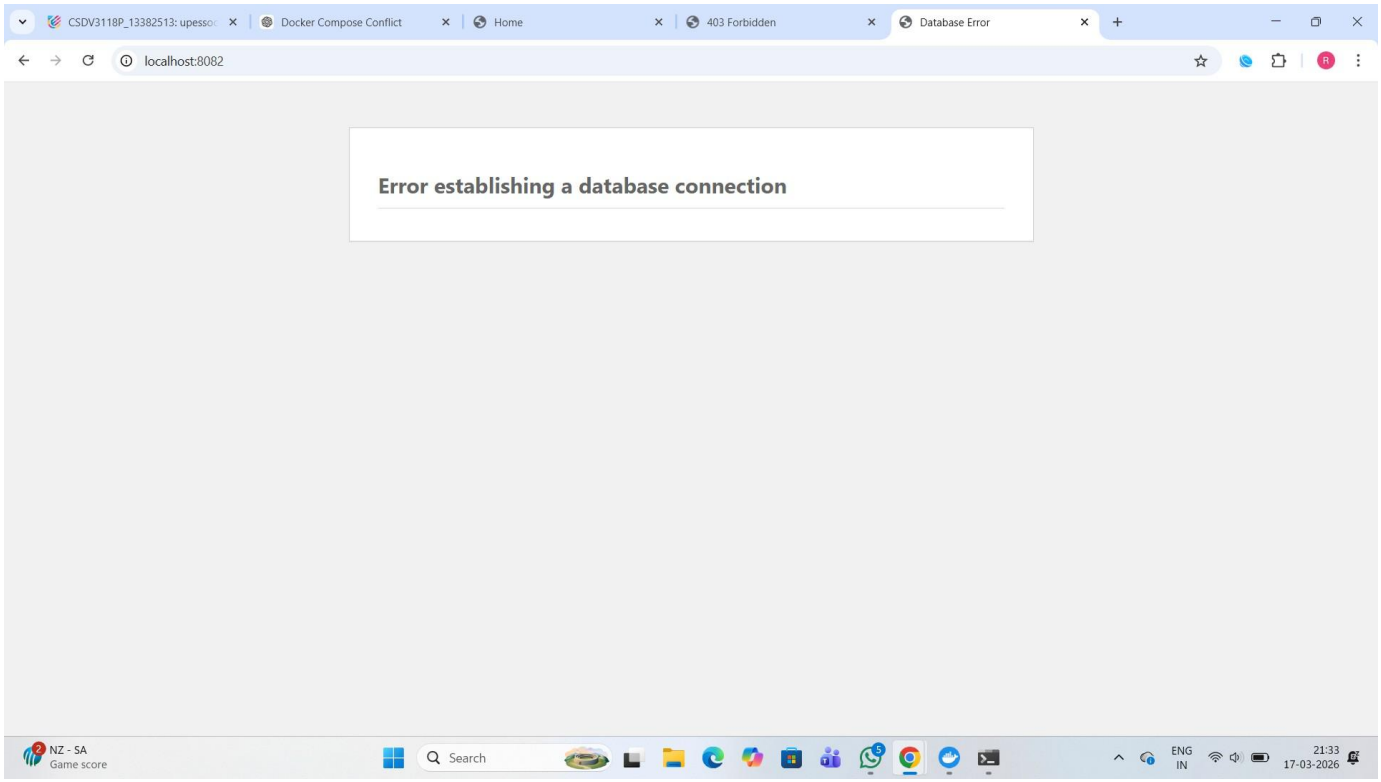
```
docker run -d \   --
name wordpress \   --
network wp-net \
  -p 8082:80 \
  -e WORDPRESS_DB_HOST=mysql \   -e
WORDPRESS_DB_PASSWORD=secret \
wordpress:latest
```

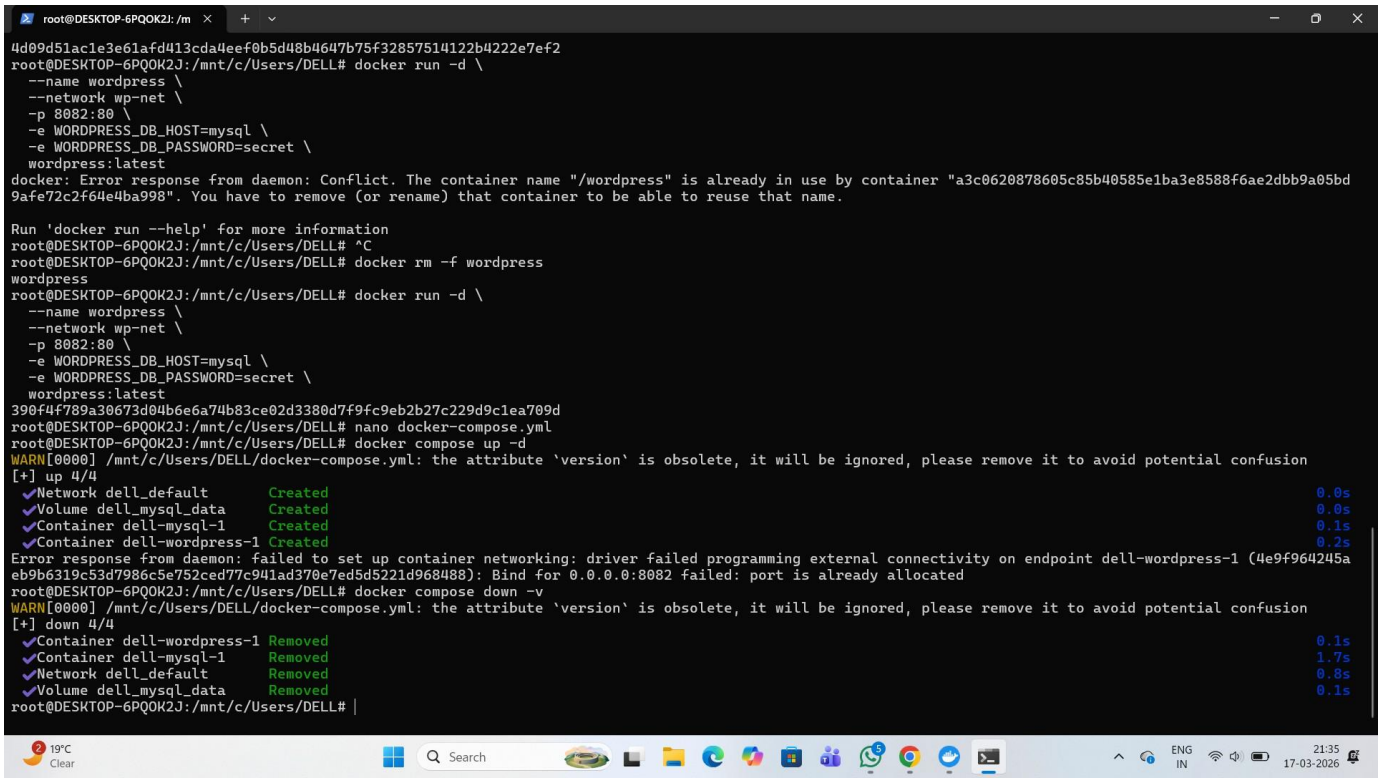
Open in browser

```
http://localhost:8082
```

Screenshot – WordPress Setup Page







Using Docker Compose

```
version: '3.8'

services:

  mysql:
    image: mysql:5.7
environment:
MYSQL_ROOT_PASSWORD: secret
MYSQL_DATABASE: wordpress
volumes:
      - mysql_data:/var/lib/mysql

  wordpress:
    image: wordpress:latest
ports:       - "8082:80"
environment:
WORDPRESS_DB_HOST: mysql
WORDPRESS_DB_PASSWORD: secret
depends_on:       - mysql

volumes:
mysql_data:
```

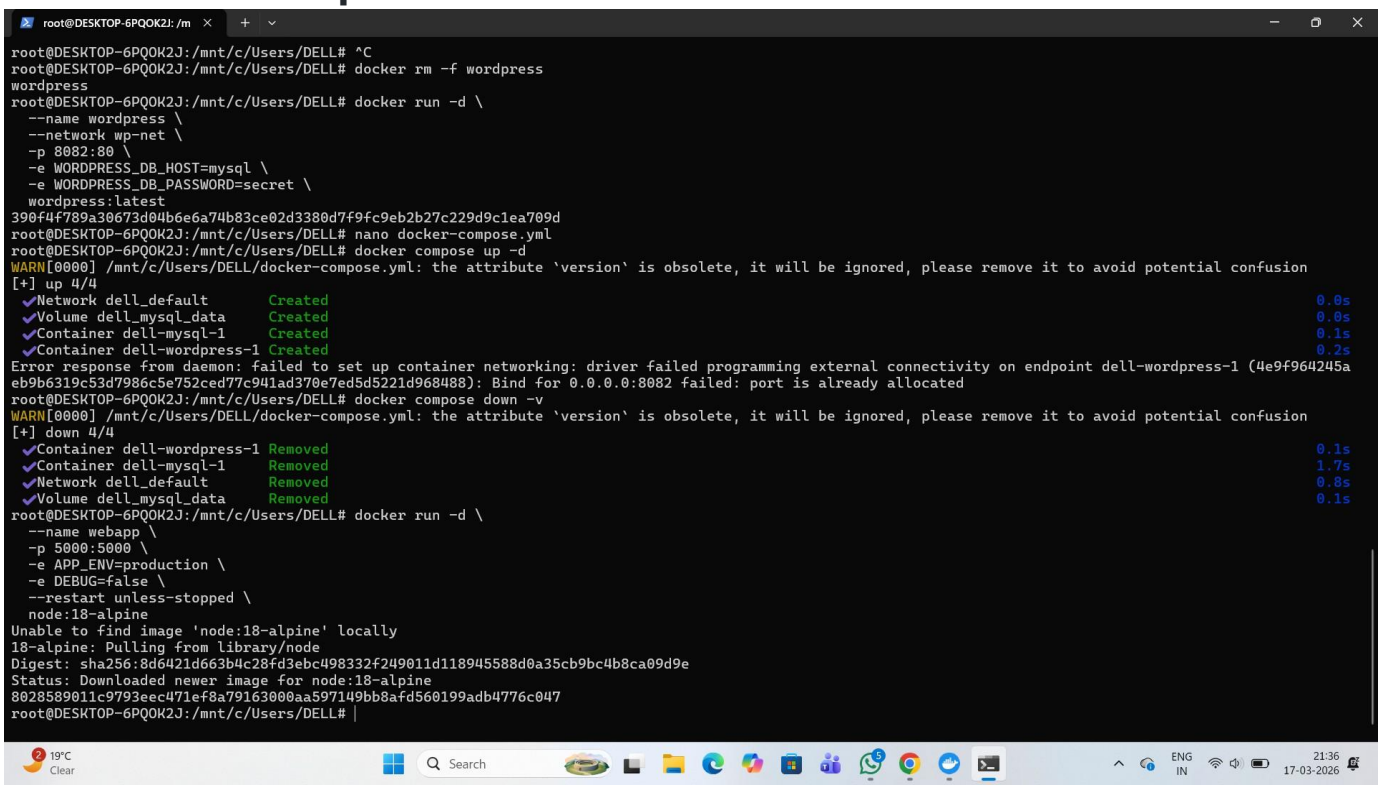
Run

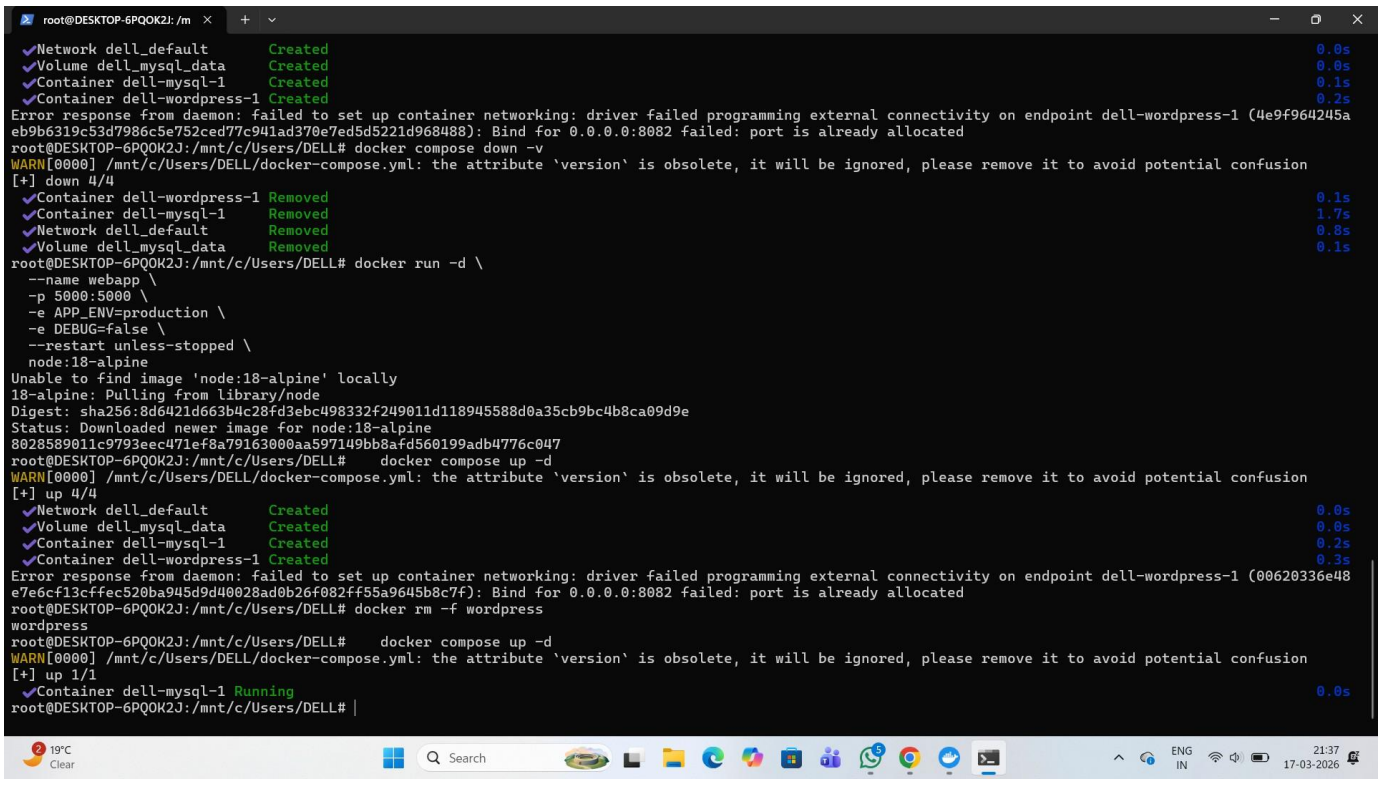
```
docker compose up -d
```

Stop and remove volumes

```
docker compose down -v
```

Screenshot – Compose WordPress Containers





7. Resource Limiting Example

Docker Run

```
docker run -d \
  --name limited-app \
  -p 9000:9000 \
  --memory="256m" \
  --cpus="0.5" \   --
restart always \
nginx:alpine
```

Docker Compose

```
deploy:
resources:
limits:
      memory: 256M
cpus: "0.5"
```

8. Docker Compose Build Example (Node App)

app.js

```
const http = require('http');

http.createServer((req, res) => {
res.end("Docker Compose Build Lab");
}).listen(3000);
```

Dockerfile

```
FROM node:18-alpine
```

```
WORKDIR /app

COPY app.js .
EXPOSE 3000

CMD ["node", "app.js"]
```

docker-compose.yml

```
version: '3.8'

services:

  nodeapp:
build:
context: .
      dockerfile: Dockerfile
container_name: custom-node-app
ports:       - "3000:3000"
```

Build and run

```
docker compose up --build -d
```

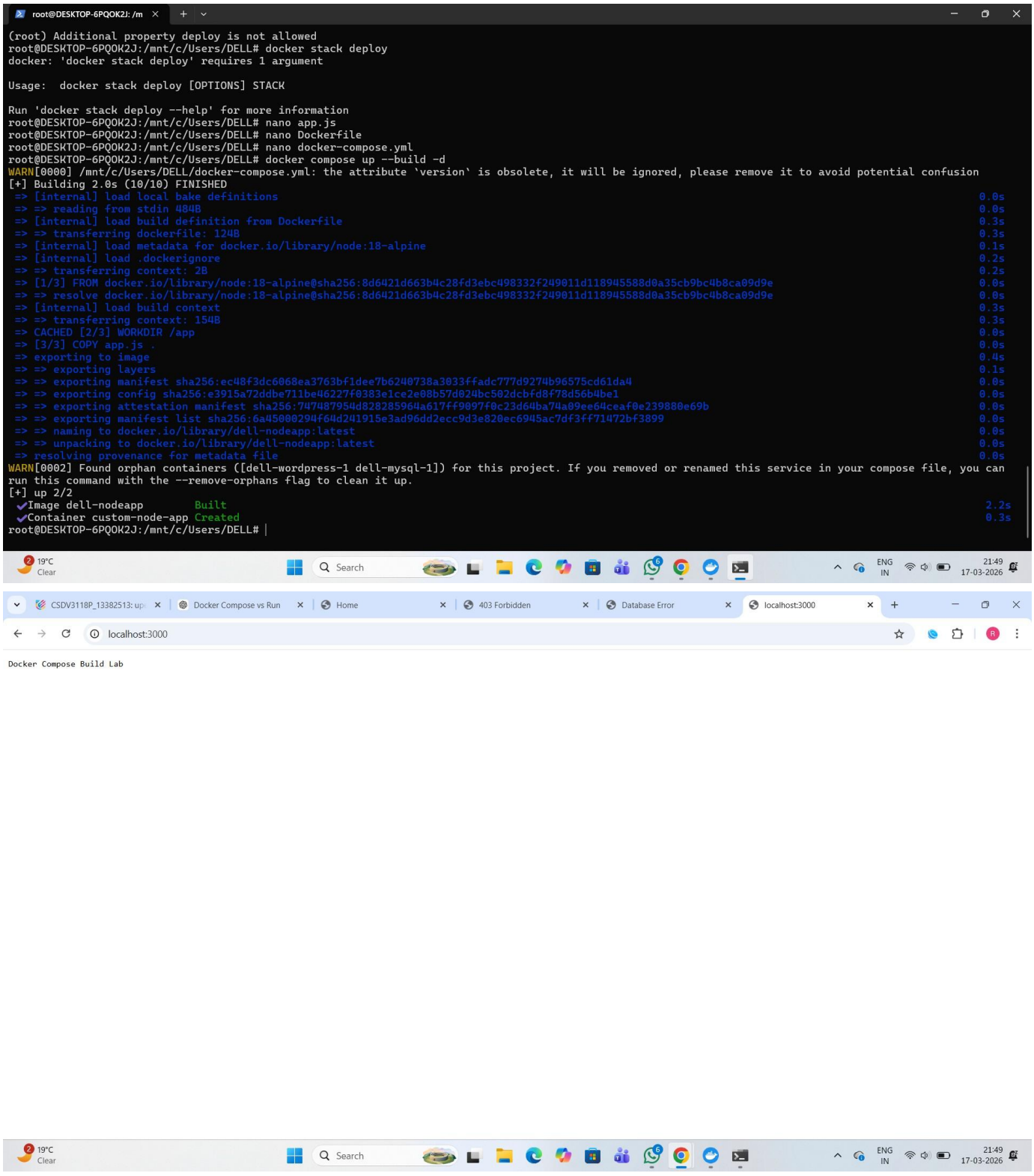
Open browser

```
http://localhost:3000
```

Check images

```
docker images
```

Screenshot – Node App Output



9. Scaling Containers

```
docker compose up --scale web=3
```

This command starts **3 instances of the same service**.

Screenshot – Scaled Containers

10. Conclusion

Docker Run

- Used to start **single containers manually**
- Configuration given via **command line flags**

Docker Compose

- Used to run **multiple containers together**
- Configuration written in **docker-compose.yml**

Docker Compose simplifies development, testing, and deployment of **multi-container applications**.

Experiment 7

Name: Ashmeet Negi

Course: Containerization and DevOps Lab

Lab Experiment 7: CI/CD Pipeline using Jenkins, GitHub and Docker Hub

Aim

To design and implement a complete CI/CD pipeline using **Jenkins**, integrating source code from **GitHub**, and building & pushing Docker images to **Docker Hub** automatically on every code push.

Workflow

```
Developer → git push → GitHub → Webhook → Jenkins → Docker Build → Docker Hub
```

Project Structure

```
my-app/ ├── app.py              # Flask web
application
├── requirements.txt    # Python dependencies
├── Dockerfile          # Docker image build instructions
└── Jenkinsfile         # Jenkins pipeline definition
```

Application Code

app.py

```
from flask import Flask
app = Flask(__name__)

@app.route("/") def home():     return
"Hello from CI/CD Pipeline!"
app.run(host="0.0.0.0", port=80)
```

requirements.txt

```
flask
```

Dockerfile

```
FROM python:3.10-slim

WORKDIR /app
COPY . .

RUN pip install -r requirements.txt

EXPOSE 80
CMD ["python", "app.py"]
```

Jenkinsfile

```
pipeline {
agent any

    environment {
        IMAGE_NAME = "your-dockerhub-username/myapp"
    }

stages {

        stage('Clone Source') {             steps {                 git branch: 'main',
url: 'https://github.com/your-username/my-app.git'
            }
}

        stage('Build Docker Image') {
steps {
                sh 'docker build -t $IMAGE_NAME:latest .'
```

```
            }
}

        stage('Login to Docker Hub') {             steps {
withCredentials([string(credentialsId: 'dockerhub-token', variable: 'DOCKE
sh 'echo $DOCKER_TOKEN | docker login -u your-dockerhub-username --pas
                }
            }
}

        stage('Push to Docker Hub') {
steps {                 sh 'docker push
$IMAGE_NAME:latest'             }
        }
    }
}
```

Setup Instructions

Prerequisites

- Docker Desktop installed (Apple Silicon / ARM64 for Mac M1)
- GitHub account Docker Hub
- account ngrok account (for
- webhook)

Step 1: Jenkins Setup using Docker

Dockerfile for Jenkins (ARM64):

```
FROM jenkins/jenkins:lts

USER root

RUN apt-get update -y && \     apt-get install -y curl ca-certificates gnupg && \
install -m 0755 -d /etc/apt/keyrings && \     curl -fsSL
https://download.docker.com/linux/debian/gpg | gpg --dearmor -o /etc/apt/ke     chmod a+r
/etc/apt/keyrings/docker.gpg && \     echo "deb [arch=arm64 signed-
by=/etc/apt/keyrings/docker.gpg] https://download.docker.
    apt-get update -y && \     apt-get
install -y docker-ce-cli && \
groupadd -f docker && \     usermod -aG
docker jenkins

USER jenkins
```

Build and Run Jenkins:

```
# Build ARM64 Jenkins image
docker build --platform linux/arm64 -t jenkins-arm64 .

# Run Jenkins container
docker run -d \
  --name jenkins \
  --platform linux/arm64 \
  -p 8080:8080 \
  -p 50000:50000 \
  -v jenkins_home:/var/jenkins_home \
  -v /var/run/docker.sock:/var/run/docker.sock \
  -u root \
  jenkins-arm64

# Fix Docker socket permissions
docker exec -it --user root jenkins chmod 666 /var/run/docker.sock

# Create symlink
docker exec -it --user root jenkins ln -sf /usr/bin/docker /usr/local/bin/docker
```

Get Jenkins unlock password:

```
docker exec -it jenkins cat /var/jenkins_home/secrets/initialAdminPassword
```

Step 2: Jenkins Configuration

Add Docker Hub Credentials:

```
Manage Jenkins → Credentials → (global) → Add Credentials
  Kind: Secret text
  ID: dockerhub-token
  Secret: <your Docker Hub access token>
```

Step 3: GitHub Webhook Setup

Install ngrok:

```
brew install ngrok ngrok config add-
authtoken YOUR_NGROK_TOKEN ngrok http 8080
```

Pipeline Stages

Stage	Description	Status
Checkout SCM	Jenkins fetches Jenkinsfile from GitHub	Done
Clone Source	Pulls latest application code	Done
Build Docker Image	Builds image using Dockerfile	Done
Login to Docker Hub	Authenticates using stored token	Done
Push to Docker Hub	Pushes image to Docker Hub registry	Done

Key Concepts

Why Docker in CI/CD?

Docker ensures **consistent builds** across any environment.

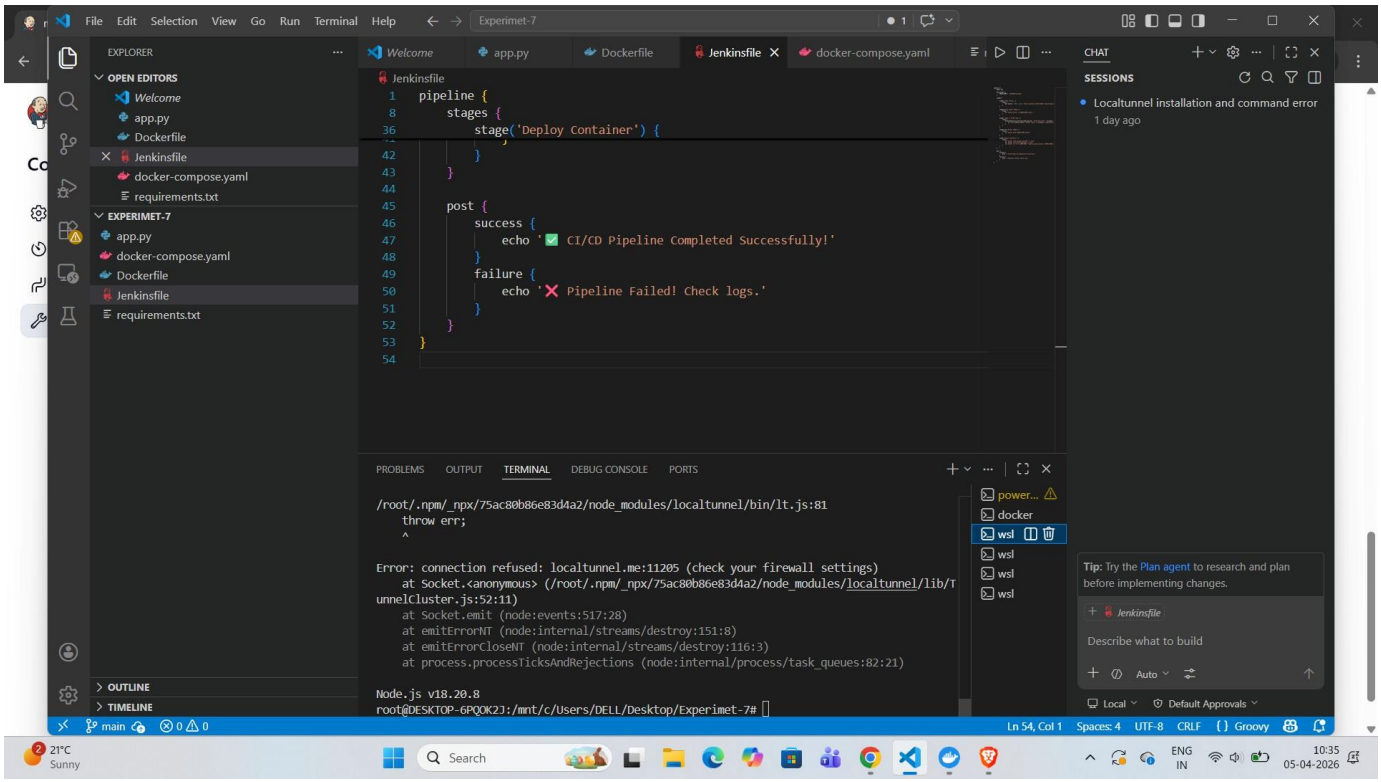
Why store credentials in Jenkins?

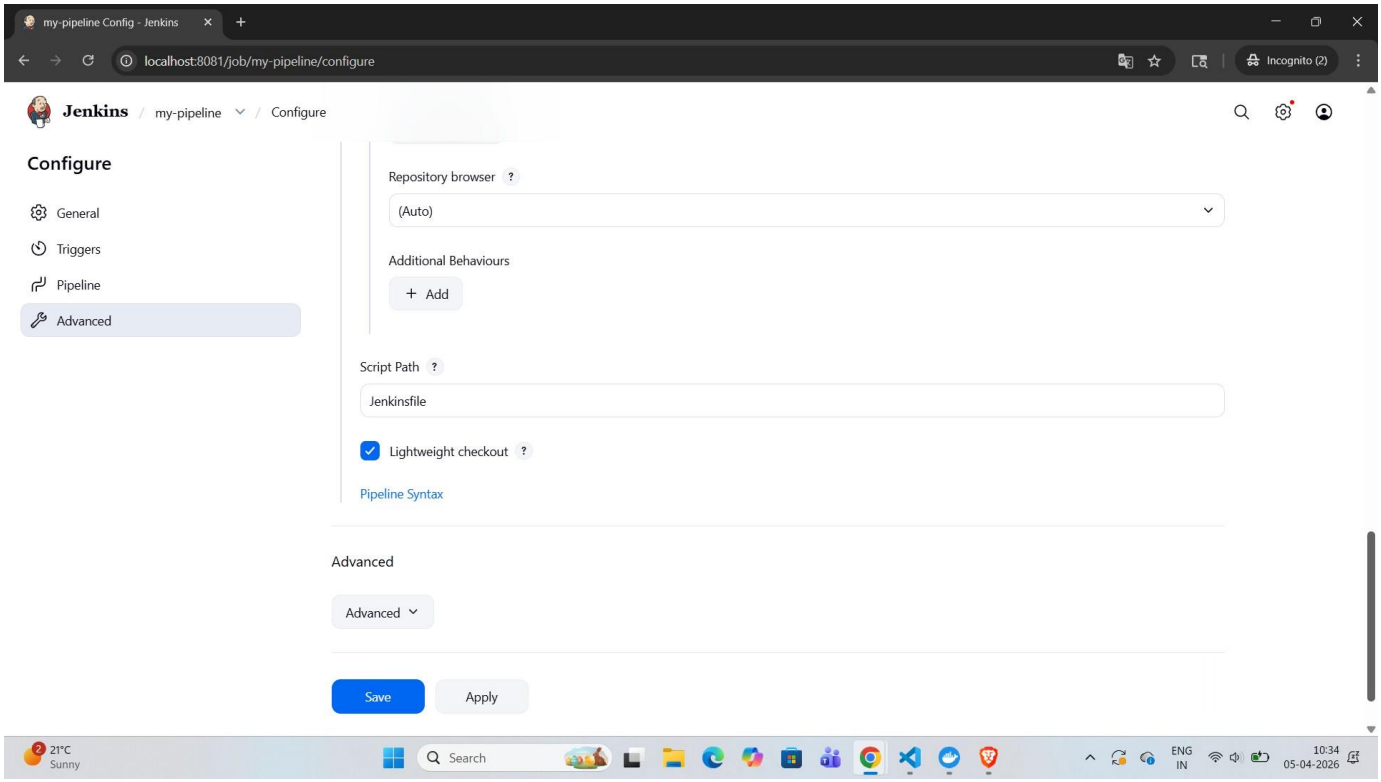
Security — Secrets are encrypted and injected only at runtime.

Role of Docker Socket Mount

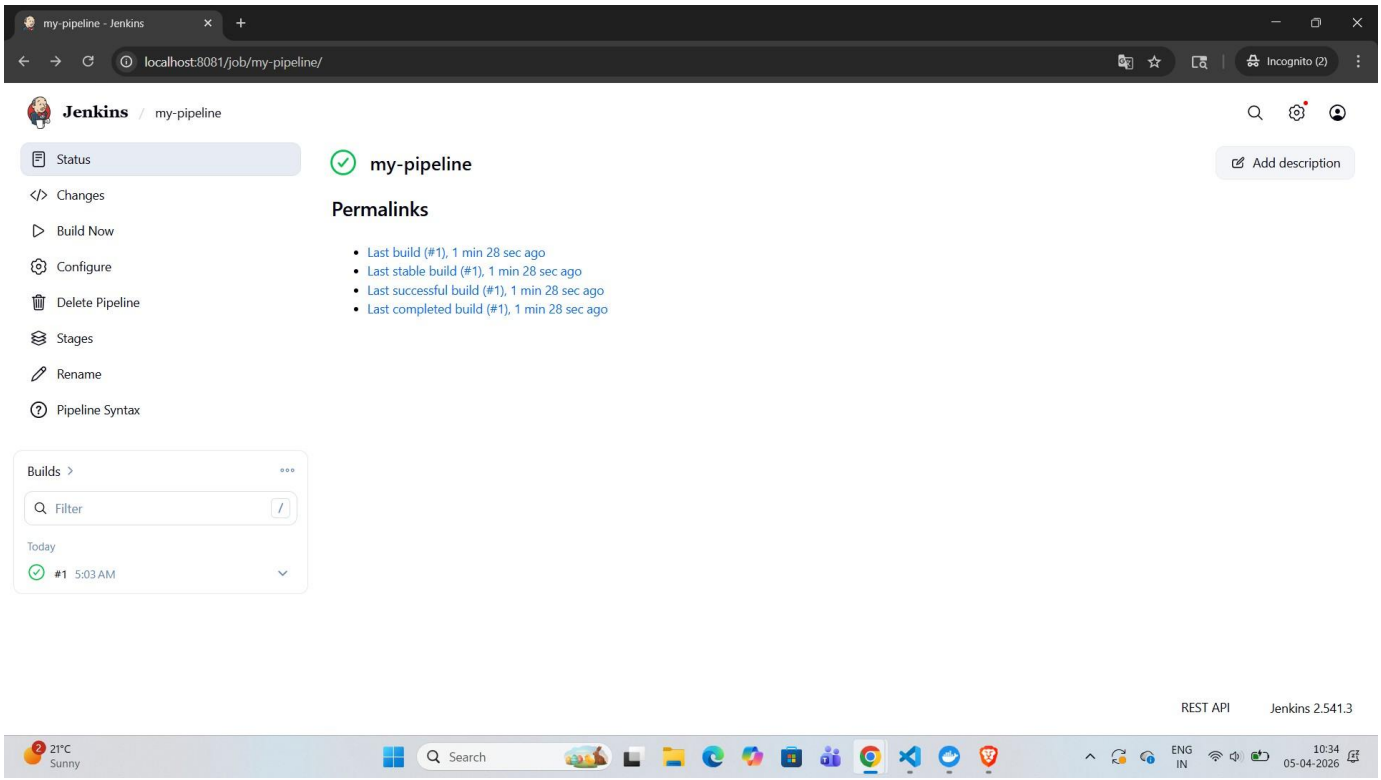
Allows Jenkins container to use the host's Docker daemon directly.

Screenshot – Compose Containers





compose ps



compose ps

Result

Successfully implemented a complete **automated CI/CD pipeline** using Jenkins, GitHub, and Docker Hub.

Observations

- Jenkins GUI simplifies pipeline management
- GitHub + Webhook = true automation
- Docker ensures reproducible builds
- Custom ARM64 Jenkins image is required for Mac M1

Experiment 9

Name: Ashmeet Negi

Course: Containerization and DevOps Lab

🚀 Experiment 9 – Ansible Automation with Docker

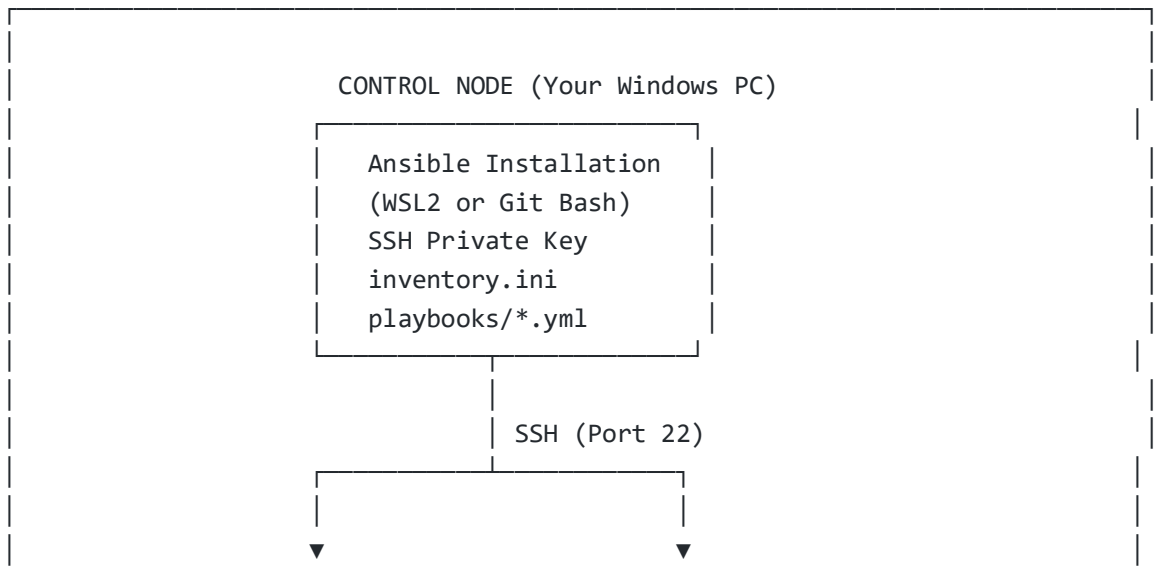
Course: DevOps / Cloud Computing Lab
Objective: Learn Ansible for automated server configuration management using Docker containers as managed nodes on **Windows**

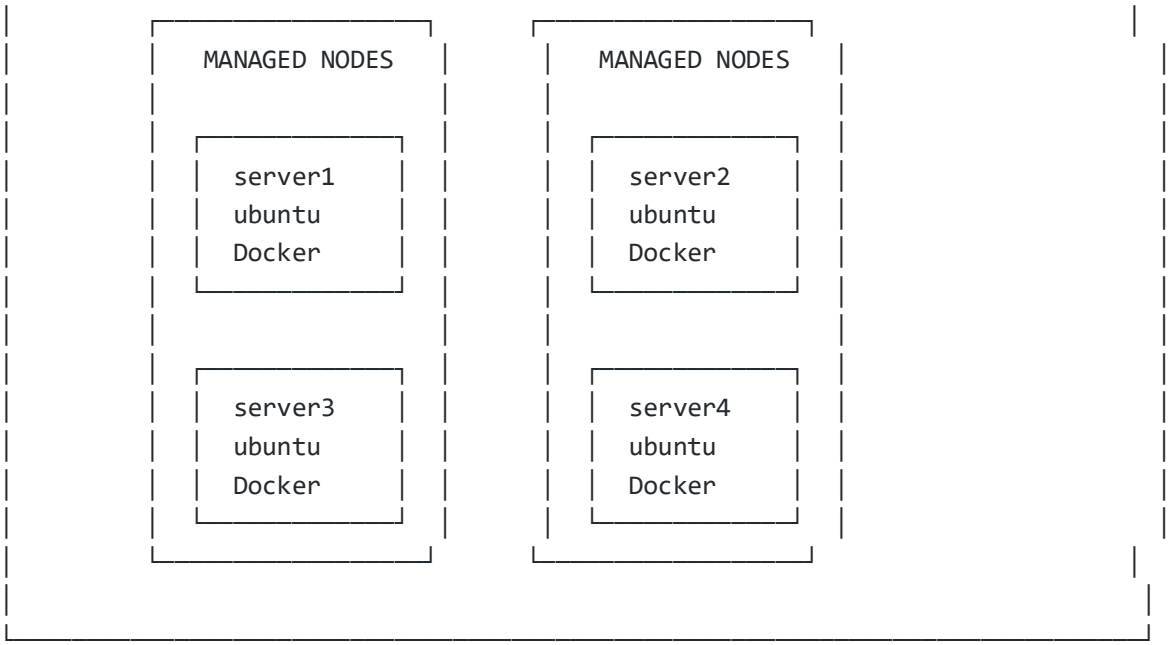
🧠 Theory

What is Ansible?

```
╔═══════════════════════════════════════════════════════════════════════════════════════════════════════════════════════════════╗
═══╗ ‖ ‖ ‖ 🔧 ANSIBLE – The Automation Engine ‖ ‖ ‖ ‖ • Configuration Management • Application Deployment ‖ ‖ • Orchestration • Multi-server Workflows ‖ ‖ ‖ ‖ "Agentless • YAML-based • Idempotent • Push-based" ‖ ‖ ‖
╚═══════════════════════════════════════════════════════════════════════════════════════════════════════════════════════════════╝
═══╝
```

How Ansible Works





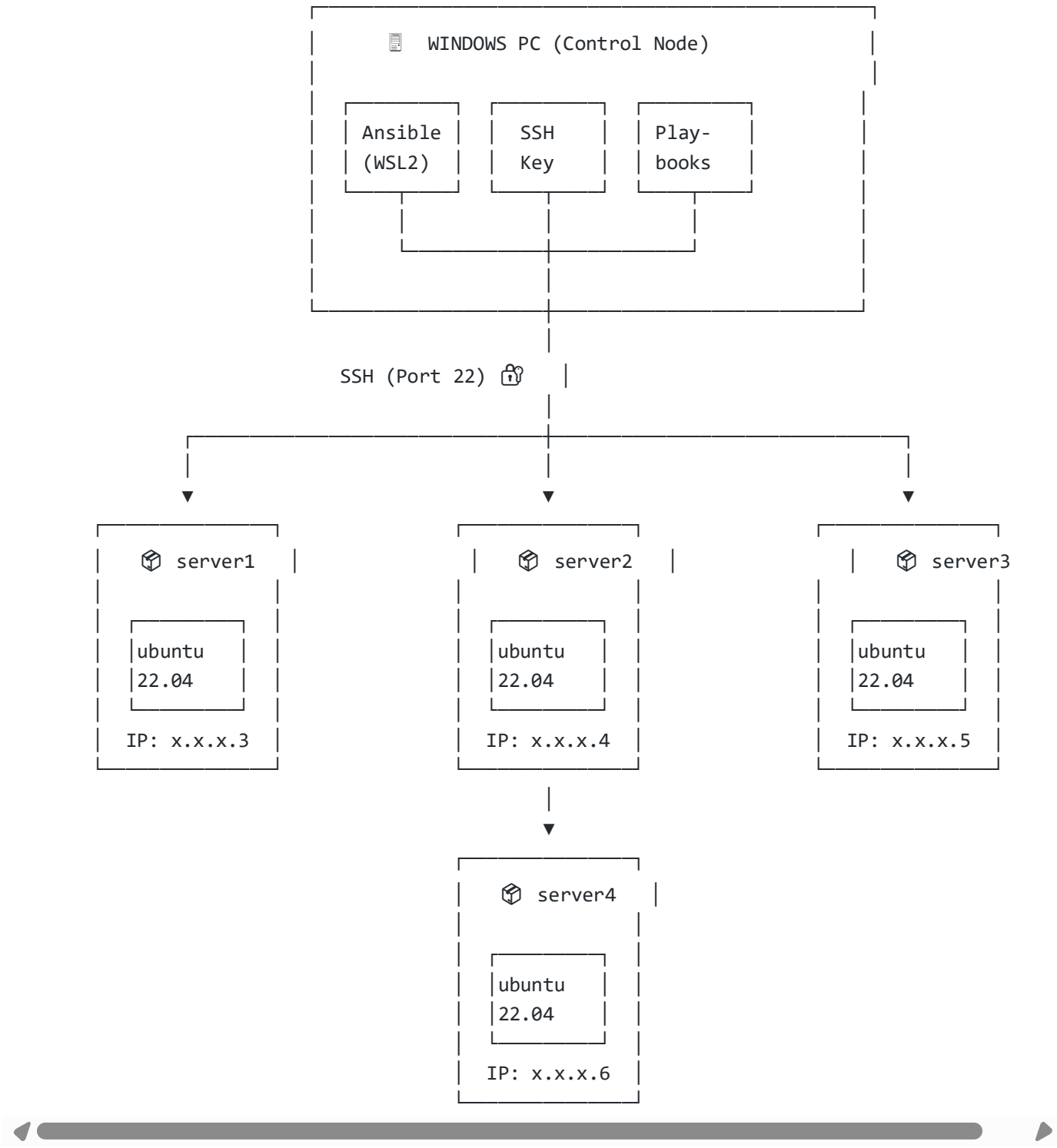
Key Components at a Glance

🧩 Component	📝 Description	🎯 Analogy
Control Node	Machine with Ansible installed (your Windows PC via WSL2)	🎮 The Commander
Managed Nodes	Target servers — no agent needed	🎯 The Targets
Inventory	`inventory.ini` — lists all nodes	📋 Guest List
Playbook	YAML file with automation steps	📜 Recipe Book
Task	Individual action in a playbook	🔧 Single Step
Module	Built-in functions (`apt` , `copy` , etc.)	🛠️ Toolbox
Role	Reusable automation scripts	📦 Pre-packaged Kit

Why Ansible Stands Out

| ✨ Feature | 💡 Benefit | |:----------|:----------| | **Agentless** | Uses SSH — no software needed on servers | | **Idempotent** | Run playbooks multiple times safely | | **Declarative** | Describe desired state, not the steps | | **Push-based** | Control node initiates all changes | | **YAML Syntax** | Human-readable, easy to learn |

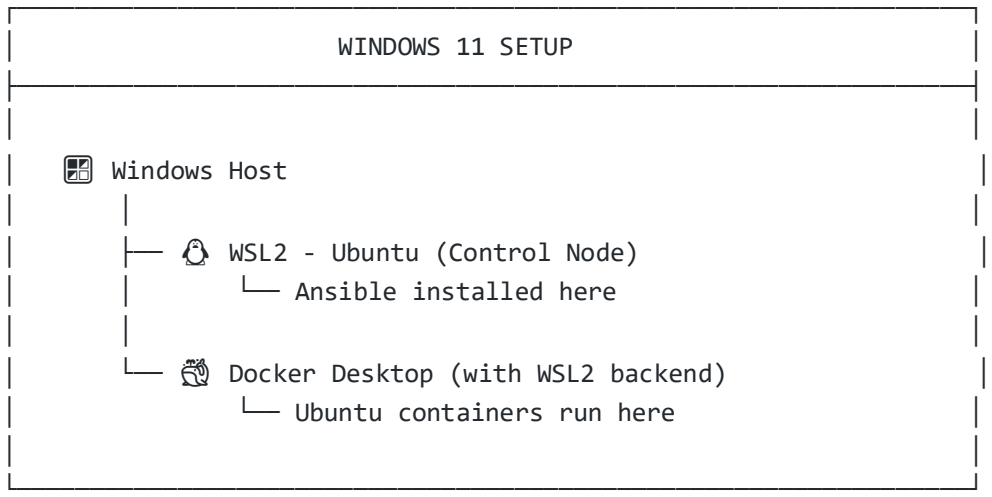
🏛️ Architecture Overview



Prerequisites (Windows)

| ☑ Requirement | 🔍 How to Install | 💻 Check Command | |:---|:---|:---| | **WSL2** (Windows Subsystem for Linux) | `wsl --install` in PowerShell (Admin) | `wsl -l -v` | | **Ubuntu** on WSL2 | Install from Microsoft Store | `wsl -d Ubuntu` | | **Docker Desktop** with WSL2 backend | docker.com | `docker --version` | | **Git for Windows** (includes Git Bash) | git-scm.com | `git --version` | | **VS Code** (recommended) | code.visualstudio.com | `code --version` |

🎯 Recommended Setup Approach



🔐 Part A – Setup & SSH Configuration

Step 1: Launch WSL2 Ubuntu

```
# In PowerShell (as Administrator)
wsl --update wsl --set-default-
version 2

# Launch Ubuntu
wsl -d Ubuntu
```

Step 2: Install Ansible in WSL2

```
# Update package list
sudo apt update

# Install Ansible sudo apt
install ansible -y
# Verify installation
```

```
ansible       --version
```

Expected output:

```
ansible [core 2.x.x]   config file =
/etc/ansible/ansible.cfg   python
version = 3.x.x
```

Step 3: Generate SSH Key Pair

```
# 🔑 Generate RSA 4096-bit key pair ssh-
keygen -t rsa -b 4096
# Press Enter for all prompts (use default path, no passphrase)

# Create working directory mkdir -p
~/ansible-exp9 && cd ~/ansible-exp9

# Copy keys to project directory
cp ~/.ssh/id_rsa.pub . cp
~/.ssh/id_rsa .

# Verify
ls -la
```

Key placement guide:

File	Location	Purpose
`id_rsa` (Private)	WSL2 (`~/.ssh/id_rsa`)	Used by Ansible/SSH to authenticate
`id_rsa.pub` (Public)	Docker container (`~/.ssh/authorized_keys`)	Grants access to matching private key

Step 4: Create the Dockerfile

```
cd ~/ansible-exp9
```

Create `Dockerfile` :

```
FROM ubuntu:22.04

# Install required packages
RUN apt update -y && \     apt install -y python3 python3-
pip openssh-server && \     apt clean

# Create SSH runtime directory
RUN mkdir -p /var/run/sshd

# Configure SSH for key-based authentication RUN mkdir -p /run/sshd && \     echo
'root:password' | chpasswd && \     sed -i 's/#PermitRootLogin prohibit-
password/PermitRootLogin yes/' /etc/ssh/sshd_confi     sed -i 's/#PasswordAuthentication
yes/PasswordAuthentication no/' /etc/ssh/sshd_config     sed -i 's/#PubkeyAuthentication
yes/PubkeyAuthentication yes/' /etc/ssh/sshd_config

# Setup SSH directory and permissions
RUN mkdir -p /root/.ssh && \
chmod 700 /root/.ssh
```

```
# Copy SSH keys
COPY id_rsa /root/.ssh/id_rsa
COPY id_rsa.pub /root/.ssh/authorized_keys

# Set proper permissions
RUN chmod 600 /root/.ssh/id_rsa && \
chmod 644 /root/.ssh/authorized_keys

# Fix PAM for SSH login
RUN sed -i 's@session\s*required\s*pam_loginuid.so@session optional pam_loginuid.so@g' /et

EXPOSE 22

CMD ["/usr/sbin/sshd", "-D"]
```

Step 5: Build Docker Image

```
# Build the custom ubuntu-server image
docker build -t ubuntu-server .

# Verify image was created docker
images | grep ubuntu-server
```

Step 6: Test SSH with Single Container

```
# Start a test container docker run -d --rm -p 2222:22 --name ssh-
test-server ubuntu-server
# Get container IP docker inspect -f
'' ssh-test-server

# Test SSH login with key (no password should be prompted) ssh -i
~/.ssh/id_rsa -o StrictHostKeyChecking=no -p 2222 root@localhost

# Once logged in, verify
whoami hostname

# Exit and stop
exit
docker stop ssh-test-server
```

Part B – Ansible with Docker Servers

Step 7: Launch 4 Server Containers

```
for i in {1..4}; do   echo -e "\n📦 Creating server${i}\n"   docker
run -d --rm -p 220${i}:22 --name server${i} ubuntu-server   echo -e
"📍 IP of server${i} is $(docker inspect -f '' server${i})" done

# Verify all 4 are running
docker ps
```

Expected output:

```
CONTAINER ID    IMAGE              NAMES       STATUS            PORTS abc123...
ubuntu-server      server1      Up 5 seconds      0.0.0.0:2201->22/tcp def456...
ubuntu-server      server2      Up 5 seconds      0.0.0.0:2202->22/tcp ghi789...
ubuntu-server      server3      Up 5 seconds      0.0.0.0:2203->22/tcp jkl012...
ubuntu-server    server4    Up 5 seconds    0.0.0.0:2204->22/tcp
```

Step 8: Create Ansible Inventory

```
# Auto-generate inventory.ini with container IPs
echo "[servers]" > inventory.ini for i in {1..4};
do   docker inspect -f '' server${i} >>
inventory.ini done
# Add Ansible connection variables
cat << EOF >> inventory.ini

[servers:vars] ansible_user=root
ansible_ssh_private_key_file=/home/$(whoami)/.ssh/id_rsa
ansible_python_interpreter=/usr/bin/python3
ansible_ssh_common_args='-o StrictHostKeyChecking=no' EOF

# Review the inventory
cat inventory.ini
```

Expected `inventory.ini` :

```
[servers]
172.17.0.3
172.17.0.4
172.17.0.5
172.17.0.6

[servers:vars] ansible_user=root
ansible_ssh_private_key_file=/home/username/.ssh/id_rsa
ansible_python_interpreter=/usr/bin/python3
ansible_ssh_common_args='-o StrictHostKeyChecking=no'
```

Step 9: Test Ansible Connectivity (Ping All)

```
# Disable host key checking export
ANSIBLE_HOST_KEY_CHECKING=False

# Ping all servers ansible all -i
```

inventory.ini -m ping **Expected output:**

```
172.17.0.3 | SUCCESS => {
    "ansible_facts": {
        "discovered_interpreter_python": "/usr/bin/python3"
    },
    "changed": false,
    "ping": "pong"
}
172.17.0.4 | SUCCESS => { ... }
172.17.0.5 | SUCCESS => { ... }
172.17.0.6 | SUCCESS => { ... }
```

For verbose output (useful for debugging):

```
ansible all -i inventory.ini -m ping -vvv
```

Playbooks

Playbook 1: Update + Install Packages + Create File

Create update.yml :

```
---
- name: Update and configure servers
  hosts: all   become: yes

  tasks:
- name: Update apt packages       apt:
        update_cache: yes
upgrade: dist

- name: Install required packages
  apt:
        name: ["vim", "htop", "wget", "curl"]
state: present
```

```
- name: 📝 Create test file       copy:
        dest: /root/ansible_test.txt
content: |
          ☑️ Configured by Ansible
  📍 Server:
          🕐 Time:
          📆 Date:
```

Playbook 2: Full Configuration with System Info

Create `playbook1.yml` :

```
---
- name: ⚙️ Configure multiple servers
  hosts: servers
  become: yes

  tasks:
- name: 🔄 Update apt package index
  apt:          update_cache: yes

- name: 🌀 Install Python 3 (latest)
  apt:
        name: python3
state: latest

- name: 📄 Create test file with content
  copy:
        dest: /root/test_file.txt
content: |
          ═══════════════════════════════════
          📋 SERVER CONFIGURATION REPORT
          ═══════════════════════════════════
                    📄
Server:
          📆 Date:
          🕐 Time:
          🐧 OS:

          ═══════════════════════════════════

- name: 📄 Display system information
  command: uname -a       register:
  uname_output

- name: 💾 Show disk space       command:
  df -h       register: disk_space
```

```
- name: 📊 Print results       debug:
  msg:
- "🚀 System info: "
- "🛞 Disk space: "
- "🛞 Disk space: "
```

Step 10: Run the Playbooks

```
# Run update.yml echo "🚀 Running update
playbook..." ansible-playbook -i
inventory.ini update.yml

# Run playbook1.yml
echo "🚀 Running configuration playbook..."

ansible-playbook -i inventory.ini playbook1.yml
```

Sample PLAY RECAP:

```
PLAY RECAP *********************************************************************
172.17.0.3    : ok=6  changed=4  unreachable=0  failed=0  skipped=0  rescued=0  ignored=0
172.17.0.4    : ok=6  changed=4  unreachable=0  failed=0  skipped=0  rescued=0  ignored=0
172.17.0.5    : ok=6  changed=4  unreachable=0  failed=0  skipped=0  rescued=0  ignored=0
172.17.0.6    : ok=6  changed=4  unreachable=0  failed=0  skipped=0  rescued=0  ignored=0
```

☑ Verification

Step 11: Verify Changes on All Servers

```
# Check test file via Ansible echo "📋 Verifying test files on all
servers..." ansible all -i inventory.ini -m command -a "cat
/root/test_file.txt"

# Manually verify via Docker exec echo "🔍 Manual
verification via Docker exec:" for i in {1..4}; do
echo ""   echo
"━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━"   echo
"  📦 server${i}"   echo
"━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━"
docker exec server${i} cat /root/test_file.txt
done

# Verify installed packages echo "☑
Verifying installed packages..."
```

```
ansible all -i inventory.ini -m command -a "which vim htop wget curl"
```

🎯 Bonus: Ad-hoc Ansible Commands

```
# Check uptime on all servers ansible all -i
inventory.ini -m command -a "uptime"

# Check OS info ansible all -i inventory.ini -m
command -a "uname -a"

# Check memory usage
ansible all -i inventory.ini -m command -a "free -h"

# Create a user on all servers
ansible all -i inventory.ini -m user -a "name=devops state=present"

# List available modules ansible-
doc -l | head -20

# View specific module docs
ansible-doc apt ansible-doc
copy
```

🔧 Optional Part C – Local nginx Install (WSL2 Ubuntu)

Create local inventory and playbook

```
# Create local inventory for WSL2
cat << EOF > local_inventory.ini
[local]
localhost ansible_connection=local
EOF

# Create nginx playbook cat <<
EOF > install_nginx.yml
---
-    name: 🌐 Install Nginx on localhost
(WSL2)   hosts: local   become: yes
tasks:
-    name: 📥 Install nginx package
apt:
        name: nginx
state: present
```

```
-    name: ▶ Start nginx service
service:
        name: nginx
state: started
enabled: yes
-    name: ℹ Get nginx status
command: systemctl status nginx
register: nginx_status      - name:
Display nginx status        debug:
msg: ""
EOF
# Run it
ansible-playbook -i local_inventory.ini install_nginx.yml
```

Using Ansible Vault (for secrets)

```
# Create an encrypted secrets file
ansible-vault create secrets.yml

# View encrypted file ansible-
vault view secrets.yml

# Edit encrypted file ansible-
vault edit secrets.yml

# Use with playbook
ansible-playbook -i inventory.ini playbook1.yml --ask-vault-pass
```

Install Ansible Collections

```
# Install a collection from Ansible Galaxy ansible-galaxy
collection install community.general

# List installed collections ansible-
galaxy collection list
```

Cleanup

```
# Stop and remove all server containers
for i in {1..4}; do   echo "  Stopping
```

```
server${i}..."   docker stop server${i}
done

# Remove project keys (optional) rm ~/ansible-
exp9/id_rsa ~/ansible-exp9/id_rsa.pub

# Remove docker image (optional)
docker rmi ubuntu-server

# Exit WSL2 (if done)
exit
```

Windows + WSL2 – Common Issues & Fixes

⚠ Issue	Cause	Fix
SSH fingerprint prompt blocks Ansible	First-time SSH	`export ANSIBLE_HOST_KEY_CHECKING=False`
Docker not found in WSL2	Docker Desktop not integrated	Enable WSL2 integration in Docker Desktop settings
Permission denied on `id_rsa`	Wrong file permissions	`chmod 600 ~/.ssh/id_rsa`
Ansible can't find Python on nodes	Wrong interpreter path	Set `ansible_python_interpreter=/usr/bin/python3` in inventory
WSL2 can't ping Windows localhost	Network isolation	Use container IPs instead of localhost
Docker daemon not running	Docker Desktop not started	Start Docker Desktop from Windows Start menu
`ansible` command not found	Ansible not installed in WSL2	Run `sudo apt install ansible -y`
SSH connection refused	Container not running	Check `docker ps` and restart containers

Key Concepts Summary

Ansible Workflow

```
Step 1: 🔑 SSH Keys      → Generate and copy keys to containers
Step 2: 🏗️ Build Image    → Create ubuntu-server with SSH
Step 3: 🚀 Launch        → 4 containers as managed nodes
Step 4: 📋 Inventory     → List all server IPs
Step 5: 🏓 Test Ping     → Verify connectivity
Step 6: ✍️ Write Playbook → Define automation tasks
Step 7: ▶️ Run Playbook   → Execute across all servers
Step 8: ☑️ Verify        → Check results
Step 9: 🧹 Cleanup       → Remove containers
```

Idempotency Demo

Running the same playbook twice:

Run	Changed Count	Explanation
First run	`changed=4`	Packages installed, files created
Second run	`changed=0`	Already in desired state — no changes made

> ✨ **Idempotency:** Safe to run multiple times without unintended side effects!

📚 References

Resource	Link
Official Ansible Website	ansible.com
Ansible Documentation	docs.ansible.com
Ansible for Windows	docs.ansible.com/ansible/latest/os_guide/windows_faq.html
WSL2 Documentation	learn.microsoft.com/en-us/windows/wsl/
Docker WSL2 Backend	docs.docker.com/desktop/wsl/
Ansible Galaxy	galaxy.ansible.com

📸 Screenshots Checklist

📊 Result

| ☑ | Outcome | |:--:|:--------| | 🎯 | **SUCCESS** - The experiment was completed successfully |
Using Ansible as a configuration management and automation tool, a control node (Windows PC with WSL2/Ubuntu) was configured to manage 4 Docker containers acting as remote servers. SSH key-based authentication was established between the control node and all managed nodes. An Ansible inventory file was created listing all target servers, and connectivity was verified using the `ansible ping` module — **all 4 servers returned SUCCESS**.

Two YAML-based playbooks were written and executed, which:

- Automatically updated apt packages
- Installed software packages (`vim` , `htop` , `wget` , `curl`)
- Created configuration files with dynamic content using Ansible variables
- Collected system information across all servers simultaneously

All without logging into any server manually! 🚀

🎓 Learning Outcomes

After completing this experiment, students are able to:

#	Learning Outcome
1	🧠 **Understand** the architecture of Ansible — including the roles of the control node, managed nodes, inventory, modules, tasks, and playbooks, and how they work together in an agentless, SSH-based automation model
2	🔐 **Set up** SSH key-based authentication between a control machine and multiple remote servers, and understand why this is essential for automated, passwordless server management
3	📝 **Write and interpret** Ansible inventory files (`inventory.ini`) to define and group managed nodes with connection variables
4	✍ **Write YAML-based Ansible playbooks** to automate real-world tasks such as package installation, file creation, and system information gathering across multiple servers
5	🔧 **Use Ansible modules** such as `apt` , `copy` , `command` , and `debug` to perform common system administration tasks declaratively

6	🔄 **Demonstrate idempotency** — understanding that running the same playbook multiple times produces the same result without unintended side effects
7	⚡ **Execute ad-hoc Ansible commands** for quick, one-off tasks without writing a full playbook
8	🐳 **Use Docker containers** as simulated servers to practice multi-node infrastructure management in a local environment without requiring real cloud VMs
9	💾 **Recognize the practical value** of Infrastructure as Code (IaC) — how versioncontrolled, declarative configuration files eliminate configuration drift and enable consistent, repeatable deployments at scale
#	**Learning Outcome**
10	🪟 **Configure Ansible on Windows** using WSL2, bridging the gap between Windows development environments and Linux-based automation

--- * 🔧 Experiment 9 | Ansible Automation | DevOps Lab* * 🐧 Windows + WSL2 + Docker Setup*

Experiment 10

Name: Ashmeet Negi

Course: Containerization and DevOps Lab

Lab 10 — SonarQube: Continuous Code Quality Inspection

📌 Objective

To set up SonarQube for continuous code quality inspection, analyze a Java application for bugs, vulnerabilities, and code smells, integrate it into a CI/CD pipeline using Jenkins, and understand how Quality Gates enforce code standards before deployment.

📖 Theory

1.1 What is SonarQube?

SonarQube is an open-source platform for **continuous inspection of code quality**. It performs automatic static analysis to detect:

Issue Type	Description
Bugs	Code that will break or behave unexpectedly
Vulnerabilities	Security-related issues (e.g., SQL injection)
Code Smells	Maintainability issues that slow development
Duplications	Repeated code blocks

Coverage	% of code covered by unit tests
Technical Debt	Estimated time required to fix all issues

1.2 Architecture

```
[ Your Code ]
      ↓ [ Sonar Scanner ]  →  Analyzes code
locally
      ↓ [ SonarQube Server ]  →  Stores results + shows
dashboard
      ↓
[ PostgreSQL DB ]  →  Persists all analysis data
```

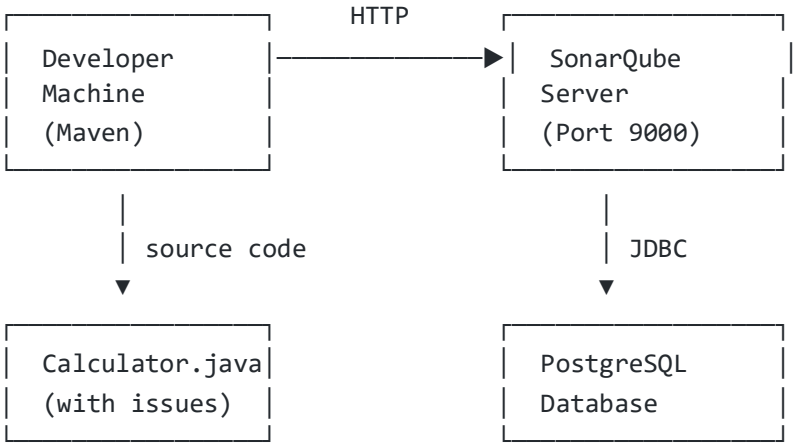
1.3 Key Components

Component	Role	Analogy
SonarQube Server	Stores & displays results	Teacher / Examiner
Sonar Scanner	Analyzes and sends results	Student writing exam
Source Code	What gets analyzed	Answer sheet

1.4 Quality Gate

A **Quality Gate** is a set of conditions that code must satisfy before it can be deployed. If the gate fails, the CI/CD pipeline is blocked — ensuring bad code never reaches production.

🏗️ Lab Architecture



🗂 Project Structure

```
Lab-10/
├── docker-compose.yml                          # SonarQube + PostgreSQL setup
└── sample-java-app/
    ├── pom.xml                                 # Maven build + Sonar plugin
    ├── Jenkinsfile                             # CI/CD pipeline definition
├── sonar-project.properties                  # Scanner configuration
    └── src/
        └── main/
            └── java/
                └── com/
                    └── example/
                        └── Calculator.java     # Sample app with intentional issues
```

🛠 Setup & Commands

Step 1 — Start SonarQube Environment

```
docker compose up -d
```

docker-compose.yml:

```
version: '3.8'

services:   sonar-
db:
    image: postgres:13
container_name: sonar-db
environment:
      POSTGRES_USER: sonar
      POSTGRES_PASSWORD: sonar
POSTGRES_DB: sonarqube     volumes:
- sonar-db-data:/var/lib/postgresql/data
  networks:       - sonarqube-lab

  sonarqube:
    image: sonarqube:lts-community
container_name: sonarqube
ports:       - "9000:9000"
environment:
      SONAR_JDBC_URL: jdbc:postgresql://sonar-db:5432/sonarqube
      SONAR_JDBC_USERNAME: sonar
SONAR_JDBC_PASSWORD: sonar
volumes:
- sonar-data:/opt/sonarqube/data
```

```
    - sonar-extensions:/opt/sonarqube/extensions
      depends_on:       - sonar-db     networks:
      - sonarqube-lab

    volumes:   sonar-db-
    data:   sonar-data:
      sonar-extensions:

    networks:   sonarqube-
    lab:     driver: bridge
```

> **Access SonarQube at:** http://localhost:9000
> **Default login:** `admin / admin`

Step 2 — Sample Java Application (with Intentional Issues)

Calculator.java — contains bugs, vulnerabilities, and code smells for analysis:

```
package com.example;

public class Calculator {

    // BUG: Division by zero not handled
public int divide(int a, int b) {
return a / b;     }

    // CODE SMELL: Unused variable     public
int add(int a, int b) {         int result = a
+ b;         int unused = 100;   // unused
variable         return result;     }

    // VULNERABILITY: SQL Injection risk
public String getUser(String userId) {
        String query = "SELECT * FROM users WHERE id = " + userId;
return query;     }

    // CODE SMELL: Duplicate code block     public int
multiply(int a, int b) {         int result = 0;         for
(int i = 0; i < b; i++) { result = result + a; }
        return result;
    }

    public int multiplyAlt(int a, int b) {         int result
= 0;         for (int i = 0; i < b; i++) { result = result +
a; }         return result;     }

    // CODE SMELL: Too many parameters
    public void processUser(String name, String email, String phone,
                            String address, String city, String state,
                            String zip, String country) {
```

```
        System.out.println("Processing: " + name);
    }

    // BUG: NullPointerException if name is null
public String getName(String name) {
return name.toUpperCase();     }

    // CODE SMELL: Empty catch block (swallowing exception)
public void riskyOperation() {         try {             int
x = 10 / 0;
        } catch (Exception e) {
            // swallowed — bad practice
        }
    }
}
```

Step 3 — Maven Configuration

pom.xml:

```
<properties>
    <maven.compiler.source>11</maven.compiler.source>
    <maven.compiler.target>11</maven.compiler.target>
    <sonar.projectKey>sample-java-app</sonar.projectKey>
    <sonar.projectName>Sample Java Application</sonar.projectName>
<sonar.host.url>http://localhost:9000</sonar.host.url>
</properties>

<build>
  <plugins>
    <plugin>
      <groupId>org.sonarsource.scanner.maven</groupId>
      <artifactId>sonar-maven-plugin</artifactId>
      <version>3.9.1.2184</version>
    </plugin>
  </plugins>
</build>
```

Step 4 — Run SonarQube Analysis

```
# Compile the project
mvn clean compile

# Run the scan (replace with your generated token) mvn
sonar:sonar -Dsonar.login=YOUR_SONAR_TOKEN
```

> **Generate a token:** SonarQube → My Account → Security → Generate Token

Step 5 — Jenkins CI/CD Integration

Jenkinsfile:

```
pipeline {
agent any

    environment {
        SONAR_HOST_URL = 'http://sonarqube:9000'
SONAR_TOKEN = credentials('sonar-token')
    }

    stages {
stage('Checkout') {
steps { checkout scm }
        }

        stage('SonarQube Analysis') {
steps {                 withSonarQubeEnv('SonarQube')
{                     sh 'mvn clean verify
sonar:sonar'
                }
            }
}

        stage('Quality Gate') {             steps {
timeout(time: 1, unit: 'HOURS') {
waitForQualityGate abortPipeline: true
                }
            }
        }
        stage('Build') {
steps { sh 'mvn package' }
        }

        stage('Deploy') {             steps {
sh 'docker build -t sample-app .'
                sh 'docker run -d -p 8080:8080 sample-app'
            }
        }
    }
}
```

Step 6 — API Verification

```
# Fetch all bugs curl -u
admin:admin123 \
  "http://localhost:9000/api/issues/search?projectKeys=sample-java-app&types=BUG"

# Fetch vulnerabilities
curl -u admin:admin123 \
  "http://localhost:9000/api/issues/search?projectKeys=sample-java-app&types=VULNERABILITY

# Full metrics summary (pretty printed)
curl -u admin:admin123 \
  "http://localhost:9000/api/measures/component?component=sample-java-app&metricKeys=bugs,
| python3 -m json.tool
```

📊 Analysis Results

Before Fix

Metric	Count
Bugs	2
Vulnerabilities	1
Code Smells	5+
Duplications	2 blocks
Test Coverage	0%
Metric	**Count**
Technical Debt	~2 hours

After Fixing Divide-by-Zero Bug

Fixed code:

```
// Fixed: Division by zero now handled public int divide(int a, int
b) {     if (b == 0) {         throw new
IllegalArgumentException("Cannot divide by zero");
    }
    return a / b;
}
```

Re-run scan:

```
mvn clean compile
mvn sonar:sonar -Dsonar.login=YOUR_SONAR_TOKEN
```

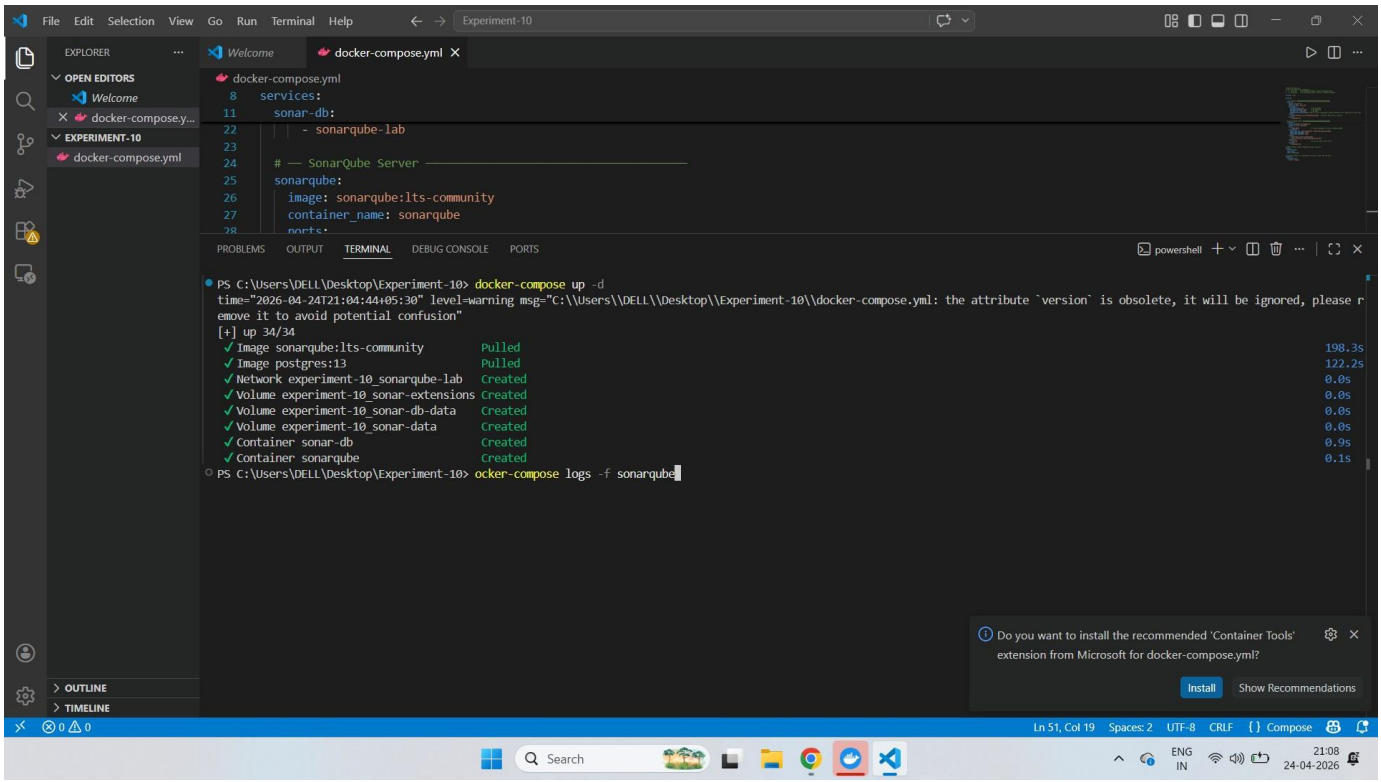
Metric	Before	After Fix
Bugs	2	1 ☑
Vulnerabilities	1	1
Code Smells	5+	5+

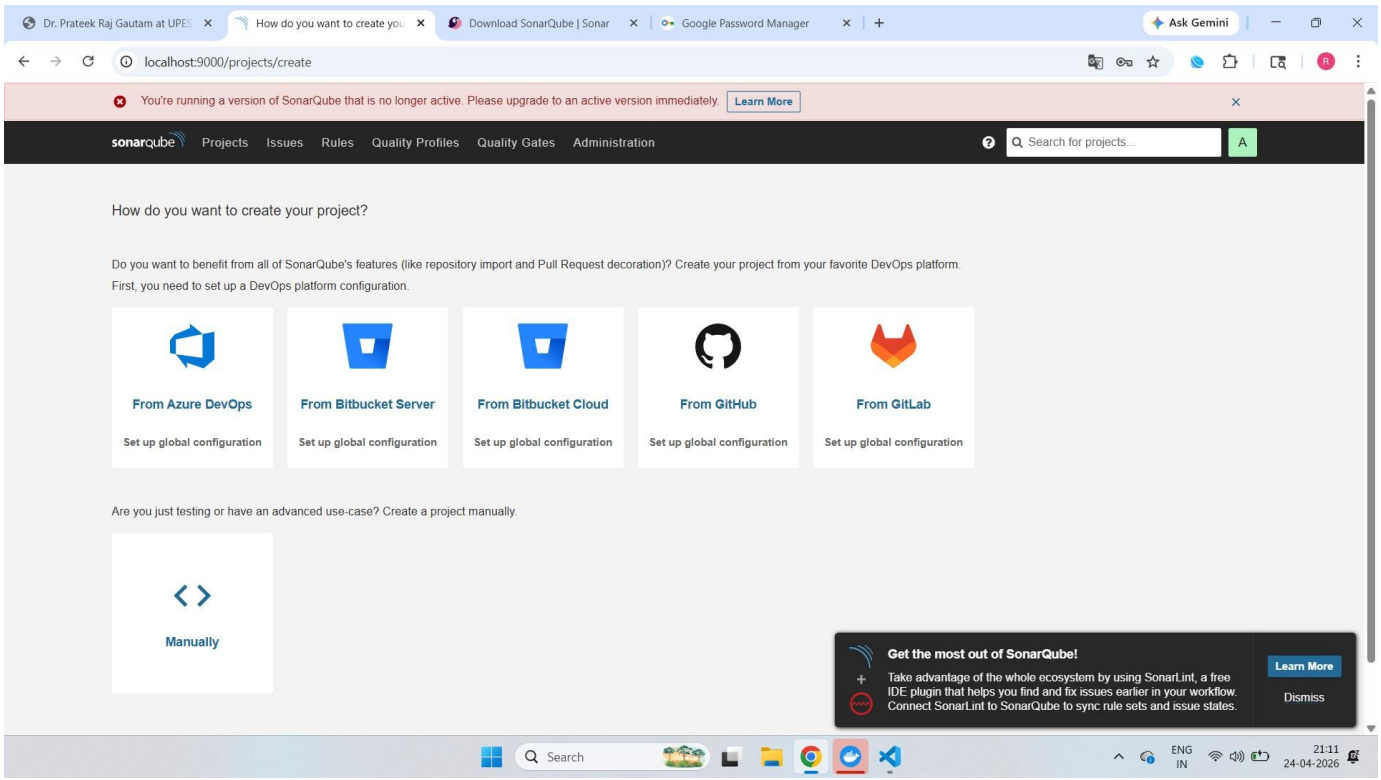
> The dashboard reflects the reduced bug count after re-scan — demonstrating SonarQube's continuous feedback loop.

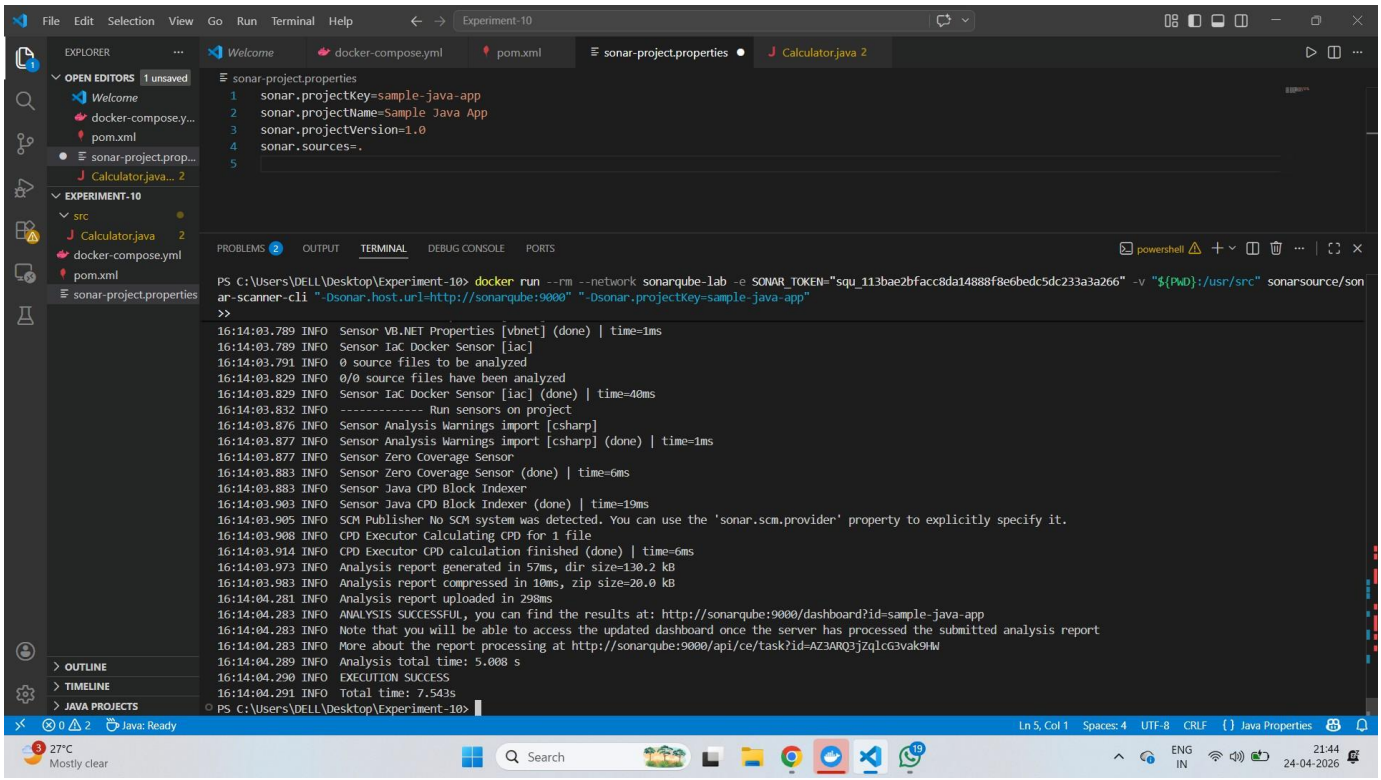
🔄 CI/CD Flow with Quality Gate

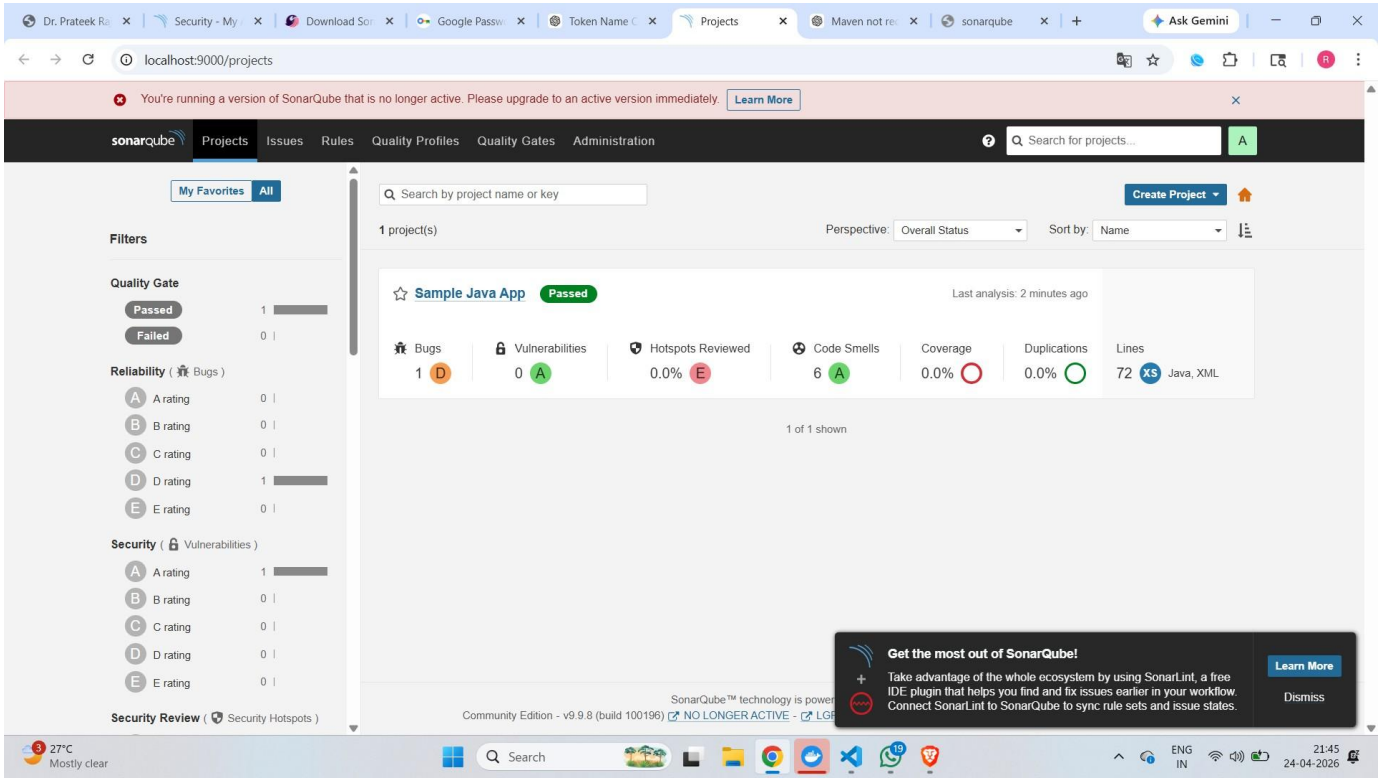
```
Developer commits code
        ↓
Jenkins triggers pipeline
↓ mvn clean verify
sonar:sonar         ↓
SonarQube analyzes code
        ↓
Quality Gate evaluated
    ↙           ↘
PASS              FAIL
  ↓                 ↓
 Build continues   Pipeline blocked
 Deploy to server  Fix issues first
```

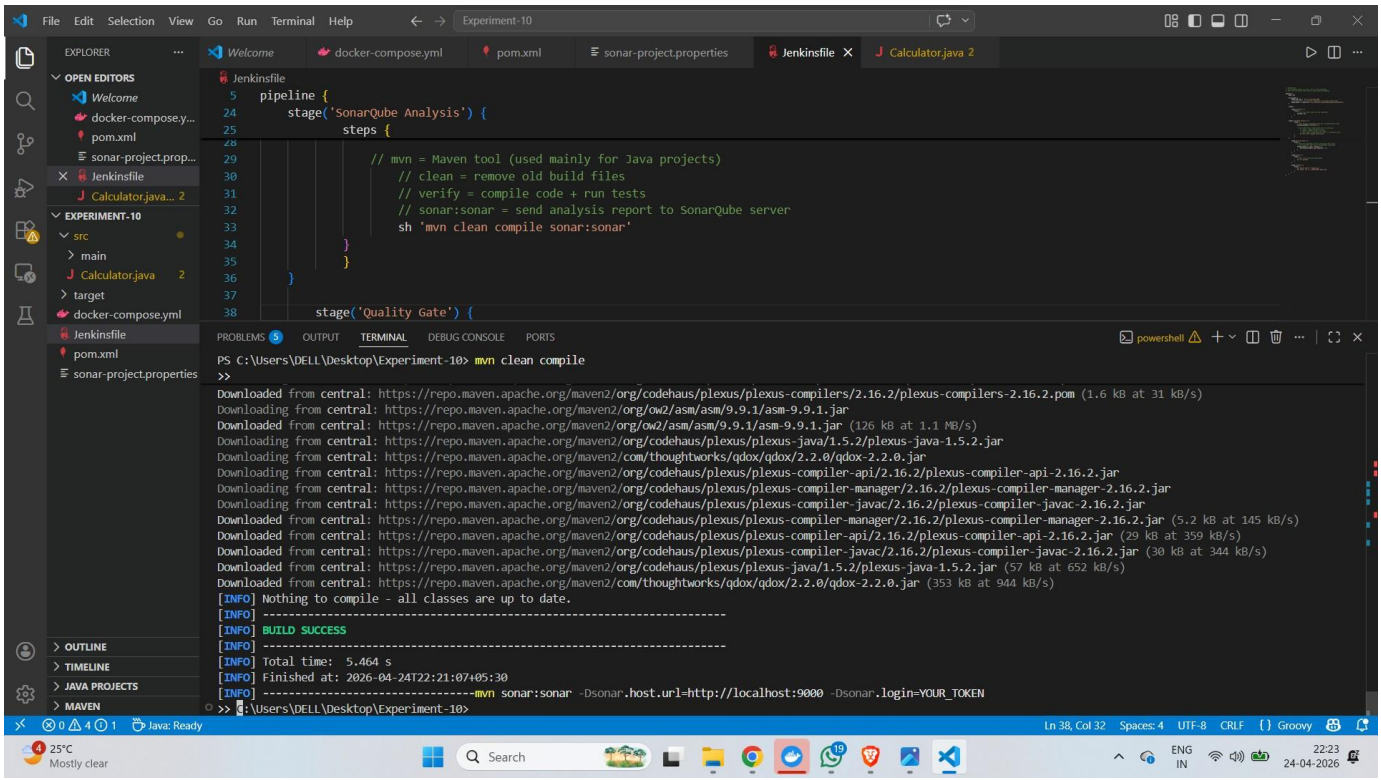
📸 Screenshots

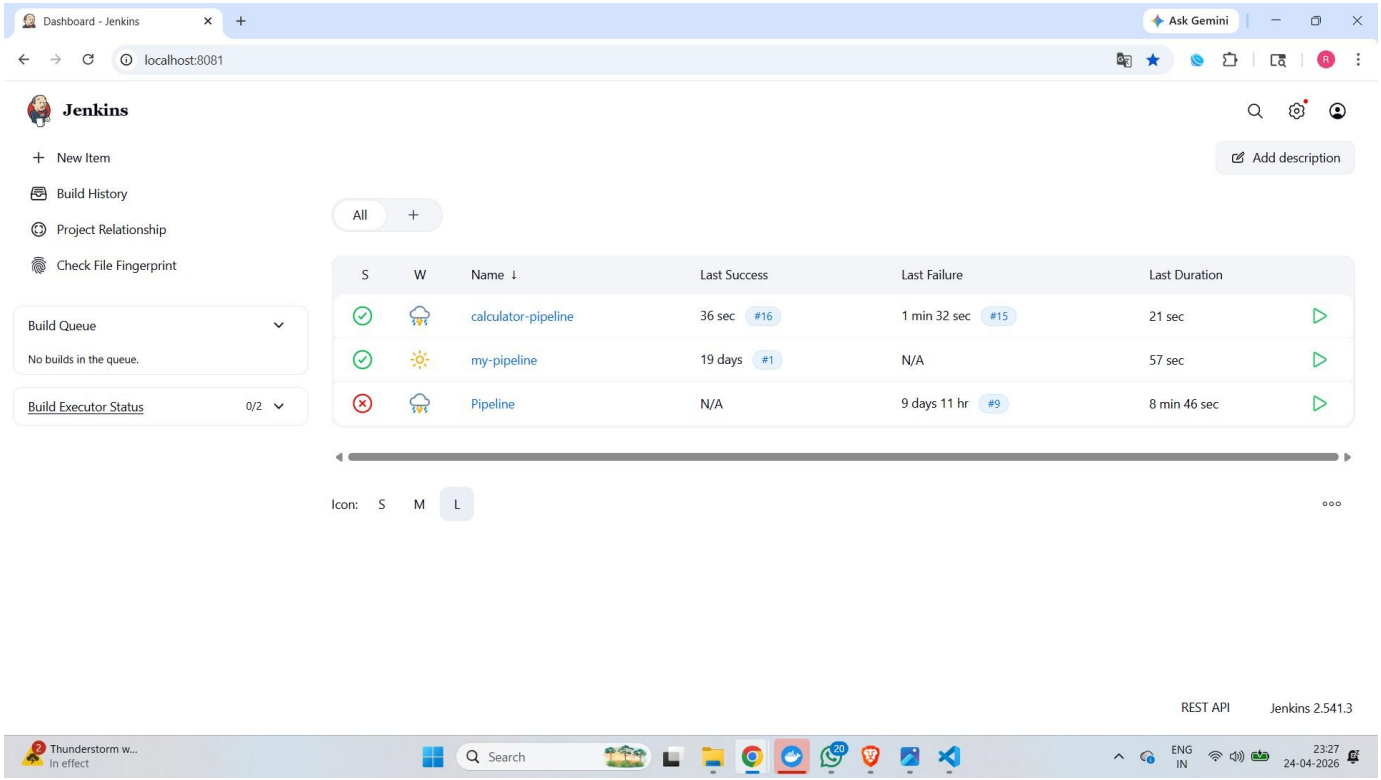

File Edit Selection View Go Run Terminal Help
Experiment-10
EXPLORER
OPEN EDITORS
Welcome
docker-compose.y...
EXPERIMENT-10
docker-compose.yml
Welcome
docker-compose.yml
docker-compose.yml
8 services:
11 sonar-db:
22 - sonarqube-lab
23
24 # — SonarQube Server ——————————————
25 sonarqube:
26 image: sonarqube:lts-community
27 container_name: sonarqube
PROBLEMS OUTPUT TERMINAL DEBUG CONSOLE PORTS
powershell
PS C:\Users\DELL\Desktop\Experiment-10> docker-compose up -d
time="2026-04-24T21:04:44+05:30" level=warning msg="C:\\Users\\DELL\\Desktop\\Experiment-10\\docker-compose.yml: the attribute `version` is obsolete, it will be ignored, please r
emove it to avoid potential confusion"
[+] up 34/34
✓ Image sonarqube:lts-community Pulled 198.3s
✓ Image postgres:13 Pulled 122.2s
✓ Network experiment-10_sonarqube-lab Created 0.0s
✓ Volume experiment-10_sonar-extensions Created 0.0s
✓ Volume experiment-10_sonar-db-data Created 0.0s
✓ Volume experiment-10_sonar-data Created 0.0s
✓ Container sonar-db Created 0.9s
✓ Container sonarqube Created 0.1s
PS C:\Users\DELL\Desktop\Experiment-10> ocker-compose logs -f sonarqube
Do you want to install the recommended 'Container Tools' extension from Microsoft for docker-compose.yml?
Install
Show Recommendations
OUTLINE
TIMELINE
Ln 51, Col 19 Spaces: 2 UTF-8 CRLF { } Compose
Search
ENG IN
21:08 24-04-2026


Dr. Prateek Raj Gautam at UPES
How do you want to create you
Download SonarQube | Sonar
Google Password Manager
Ask Gemini
localhost:9000/projects/create
You're running a version of SonarQube that is no longer active. Please upgrade to an active version immediately.
Learn More
sonarqube
Projects Issues Rules Quality Profiles Quality Gates Administration
Search for projects...
A
How do you want to create your project?
Do you want to benefit from all of SonarQube's features (like repository import and Pull Request decoration)? Create your project from your favorite DevOps platform.
First, you need to set up a DevOps platform configuration.
From Azure DevOps
Set up global configuration
From Bitbucket Server
Set up global configuration
From Bitbucket Cloud
Set up global configuration
From GitHub
Set up global configuration
From GitLab
Set up global configuration
Are you just testing or have an advanced use-case? Create a project manually.
Manually
Get the most out of SonarQube!
Take advantage of the whole ecosystem by using SonarLint, a free IDE plugin that helps you find and fix issues earlier in your workflow. Connect SonarLint to SonarQube to sync rule sets and issue states.
Learn More
Dismiss
Search
ENG IN
21:11 24-04-2026


File Edit Selection View Go Run Terminal Help
Experiment-10
EXPLORER
OPEN EDITORS 1 unsaved
Welcome
docker-compose.yml
pom.xml
sonar-project.properties
Calculator.java 2
EXPERIMENT-10
src
Calculator.java 2
docker-compose.yml
pom.xml
sonar-project.properties
sonar-project.properties
1 sonar.projectKey=sample-java-app
2 sonar.projectName=Sample Java App
3 sonar.projectVersion=1.0
4 sonar.sources=.
5
PROBLEMS 2 OUTPUT TERMINAL DEBUG CONSOLE PORTS
powershell
PS C:\Users\DELL\Desktop\Experiment-10> docker run --rm --network sonarqube-lab -e SONAR_TOKEN="squ_113bae2bfacc8da14888f8e6bedc5dc233a3a266" -v "${PWD}:/usr/src" sonarsource/sonar-scanner-cli "-Dsonar.host.url=http://sonarqube:9000" "-Dsonar.projectKey=sample-java-app"
>>
16:14:03.789 INFO Sensor VB.NET Properties [vbnet] (done) | time=1ms
16:14:03.789 INFO Sensor IaC Docker Sensor [iac]
16:14:03.791 INFO 0 source files to be analyzed
16:14:03.829 INFO 0/0 source files have been analyzed
16:14:03.829 INFO Sensor IaC Docker Sensor [iac] (done) | time=40ms
16:14:03.832 INFO ------------ Run sensors on project
16:14:03.876 INFO Sensor Analysis Warnings import [csharp]
16:14:03.877 INFO Sensor Analysis Warnings import [csharp] (done) | time=1ms
16:14:03.877 INFO Sensor Zero Coverage Sensor
16:14:03.883 INFO Sensor Zero Coverage Sensor (done) | time=6ms
16:14:03.883 INFO Sensor Java CPD Block Indexer
16:14:03.903 INFO Sensor Java CPD Block Indexer (done) | time=19ms
16:14:03.905 INFO SCM Publisher No SCM system was detected. You can use the 'sonar.scm.provider' property to explicitly specify it.
16:14:03.908 INFO CPD Executor Calculating CPD for 1 file
16:14:03.914 INFO CPD Executor CPD calculation finished (done) | time=6ms
16:14:03.973 INFO Analysis report generated in 57ms, dir size=130.2 kB
16:14:03.983 INFO Analysis report compressed in 10ms, zip size=20.0 kB
16:14:04.281 INFO Analysis report uploaded in 298ms
16:14:04.283 INFO ANALYSIS SUCCESSFUL, you can find the results at: http://sonarqube:9000/dashboard?id=sample-java-app
16:14:04.283 INFO Note that you will be able to access the updated dashboard once the server has processed the submitted analysis report
16:14:04.283 INFO More about the report processing at http://sonarqube:9000/api/ce/task?id=AZ3ARQ3jZqlcG3vak9HW
16:14:04.289 INFO Analysis total time: 5.008 s
16:14:04.290 INFO EXECUTION SUCCESS
16:14:04.291 INFO Total time: 7.543s
PS C:\Users\DELL\Desktop\Experiment-10>
OUTLINE
TIMELINE
JAVA PROJECTS
Java: Ready
Ln 5, Col 1 Spaces: 4 UTF-8 CRLF Java Properties
27°C Mostly clear
Search
ENG IN
21:44 24-04-2026


Projects
Ask Gemini
localhost:9000/projects
You're running a version of SonarQube that is no longer active. Please upgrade to an active version immediately. Learn More
sonarqube Projects Issues Rules Quality Profiles Quality Gates Administration
Search for projects...
My Favorites All
Search by project name or key
Create Project
Filters
1 project(s)
Perspective: Overall Status
Sort by: Name
Quality Gate
Passed 1
Failed 0
Reliability (Bugs)
A rating 0
B rating 0
C rating 0
D rating 1
E rating 0
Security (Vulnerabilities)
A rating 1
B rating 0
C rating 0
D rating 0
E rating 0
Security Review (Security Hotspots)
Sample Java App Passed
Last analysis: 2 minutes ago
Bugs 1 D
Vulnerabilities 0 A
Hotspots Reviewed 0.0% E
Code Smells 6 A
Coverage 0.0%
Duplications 0.0%
Lines 72 XS Java, XML
1 of 1 shown
Get the most out of SonarQube!
Take advantage of the whole ecosystem by using SonarLint, a free IDE plugin that helps you find and fix issues earlier in your workflow. Connect SonarLint to SonarQube to sync rule sets and issue states.
Learn More
Dismiss
SonarQube™ technology is power
Community Edition - v9.9.8 (build 100196) NO LONGER ACTIVE - LGP
27°C Mostly clear
Search
ENG IN
21:45 24-04-2026





Key Concepts Covered

Concept	What Was Demonstrated
Quality Gate	Default “Sonar way” gate applied to project
Technical Debt	~2 hours estimated fix time shown in dashboard

Static Analysis	Maven plugin scanned code without running it

Concept	What Was Demonstrated
CI/CD Integration	Jenkinsfile blocks deployment on gate failure
Multi-issue Detection	Bugs, vulnerabilities, code smells all detected
Feedback Loop	Fixed bug → re-scanned → dashboard updated

🔍 Tool Comparison

Feature	Jenkins	Ansible	Chef	SonarQube
Primary Purpose	CI/CD Automation	Config Management	Config Management	Code Quality
Architecture	Master-Agent	Agentless	Client-Server	Client-Server
Language	Groovy	YAML	Ruby	Java
Learning Curve	Moderate	Low	High	Low
Idempotency	No	Yes	Yes	N/A

🧹 Cleanup

```
docker compose down -v
```

This removes all containers and volumes.

🔗 References

- SonarQube Official Documentation
- SonarQube Docker Hub
- Maven Sonar Plugin
- SonarQube Quality Gates

End of Lab Report

Experiment 11

Name: Ashmeet Negi

Course: Containerization and DevOps Lab

Experiment 11 – Orchestration using Docker Compose & Docker Swarm

Objective

To understand and implement container orchestration using Docker Swarm as a continuation of Experiment 6 (WordPress + MySQL using Docker Compose), and to demonstrate features like scaling, self-healing, and load balancing.

Prerequisites

- Docker installed with Swarm mode available
- `docker-compose.yml` from Experiment 6 (WordPress + MySQL setup)

Theory

Orchestration is the automatic management of containers across one or more hosts. Docker Swarm extends Docker Compose by adding:

Feature	Description
Scaling	Increase/decrease container replicas with one command

Self-Healing	Automatically restarts failed containers
Load Balancing	Distributes traffic across all replicas internally
Multi-Host	Can span containers across multiple machines

Progression Path:

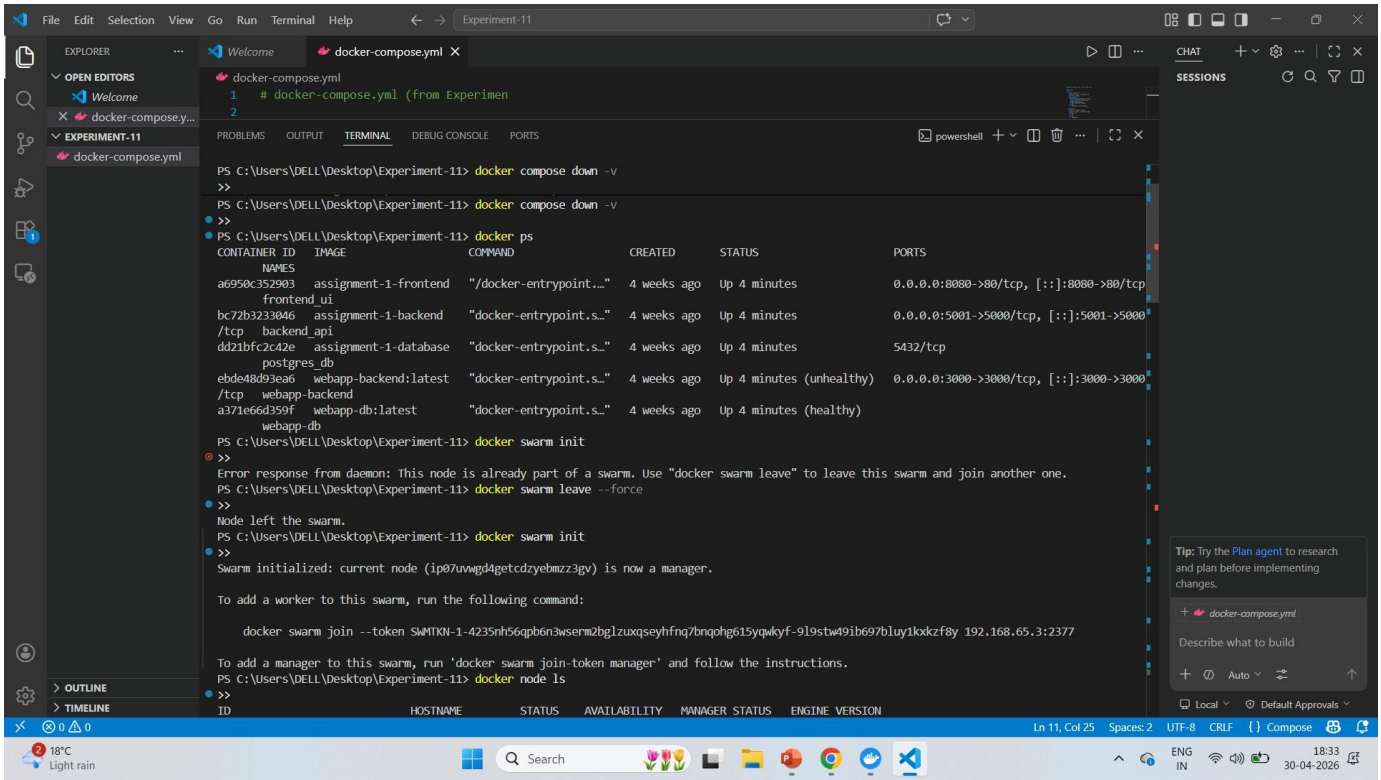
```
docker run → Docker Compose → Docker Swarm → Kubernetes
```

Procedure

Task 1 – Initialize Docker Swarm

```
docker swarm init
docker node ls
```

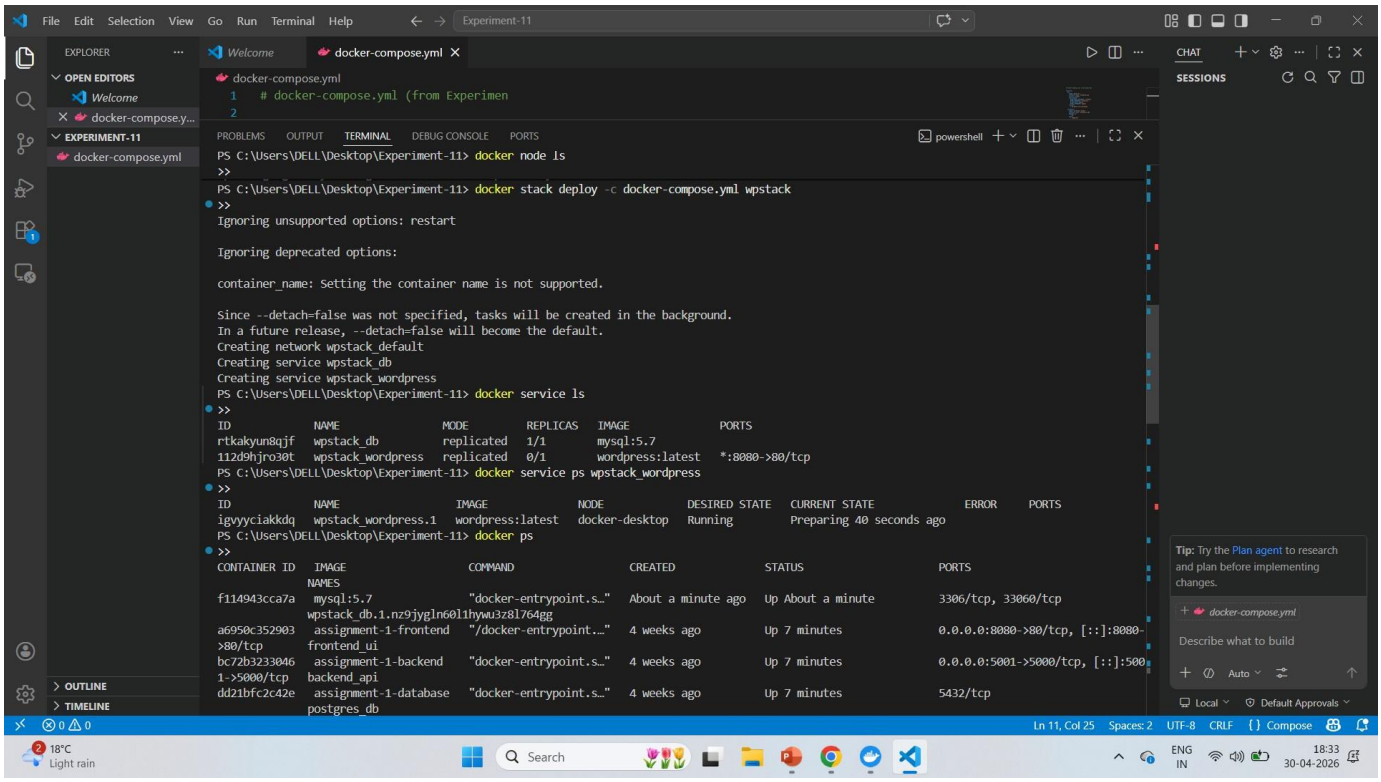
The `docker swarm init` command enables Swarm mode and makes the current machine a **manager node**.



Observation: Node status shows `Ready` , `Active` , and `Leader` — confirming Swarm is initialized. The stack deploy command created `wpstack_default` network, `wpstack_db` , and `wpstack_wordpress` services.

Task 2 – Deploy Stack & Verify Services

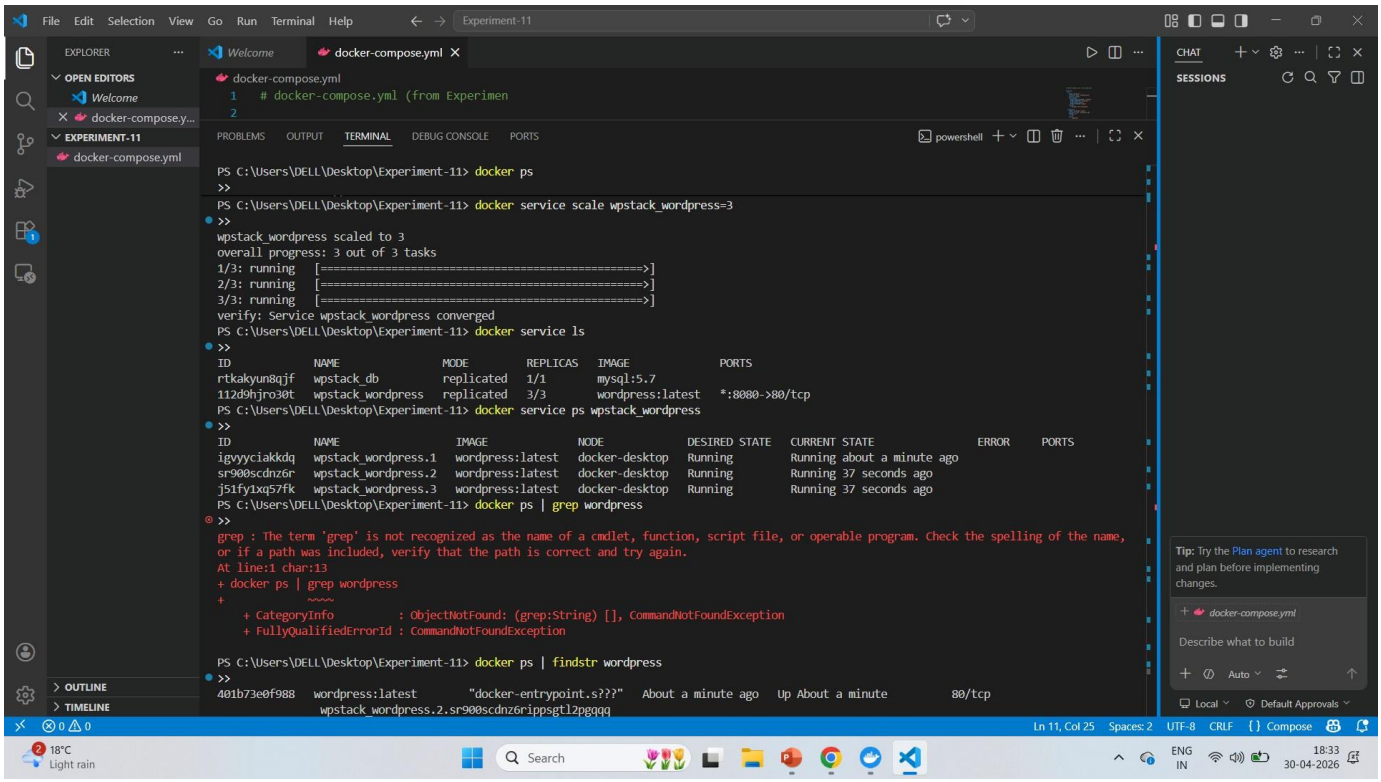
```
docker stack deploy -c docker-compose.yml wpstack docker
service ls
docker ps
```



Observation:

- `wpstack_db` — replicated, 1/1 replica running (`mysql:5.7`)
- `wpstack_wordpress` — replicated, 1/1 replica running (`wordpress:latest`) on port `*:8080->80/tcp`
- Containers are now named with the pattern `<stack>_<service>.<replica>.<id>` , managed by Swarm

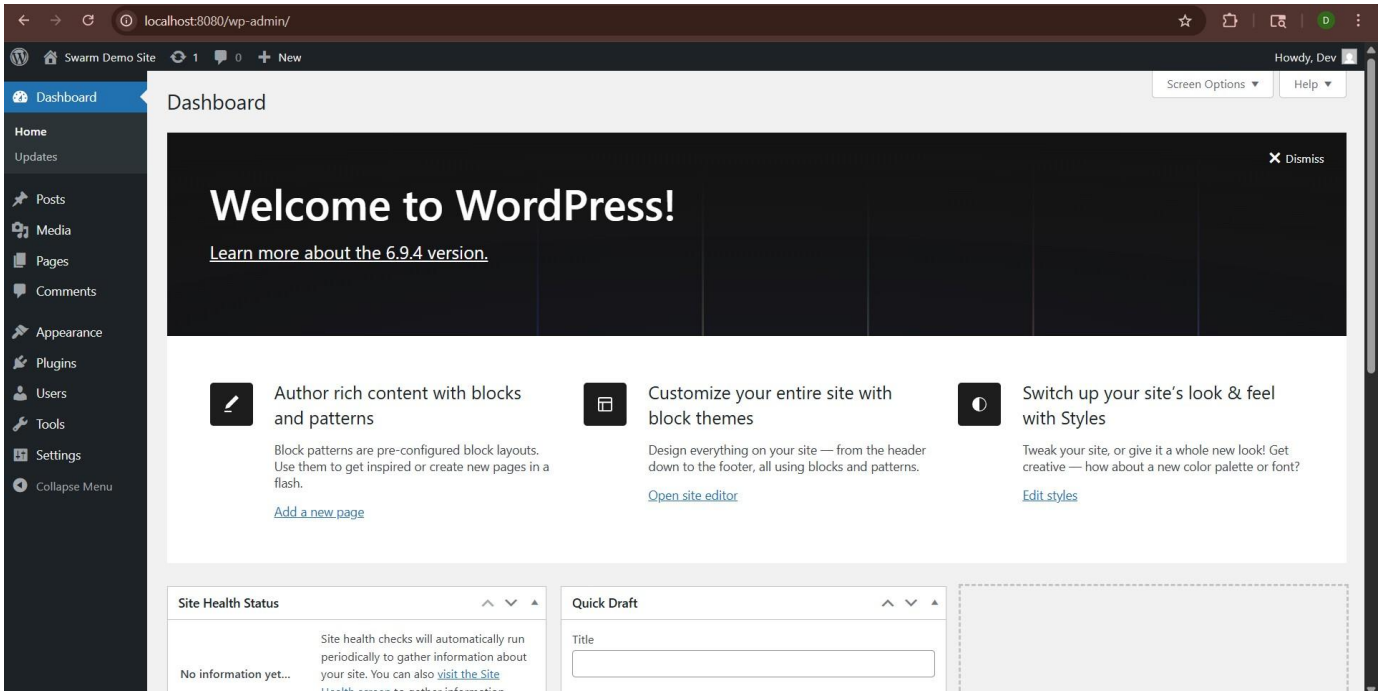
Task 3 – Access WordPress via Browser



Observation: WordPress login page is accessible, confirming the stack is running correctly under Swarm management.

Task 3 – WordPress Admin Dashboard

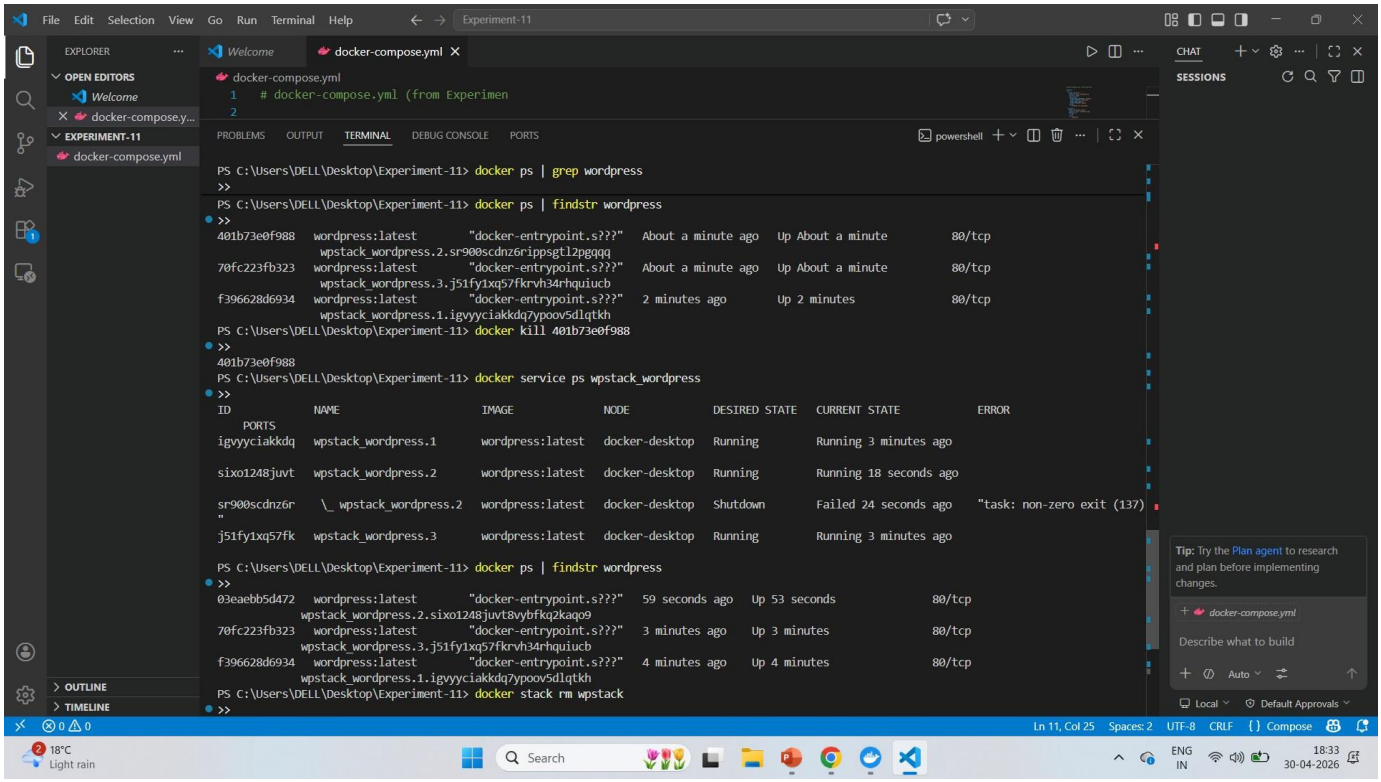
Logged in to the WordPress admin panel at `http://localhost:8080/wp-admin/`



Observation: WordPress 6.9.4 dashboard is fully functional. The site is titled **"Swarm Demo Site"**, confirming the WordPress + MySQL stack is working end-to-end under Swarm.

Task 5 – Scale the WordPress Service

```
docker service scale wpstack_wordpress=3
docker service ls docker ps
```

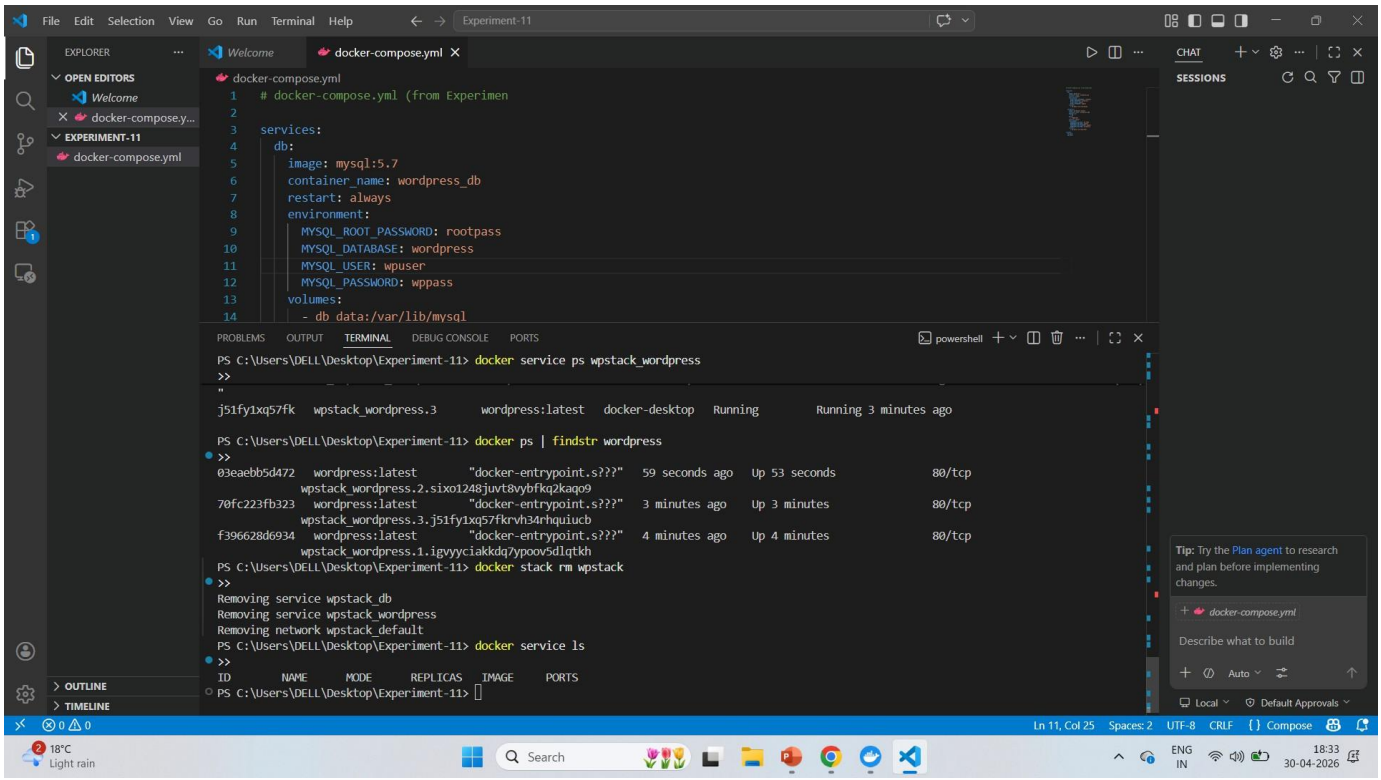


Observation:

- Scaling progressed: 1/3 → 2/3 → 3/3 tasks running `docker service ls`
- shows `REPLICAS: 3/3` for `wpstack_wordpress` `docker ps` confirms **3**
- **WordPress containers** running simultaneously:
  - `wpstack_wordpress.1 , .2 , .3`
- All 3 share port `8080` via Swarm's **internal load balancer** — no port conflicts

Task 6 – Test Self-Healing

```
docker ps | grep wordpress docker
kill 5ed7a4e16c62 docker service ps
wpstack_wordpress docker ps | grep
wordpress
```



Observation:

- Container `5ed7a4e16c62` (`wpstack_wordpress.3`) was killed manually `docker service ps`
- shows the killed container as `Shutdown` / `Failed 21 seconds ago` with exit code `137`
- Swarm **automatically spawned a new container** (`wpstack_wordpress.3.iivz0kx1gzko`) within seconds
- Final `docker ps | grep wordpress` confirms **3 containers still running** — self-healing worked

After cleanup:

```
docker stack rm wpstack docker service ls   # Empty —
all services removed docker ps           # Empty —
all containers stopped
```

Result

All tasks were completed successfully:

Task	Command	Result
Initialize Swarm	`docker swarm init`	Node became manager/leader
Deploy Stack	`docker stack deploy`	2 services created (db + wordpress)

Task	Command	Result
Verify Services	`docker service ls`	Both services running at 1/1
Access App	Browser at `localhost:8080`	WordPress fully accessible
Scale Service	`docker service scale ...=3`	3/3 WordPress replicas running
Self-Healing	`docker kill <id>`	Failed container auto-replaced
Remove Stack	`docker stack rm wpstack`	All services and networks removed

Key Observations

1. Same Compose file works for both Compose and Swarm:

Command	Mode
`docker compose up -d`	Standard Compose (no orchestration)
`docker stack deploy -c docker-compose.yml`	Swarm (with orchestration)

2. **Services vs Containers:** In Swarm, you manage **services** (definitions), not individual containers.

Swarm handles container lifecycle internally.

3. **Port Conflict Resolution:** Scaling in plain Compose would fail due to port conflicts. Swarm's internal load balancer listens on port `8080` once and routes traffic to all replicas transparently.

4. **Self-Healing:** When a container is killed (exit 137), Swarm detects the replica count dropped below desired state and automatically creates a replacement — no manual intervention needed.

Docker Compose vs Docker Swarm

Feature	Docker Compose	Docker Swarm
Scope	Single host	Multi-node cluster
Scaling	Basic (`--scale`), no load balancing	Built-in (`service scale`)
Load Balancing	No	Yes (internal VIP)
Self-Healing	No	Yes (automatic)
Rolling Updates	No	Yes

Use Case	Development / Testing	Production clusters

Quick Reference

```
docker swarm init                                          # Initialize Swarm
docker stack deploy -c docker-compose.yml <stack-name>  # Deploy stack docker
service ls                                               # List services docker
service scale <stack_service>=<n>                        # Scale service docker
service ps <service-name>                                # See service tasks docker
stack rm <stack-name>                                    # Remove stack docker swarm
leave --force                                          # Leave Swarm
```

Experiment 12

Name: Ashmeet Negi

Course: Containerization and DevOps Lab

Experiment 12 – Container Orchestration using Kubernetes

Objective

To study and analyse container orchestration using Kubernetes — deploying a WordPress application, exposing it via a Service, scaling pods, and demonstrating self-healing using `kubectl` on a k3d cluster.

Prerequisites

- Docker installed
- `k3d` and `kubectl` installed
- A k3d cluster already created (`mycluster`)

Theory

Why Kubernetes over Docker Swarm?

Reason	Explanation
Industry Standard	Most companies use Kubernetes in production

Powerful Scheduling	Automatically decides where to run containers
Large Ecosystem	Monitoring, logging, auto-scaling tools available
Cloud-Native	Works on AWS, GCP, Azure natively

Core Kubernetes Concepts

Docker Concept	Kubernetes Equivalent	Meaning
Container	**Pod**	Smallest deployable unit; one or more containers
Compose service	**Deployment**	Defines how to run pods (image, replicas)
Load balancing	**Service**	Exposes pods with a stable IP/port
Scaling	**ReplicaSet**	Ensures desired number of pod copies always run

YAML Configuration Files

`wordpress-deployment.yaml`

```
apiVersion: apps/v1
kind: Deployment
metadata:   name:
wordpress spec:
replicas: 2
selector:
matchLabels:
app: wordpress
template:
metadata:
labels:
        app: wordpress
spec:       containers:       -
name: wordpress         image:
wordpress:latest         ports:
        - containerPort: 80
```

`wordpress-service.yaml`

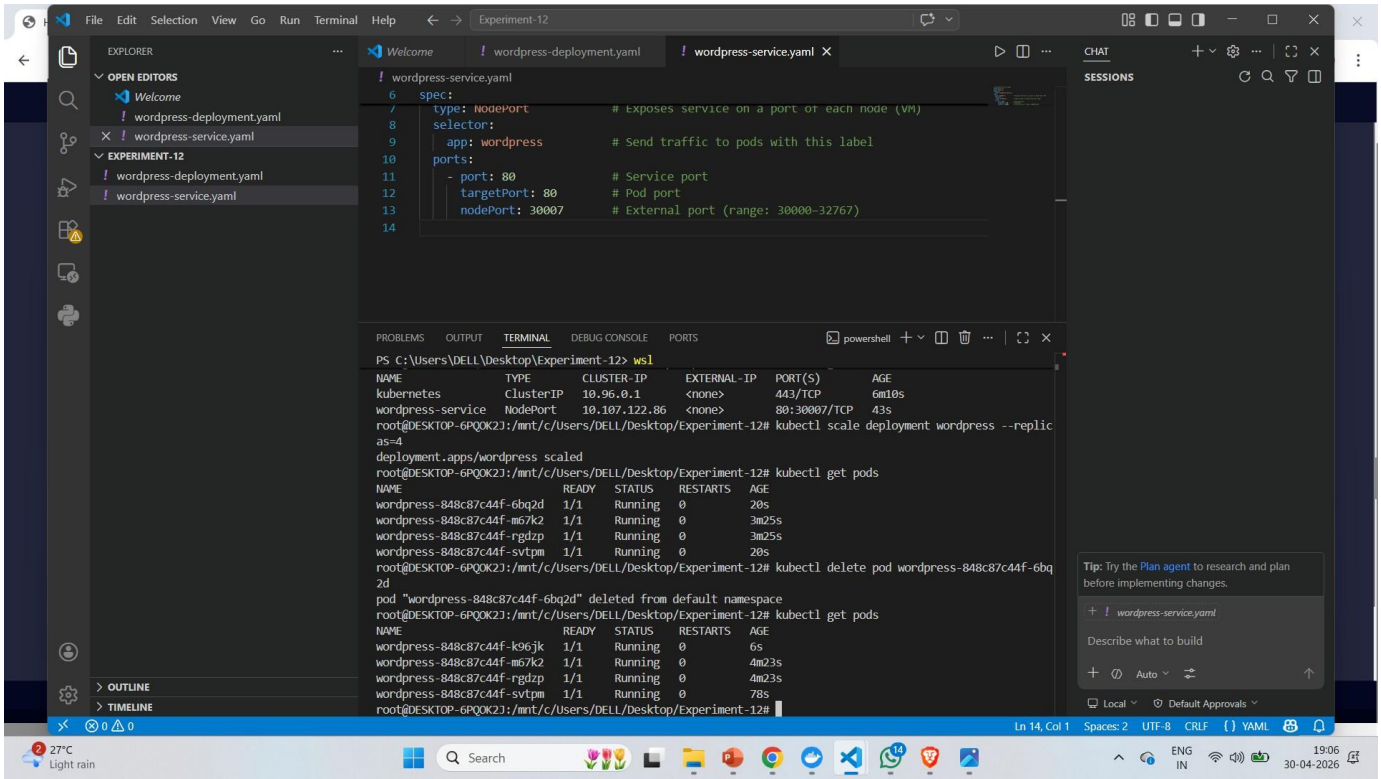
```
apiVersion: v1
kind: Service
metadata:
```

```
  name: wordpress-service
spec:
  type: NodePort
selector:
    app: wordpress
ports:     - port: 80
targetPort: 80
nodePort: 30007
```

Procedure

Task 1 – Start Cluster & Create Deployment

```
k3d cluster list k3d
cluster start mycluster
kubectl get nodes
kubectl apply -f wordpress-deployment.yaml
```



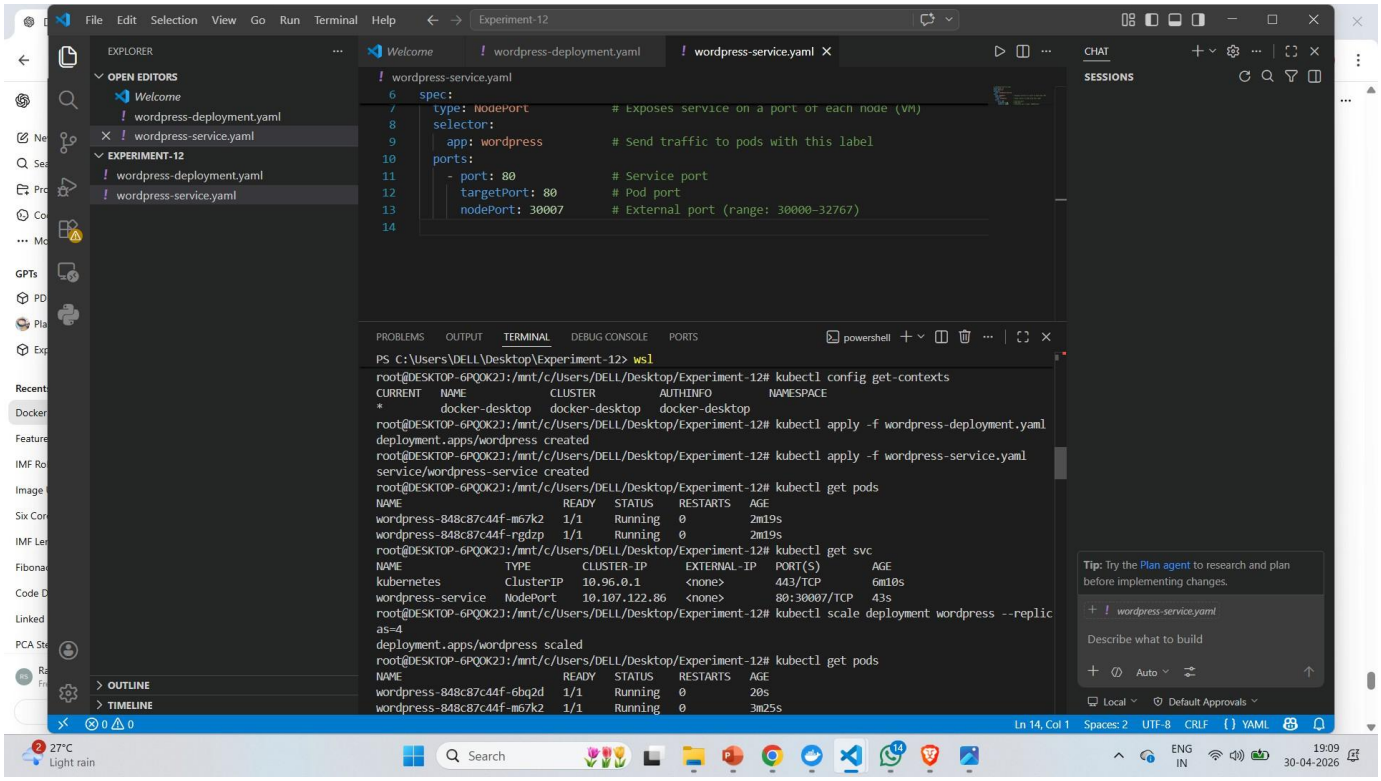
Observation:

- `k3d cluster list` shows `mycluster` with load balancer enabled
- After `k3d cluster start mycluster` , the cluster boots with tools node, server node, and load balancer
    - `kubectl get nodes` confirms node `k3d-mycluster-server-0` is `Ready` with role `control-`

plane,master running Kubernetes v1.31.5+k3s1 ● kubectl apply -f wordpress-deployment.yaml → deployment.apps/wordpress created

Task 2 – Verify Pods, Apply Service & Port-Forward

```
kubectl get pods
kubectl apply -f wordpress-service.yaml
kubectl get svc kubectl get svc
wordpress-service
kubectl port-forward service/wordpress-service 8080:80
```

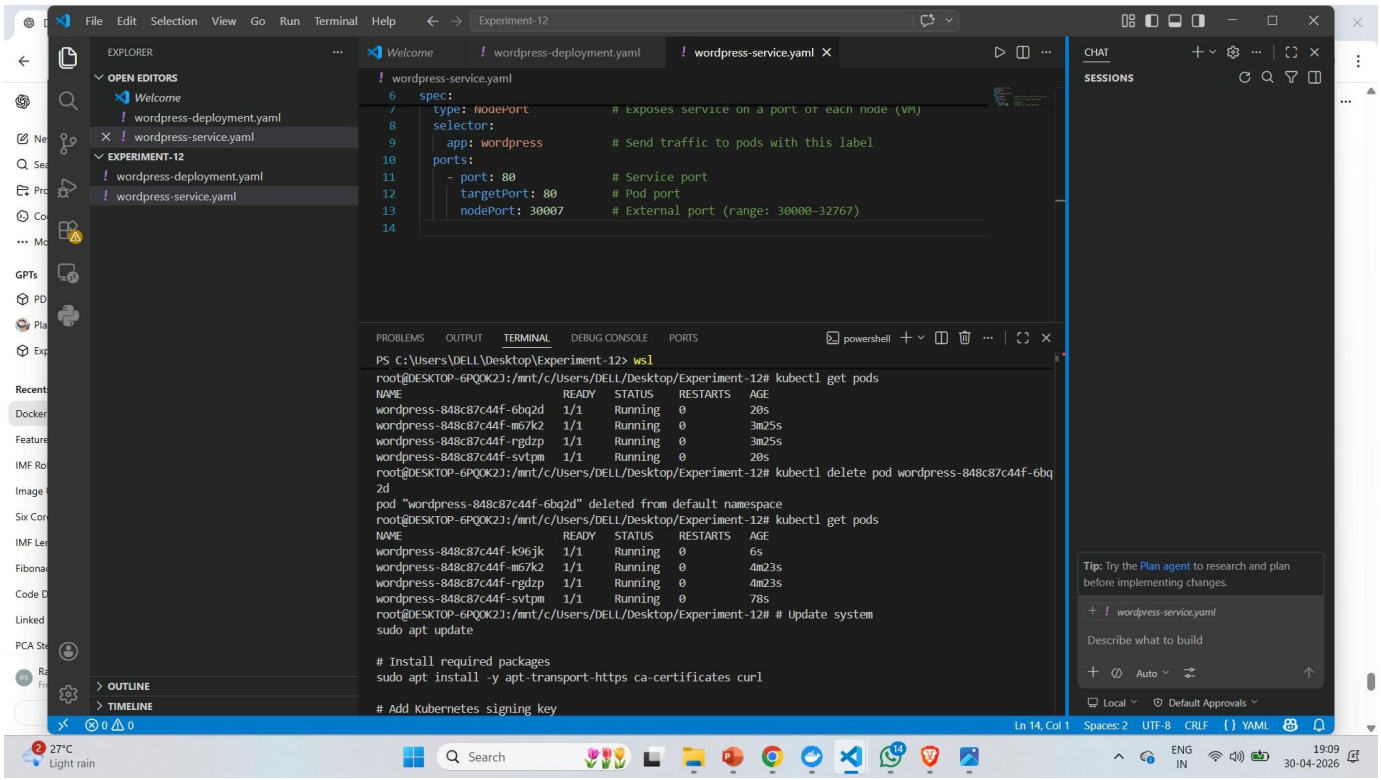


Observation:

- `kubectl get pods` shows **2 WordPress pods** running (`wordpress-7d6f6db8d8-cl77g` and `wordpress-7d6f6db8d8-vzcvn`), both `1/1 Running`
- Service `wordpress-service` created as `NodePort` on `10.43.122.223` , port `80:30007/TCP`
- Port-forward active: `127.0.0.1:8080 → 80` , connections handled successfully on port `8080`

Task 3 – Access WordPress in Browser

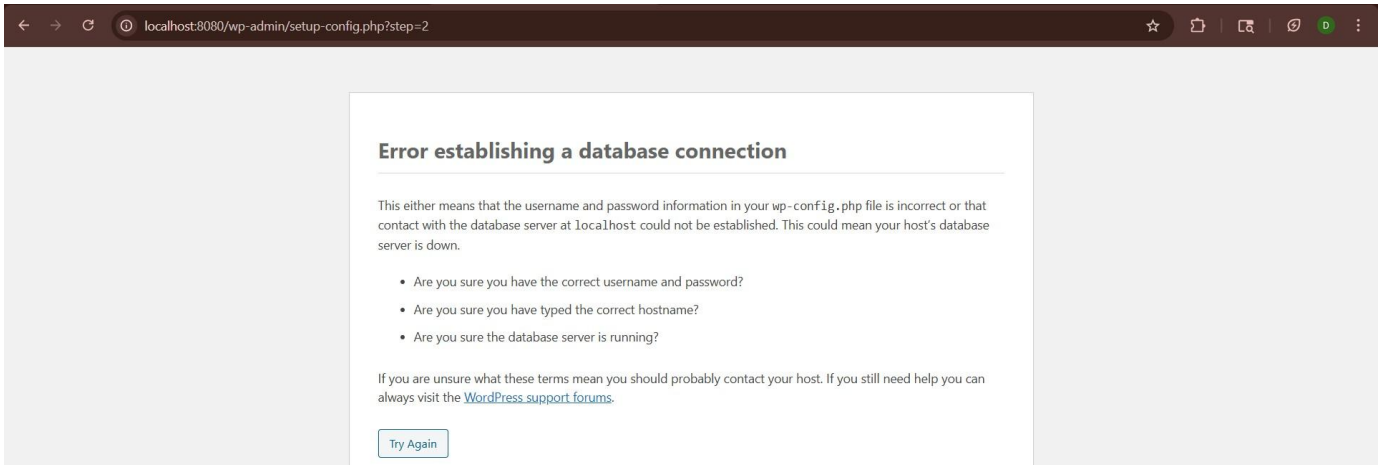
Opened browser at `http://localhost:8080`



Observation: WordPress database setup page is accessible at `localhost:8080` , confirming the deployment and service are working correctly. The page prompts for database connection details.

Task 4 – Database Connection Error (Expected)

Submitted the database form with `localhost` as Database Host.



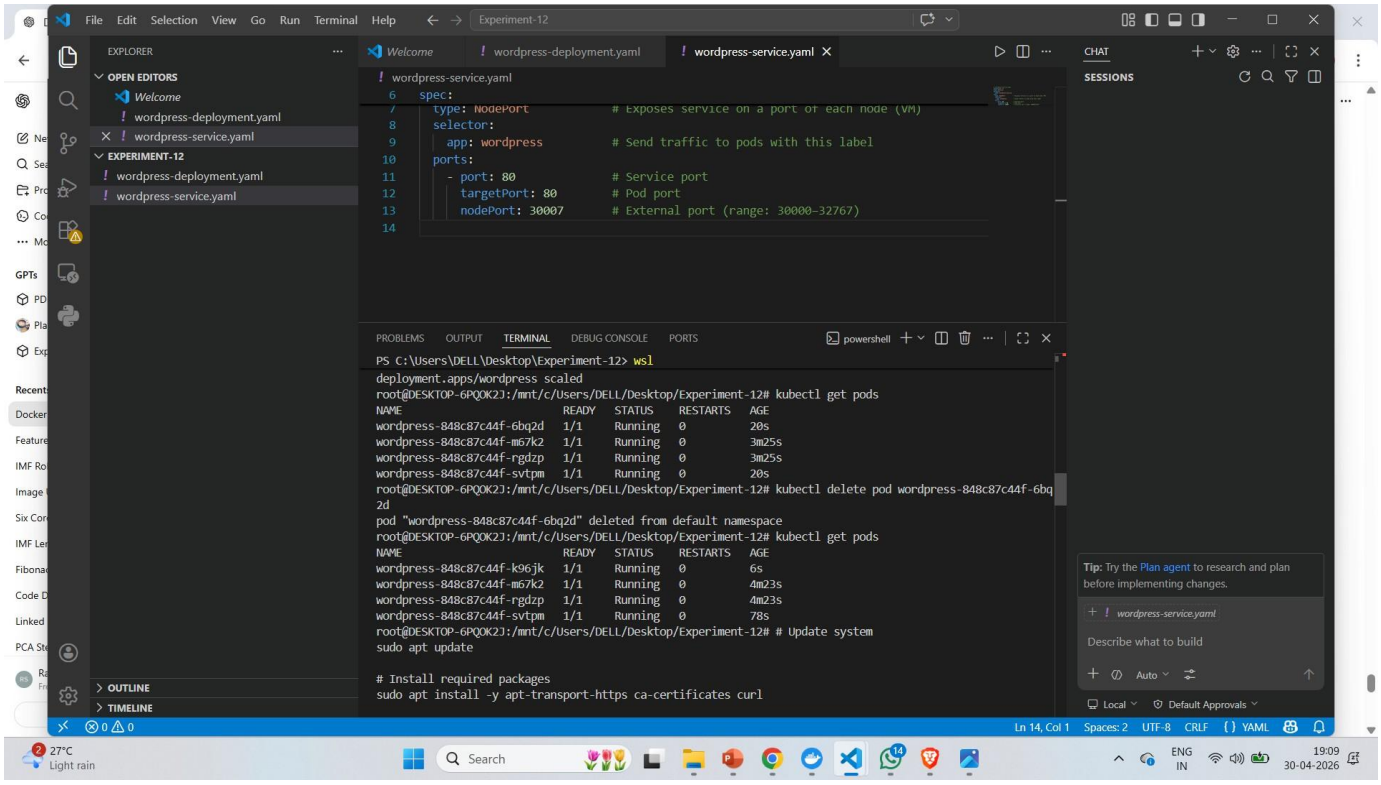
Observation: WordPress throws **"Error establishing a database connection"**.

Reason for the Error

No MySQL database pod or service was deployed in this experiment. WordPress requires a running MySQL instance to connect to, but since only the WordPress deployment was created, the connection to `localhost` fails — resulting in this error.

Task 5 – Scale the Deployment

```
kubectl scale deployment wordpress --replicas=4 kubectl
get pods
```

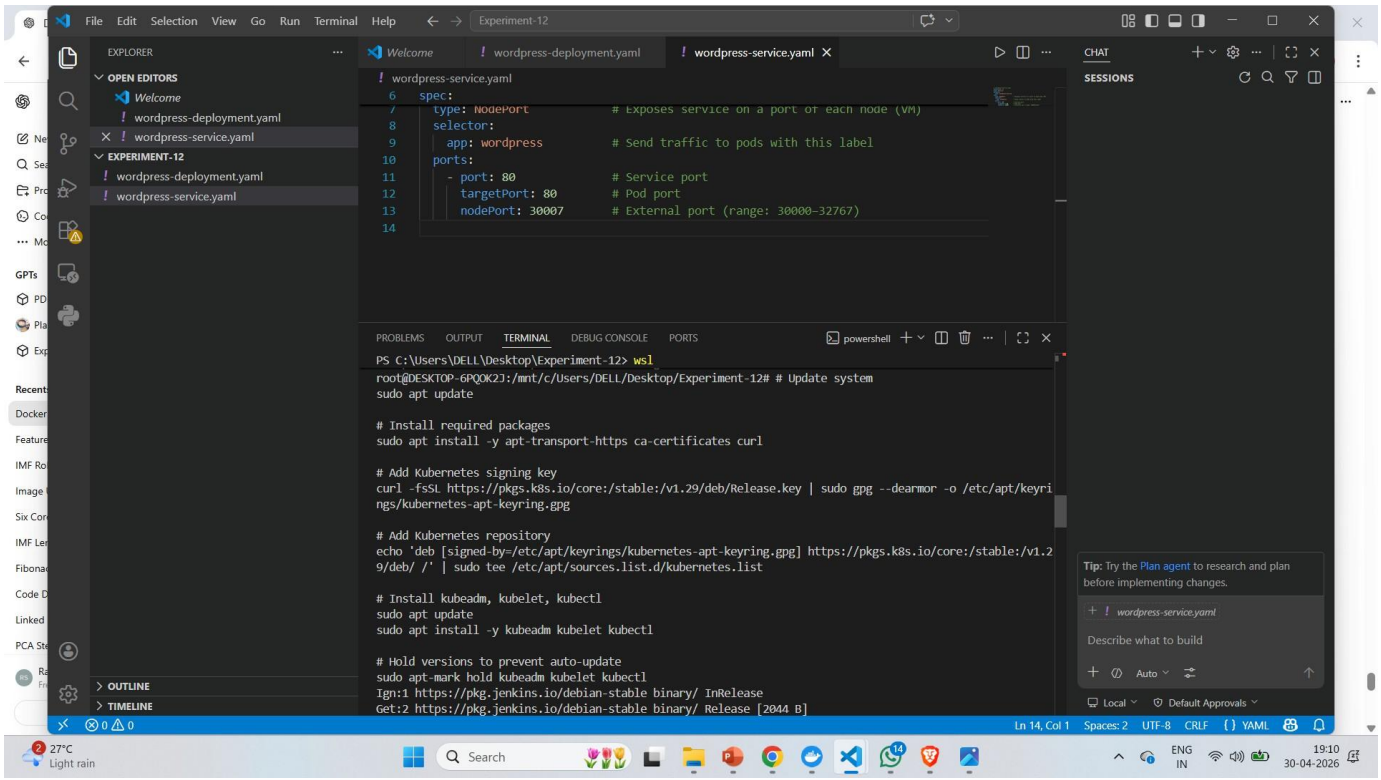


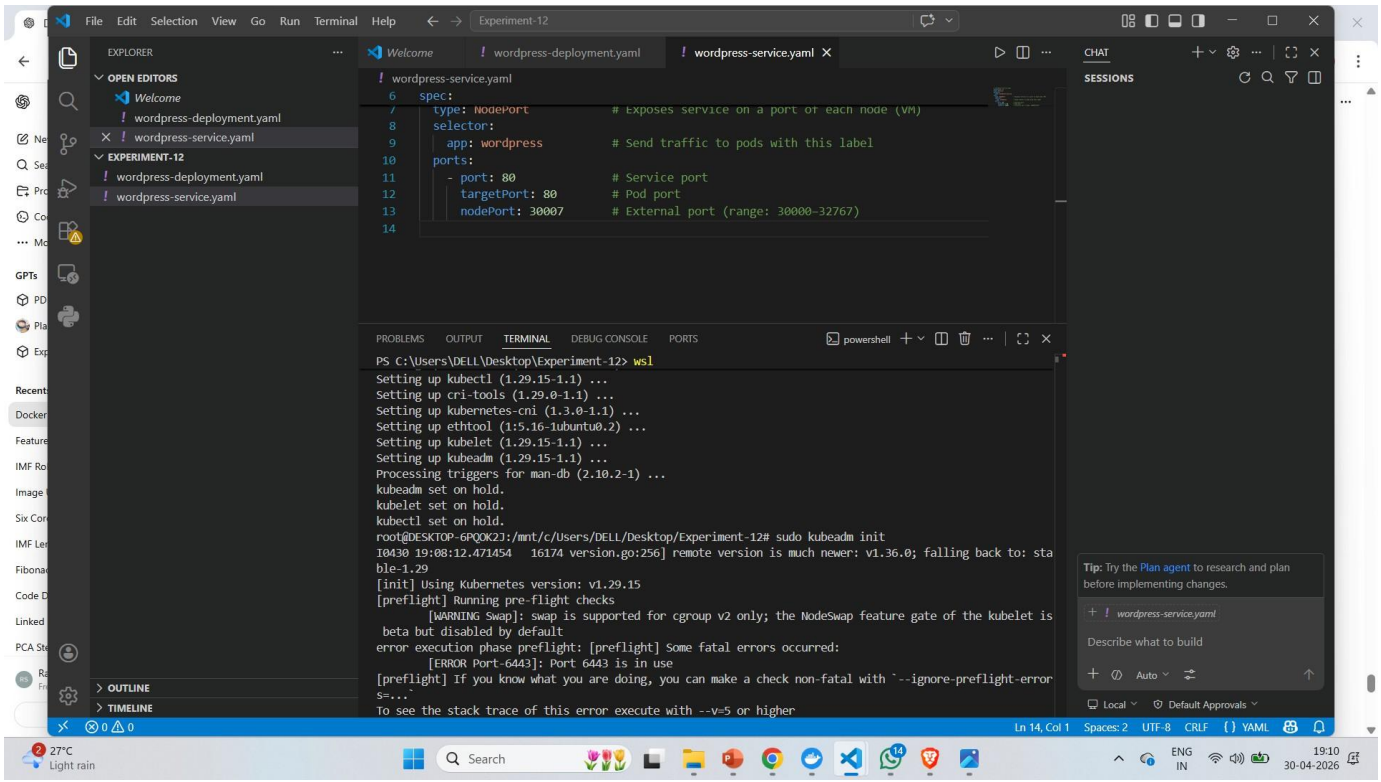
Observation:

- `deployment.apps/wordpress scaled` confirms the command executed successfully
- `kubectl get pods` shows **4 WordPress pods** all in `1/1 Running` state:
  - `wordpress-7d6f6db8d8-cl77g` — 19m (original) `wordpress-7d6f6db8d8-vzcvn`
  - — 19m (original) `wordpress-7d6f6db8d8-tx8dv` — 14s (newly created)
  - `wordpress-7d6f6db8d8-zkzcq` — 14s (newly created)
  - Kubernetes scaled from 2 → 4 replicas instantly

Task 6 – Self-Healing Demonstration

```
kubectl get pods kubectl delete pod wordpress-
7d6f6db8d8-zkzcq kubectl get pods
```



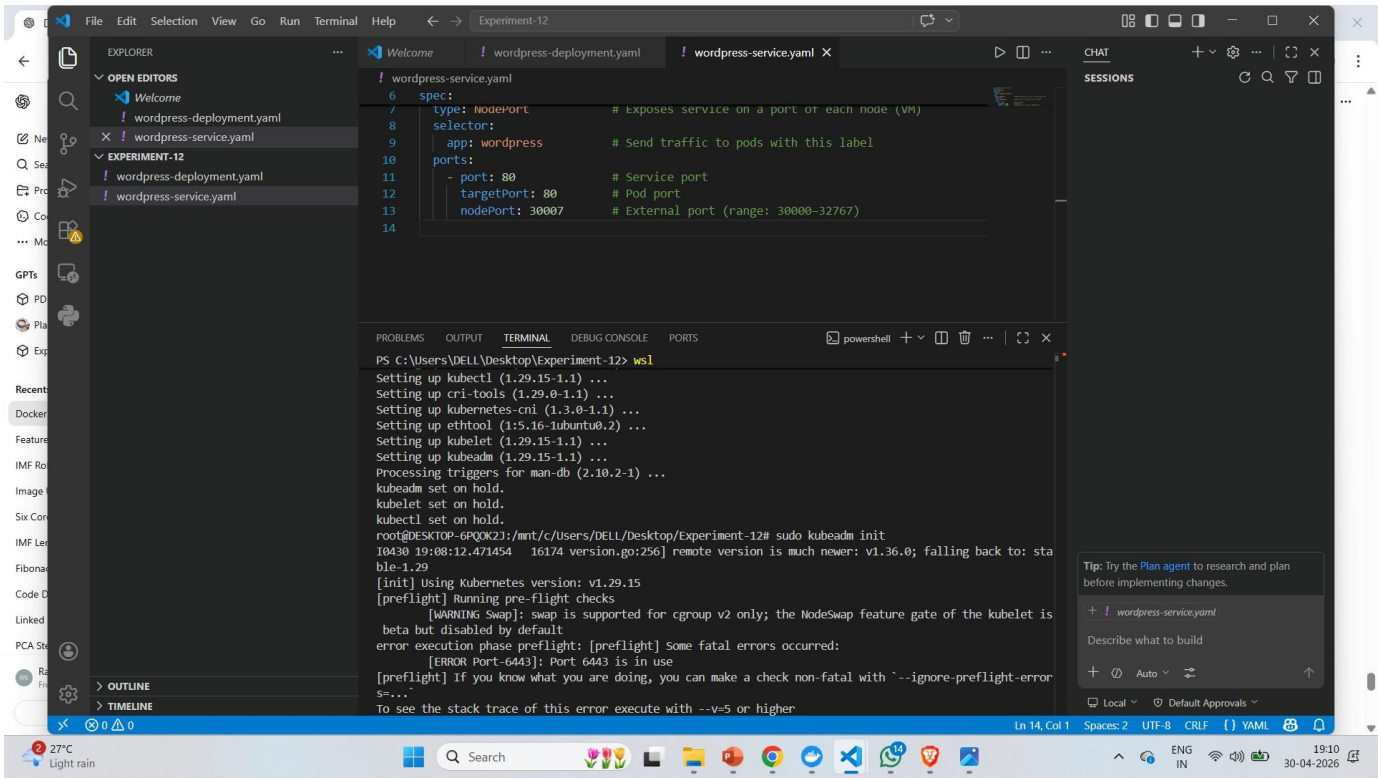


Observation:

- Pod `wordpress-7d6f6db8d8-zkzcq` was manually deleted
- Kubernetes **immediately detected** the replica count dropped below 4
- A new pod `wordpress-7d6f6db8d8-tw4zl` was **automatically created** (AGE: 11s)
- Final `kubectl get pods` still shows **4 running pods** — self-healing confirmed

Task 7 – Cleanup

```
kubectl delete -f wordpress-service.yaml
kubectl delete -f wordpress-deployment.yaml
kubectl get pods kubectl get svc
```



Observation:

- `service "wordpress-service" deleted from default namespace deployment.apps`
- `"wordpress" deleted from default namespace`
- `kubectl get pods` — only the pre-existing `apache` pod remains; all WordPress pods removed • `kubectl get svc` — `wordpress-service` is gone; only `apache` , `kubernetes` , and `web`

  services remain

Result

All tasks completed successfully:

Task	Command	Result
Start Cluster	`k3d cluster start mycluster`	Cluster ready, node `Ready`
Create Deployment	`kubectl apply -f wordpressdeployment.yaml`	2 WordPress pods running

Task	Command	Result
Expose Service	`kubectl apply -f wordpressservice.yaml`	NodePort service on port 30007
Access App	Browser at `localhost:8080`	WordPress setup page loaded
DB Error (Expected)	—	Confirms multi-tier architecture needed
Scale	`kubectl scale deployment wordpress --replicas=4`	4/4 pods running
Self-Healing	`kubectl delete pod <name>`	Pod auto-replaced, count stays at 4
Cleanup	`kubectl delete -f`	All resources removed

Docker Swarm vs Kubernetes

Feature	Docker Swarm	Kubernetes
Setup	Very easy	More complex
Scaling	Basic	Advanced (supports auto-scaling)
Ecosystem	Small	Huge (monitoring, logging, service mesh)
Industry Use	Rare	Industry standard
Cloud Support	Limited	Native on AWS, GCP, Azure

Quick Reference

```
kubectl apply -f <file.yaml>                          # Create resource from YAML
kubectl get pods                                      # List all pods kubectl get svc
# List all services kubectl get nodes                                    # List
cluster nodes kubectl scale deployment <name> --replicas=N          # Scale a
deployment kubectl delete pod <pod-name>                           # Delete a specific
pod kubectl delete -f <file.yaml>                        # Delete resource from YAML
kubectl port-forward service/<svc-name> 8080:80       # Forward local port to service
```